

spintent

SPANISH PARSE INTENTS

V0.95 2026-08-11^{*}

©2026 by Pablo González L[†]

CTAN: <https://www.ctan.org/pkg/spintent>

 <https://github.com/pablgonz/spintent>

Resumen

El paquete **spintent** proporciona una serie de utilidades para docentes de educación primaria y secundaria que necesiten crear documentos PDF *accesibles* (*tagged* PDF) en español usando Lua $\text{\LaTeX}$.

Índice

1	Descripción	1	3.3	El comando <code>\spmoney</code>	6
1.1	Carga del paquete	1	3.3.1	Llaves para <code>\spmoney</code>	7
1.2	El paquete <code>babel</code>	2	3.3.2	Monedas y alias para <code>\spmoney</code>	7
1.3	Fuentes tipográficas	2	3.4	El comando <code>\spshort</code>	7
1.4	El paquete <code>hyperref</code>	2	3.4.1	Llaves para <code>\spshort</code>	9
2	El comando <code>\setspintent</code>	3	3.4.2	Argumentos para <code>\spshort</code>	9
3	The big four	3	4	Utilidades generales	11
3.1	El comando <code>\spnum</code>	3	5	Utilidades para símbolos	11
3.1.1	Llaves para <code>\spnum</code>	3	6	Utilidades para números	12
3.2	El comando <code>\spunit</code>	4	7	Utilidades para geometría	16
3.2.1	Llaves para <code>\spunit</code>	4	8	Referencias	17
3.2.2	Argumentos para <code>\spunit</code>	4	9	Change history	18
3.2.3	El comando <code>\spnewunit</code>	6	10	Índice de la Documentación	19
3.2.4	El comando <code>\spunitalias</code>	6	11	Implementation	21
			12	Índice de la Implementación	74

Motivación y agradecimientos

Desde que desarrollé `scontents`[7] en 2019 y, más tarde, `enumext`[8] en 2024, me he ido interesando cada vez más por la accesibilidad en los documentos PDF. Al principio era un tema prácticamente desconocido para mí, pero a medida que fui aprendiendo sobre él comprendí la importancia que tiene, especialmente en el ámbito educativo.

El equipo del proyecto de $\text{\LaTeX}$ [19] ha realizado un trabajo extraordinario en este campo. Hoy es posible generar documentos etiquetados, validarlos con herramientas como `veraPDF`[20] o inspeccionar su estructura con `ngpdf`[21]. Sin embargo, siempre me quedó una duda: cuando un estudiante utiliza `NVDA`[11] junto con `MathCAT`[12], ¿realmente escucha lo que el o la docente quiso expresar?

Basta con generar un documento accesible siguiendo las recomendaciones de `The LaTeX Tagged PDF Project` y escucharlo con `NVDA/MathCAT` para darse cuenta de que, en muchos casos, la *verbalización* no coincide con lo que uno espera. Esto no resulta extraño: las reglas del español, y en particular el lenguaje matemático, dependen en gran medida del contexto en que se utilizan.

El objetivo de **spintent** nace precisamente de esta necesidad: proporcionar herramientas para que el o la docente pueda indicar explícitamente el contexto de determinadas expresiones matemáticas desde el propio código fuente de $\text{\LaTeX}$, de modo que el documento PDF resultante no solo sea *accesible*, sino que también produzca una *verbalización más natural* y adecuada para el estudiante al utilizar `NVDA/MathCAT`.

El nuevo estándar PDF 2.0[14, 18, 16], las especificaciones de WTPDF[15] y la guía más reciente de la PDF Association, «Best Practice Guide: Math in PDF»[17], coinciden en que el camino hacia una accesibilidad matemática de calidad pasa por el uso de `MathML`. En ese contexto, Lua $\text{\LaTeX}$ ofrece una plataforma especialmente adecuada para incorporar esta tecnología.

^{*}Este archivo describe la documentación para la versión v0.95 revisada por última vez el 2026-08-11.

[†]E-mail: pablgonz@educarchile.cl.

Licencia y requerimientos

El paquete **spintent** carga y requiere el paquete **unicode-math**[1] y que se ejecute bajo Lua $\TeX$ por lo que es necesario contar con una distribución moderna de $\TeX$ como $\TeX$ Live 2026. Ha sido probado con las clases estándar proporcionadas por $\LaTeX$: **book**, **report**, **article** y **letter** en tamaños 10pt, 11pt y 12pt.

Se otorga permiso para copiar, distribuir y/o modificar este paquete bajo los términos de $\LaTeX$ Project Public License (LPPL), versión 1.3 o posterior (<https://www.latex-project.org/lppl.txt>). El paquete tiene la condición de mantenido al estilo «A Work in Progress»¹.

- El requerimiento mínimo es $\LaTeX$ release 2026-06-01.

Este paquete solo se puede utilizar solo con Lua $\TeX$ y *tagged* PDF activo. Está presente en $\TeX$ Live y Mi $\TeX$, para instalarlo, utilice el gestor de paquetes de su distribución. Si desea una instalación manual, descargue **spintent.zip** y descomprímalo, luego ejecute `luatex spintent.ins`, mueva todos los archivos a las ubicaciones correspondientes y luego ejecute `mktexlsr`. Para generar la documentación, ejecute `arara spintent.dtx`.

<code>spintent.sty</code>	>	<code>TDS:tex/lualatex/spintent/</code>
<code>spintent.lua</code>	>	<code>TDS:tex/lualatex/spintent/</code>
<code>spintent.pdf</code>	>	<code>TDS:doc/lualatex/spintent/</code>
<code>README.md</code>	>	<code>TDS:doc/lualatex/spintent/</code>
<code>spintent.dtx</code>	>	<code>TDS:source/lualatex/spintent/</code>
<code>spintent.ins</code>	>	<code>TDS:source/lualatex/spintent/</code>

1 Descripción

El nombre **spintent** puede entenderse como «*Spanish Parse Intent*» o también como «*School Parse Intent*». Ambas interpretaciones reflejan el objetivo del paquete y el público para el que fue diseñado: docentes de educación primaria y secundaria.

Está pensado para crear «*apuntes*», «*hojas de ejercicios*», «*clases escritas*» enfocadas en el estudiante. Solo un docente sabe cual es el nivel de aprendizaje del estudiante. El paquete intenta sobrescribir muchas de las reglas dictadas por la heurística de **NVDA/MathCAT** teniendo esto presente.

- El paquete **spintent** usa y abusa del atributo `:intent` de **MathML** por medio del comando `\MathMLintent` definido en **latex-lab-mathintent**[30], un encapsulado de `\luamml_attribute`:^{een} proveída por **luamml**[2]. ¡Gracias Marcel!

1.1 Carga del paquete

El paquete **spintent** trabaja solo con Lua $\TeX$ y *tagged* PDF activo usando `tagging=on` dentro del comando `\DocumentMetadata` al inicio del archivo. Las llaves (*keys*) aceptadas por `\DocumentMetadata` y en general para el código de *tagged* PDF están documentadas en los paquetes **tagpdf**[29] **documentmetadata-support**[30], es deber del usuario revisar la documentación asociada a ellos.

```
\DocumentMetadata
{
  lang=es-CL,
  pdfversion=2.0,
  pdfstandard=ua-2,
  tagging=on,
  tagging-setup={math/setup={mathml-SE}},
}
\documentclass{article}
\usepackage[spanish]{babel}
\usepackage{spintent}
```

1.2 El paquete babel

El paquete **babel-spanish**[10] está marcado como «parcialmente compatible» para *tagged* PDF, pero, es absolutamente necesario para la creación de documentos en español. En caso de encontrar problemas de compatibilidad con otros paquetes se puede desactivar usando:

```
\usepackage[spanish,shorthands=off]{babel}
```

Muchas de las «shorthands» proveídas por **babel-spanish** hoy no son necesarias, más allá de `\sptext` para las abreviaturas y ordinales el resto son solo atajos de teclas para escritura que hoy en día no se ocupan. El paquete **spintent** provee el comando `\spshort` (§3.4) que cubre estos casos, de esta manera podemos cargar **babel**[9] con todas sus funcionalidades sin problemas.

¹A *Work in Progress* es un álbum en vivo de Neil Peart (* 1952–† 2020), baterista y principal letrista de Rush. El nombre no pretende formar parte de la terminología de la LPPL; simplemente describe el estado permanente de este paquete mientras sigo aprendiendo sobre *accesibilidad* en PDF.

1.3 Fuentes tipográficas

Como `spintent` es un paquete Lua $\TeX$ debemos distinguir entre las fuentes tipográficas OpenType/TrueType versus las fuentes de 8bit estándar usadas por $\TeX$ para el *modo texto* y el *modo matemático*. Para evitar problemas con *tagged* PDF preferiremos no combinar fuentes OpenType/TrueType con fuentes 8bit y trabajaremos solo con fuentes OpenType/TrueType que tengan soporte para matemáticas.

La carga de las fuentes tipográficas para Lua $\TeX$ se maneja por medio del paquete `fontspec`[3], si no se especifica ninguna se usarán las fuentes Latin Modern para el *modo texto* y Latin Modern Math para el *modo matemático*.

El tipo de fuente tipográfica escogida es preferencia de cada usuario, en lo personal utilizo las fuente Libertinus para el *modo texto* por medio del paquete `libertinus-otf`[4] y la fuente Erewhon-Math para el modo matemático cargada por el paquete `fourier-otf`[5]. Adicionalmente cargo la fuente Source Code Pro para el texto tipo «máquina de escribir» por medio del paquete `sourcecodepro`[6].

```
% \DocumentMetadata{...}
\documentclass{article}
\usepackage[spanish]{babel}
\usepackage[osf,nomath,mono=false]{libertinus-otf}
\usepackage[osf,semibold]{sourcecodepro}
\usepackage[no-text]{fourier-otf}
\usepackage{spintent}
```

1.4 El paquete hyperref

El paquete `hyperref`[23] es «casi obligatorio» cuando se crean documentos *tagged* PDF, se encarga de las referencias cruzadas `\label` y `\ref`, las notas al pie `\footnote`, los hipervínculos `\href` y muchos otros elementos como marcadores, índices y metadatos.

```
\DocumentMetadata
{
  lang=es-CL,
  pdfversion=2.0,
  pdfstandard=ua-2,
  tagging=on,
  tagging-setup={math/setup={mathml-SE}},
}
\documentclass{article}
\usepackage[spanish]{babel}
\usepackage[osf,nomath,mono=false]{libertinus-otf}
\usepackage[osf,semibold]{sourcecodepro}
\usepackage[no-text]{fourier-otf}
\setmathfont{Erewhon-Math}[range={cal,bfcal},StylisticSet=1]
\usepackage{spintent, hyperref}
\hypersetup
{
  colorlinks      = true,
  pdfstartview    = FitH,
  pdfdisplaydoctitle = true,
  pdftitle        = {Test para el paquete spintent},
  pdfinfo         =
  {
    Title   = {Test},
    Subject = {spintent},
  },
}
```

2 El comando \setspintent

`\setspintent` `\setspintent{⟨comando⟩}{⟨key-uno = valor, key-dos = valor, key-tres = valor, ...⟩}`

El comando `\setspintent` establece las llaves (*⟨keys⟩*) de forma global para los comandos provistos por `spintent`. Puede utilizarse tanto en el preámbulo como en el cuerpo del documento tantas veces como se desee. Las llaves (*⟨keys⟩*) definidas en el *argumento opcional* [*⟨key = val⟩*] de los comandos tienen prioridad sobre las establecidas por este, anulando las configuraciones globales de `\setspintent`.

Muchas de las utilidades que provee el paquete `spintent` implementan el sistema [*⟨key = val⟩*] y están pensadas para ser utilizadas en la generación de «*hojas de ejercicios*» o «*clases escritas*». Se recomienda usar `\setspintent` y establecerlas de manera global para cada documento, para mantener la coherencia en la verbalización de `NVDA/MathCAT` a lo largo del mismo.

El sistema [*⟨key = val⟩*] utilizado por `spintent` se implementa usando el módulo `l3keys` de `expl3`[24]. Por diseño, las llaves que reciben un *valor* después del signo '=' NO aceptan *valores vacíos* y es obligatorio pasar uno válido; las llaves documentadas como *⟨valor prohibido⟩* no reciben *ningun valor*. En caso de omitir un *valor* válido o pasar un *valor vacío* a una llave que acepte *valor* o pasar un *valor* a una llave del tipo *⟨valor prohibido⟩* se devolverá un "error" y se detendrá la compilación del documento.

3 The big four

El paquete `spintent` provee cuatro grandes comandos («*the big four*²»): `\spnum` para el manejo de números, `\spunit` para el manejo de unidades, `\spmoney` para el manejo de dinero y `\spshort` para el manejo de abreviaturas, acrónimos y números ordinales.

🔗 También se incluyen las utilidades `\spnewunit` y `\spunitalias` asociadas a al comando `\spunit`.

3.1 El comando \spnum

`\spnum` `\spnum{⟨número⟩}` `\spnum[⟨key = val⟩]{⟨número,número;número⟩}`
`\spnum[⟨key = val⟩]{⟨+número⟩}` `\spnum[⟨key = val⟩]{⟨número.número;número⟩}`
`\spnum[⟨key = val⟩]{⟨-número⟩}` `\spnum[⟨key = val⟩]{⟨número;número⟩}`
`\spnum[⟨key = val⟩]{⟨número,número⟩}` `\spnum[⟨key = val⟩]{⟨número e exponente⟩}`
`\spnum[⟨key = val⟩]{⟨número.número⟩}` `\spnum[⟨key = val⟩]{⟨número E exponente⟩}`

El comando `\spnum` trabaja en *modo matemático* y recibe el *argumento obligatorio* {*⟨número⟩*}, donde la «*parte entera*» puede iniciar con los signos '+' o '-', la «*parte decimal finita*» **debe** iniciar con una coma ',' o con un punto '.', y la «*parte decimal infinita (periódica)*» **debe** iniciar con un punto y coma '; '.

Si no se proporciona el *argumento opcional* [*⟨key = val⟩*] y se ejecuta `\spnum{+23}` se imprimirá '+23' en la salida PDF y `NVDA/MathCAT` verbalizará «*más veintitrés*», si se ejecuta `\spnum{3,14}` se imprimirá '3,14' en la salida PDF y `NVDA/MathCAT` verbalizará «*tres coma catorce*», si se ejecuta `\spnum{3,1;4}` se imprimirá '3,14' en la salida PDF y `NVDA/MathCAT` verbalizará «*tres coma uno con cuatro periódico*».

Además, el *argumento obligatorio* {*⟨número⟩*} admite al final de este un sufijo de «*notación científica*». Este se indica mediante las letras 'e' o 'E', acompañadas de los signos opcionales '+' o '-' seguido por los dígitos del exponente. Por ejemplo `\spnum{3,14e3}` imprimirá '3,14 · 10³' en la salida PDF y `NVDA/MathCAT` verbalizará «*tres coma catorce por diez al cubo*».

🔗 La coma ',' o el punto '.' son *separadores decimales* y NO de millares, por lo tanto solo pueden aparecer una «única vez» dentro del {*⟨argumento⟩*}. El {*⟨argumento⟩*} puede contener espacios matemáticos válidos, pero, NO puede contener comandos con o sin argumentos como `\textstyle`, `\mathrm{#1}` o `\mathbf{#1}` por ejemplo. Cualquier entrada fuera de esta sintaxis devolverá un "error" deteniendo la compilación del documento.

3.1.1 Llaves para \spnum

`cifras-sep = {⟨true | false⟩}`

default: *true*

Establece el *espaciado* utilizado para agrupar los dígitos en bloques de tres, tanto en la parte entera como en la parte decimal finita. Si se establece en *true* se siguen las normas de la RAE y se utiliza un *espacio fino* '\,' entre las cifras; si se establece en *false*, se respetan los espacios matemáticos introducidos en el argumento. Por ejemplo `\spnum{100000}` imprimirá '100 000' en la salida PDF y `\spnum[cifras-sep=false]{10\,00\,00}` imprimirá '100000' en la salida PDF.

`read-sign = {⟨terse | clear⟩}`

default: *terse*

Establece la *forma* en la que `NVDA/MathCAT` verbalizará los signos '+' o '-'. Si se establece en *terse*, `NVDA/MathCAT` verbalizará «*más*» o «*menos*» **antes** de leer el número; si se establece en *clear*, `NVDA/MathCAT` verbalizará «*positivo*» o «*negativo*» **después** de leer el número. Por ejemplo `\spnum{-23}` imprimirá '-23' en la salida PDF y `NVDA/MathCAT` verbalizará «*menos veintitrés*» y `\spnum[read-sign=clear]{-23}` imprimirá '-23' en la salida PDF y `NVDA/MathCAT` verbalizará «*veintitrés negativo*».

`read-period-sep = {⟨coma | punto⟩}`

default: *coma*

²Los «*cuatro grandes*» del Thrash Metal: Metallica, Megadeth, Slayer y Anthrax, que me acompañan mientras desarrollo `spintent`.

Establece la *forma* en la que NVDA/MathCAT verbalizará e imprimirá el signo de separación ‘;’ de la «*parte decimal infinita (periódica)*» cuando no se tiene la «*parte decimal finita*». Si se establece en *coma*, se imprimirá ‘,’ en la salida PDF y NVDA/MathCAT verbalizará «*coma*»; si se establece en *punto*, se imprimirá ‘.’ en la salida PDF y NVDA/MathCAT verbalizará «*punto*». Por ejemplo $\$ \backslash \text{spnum}\{3;4\}$ imprimirá ‘3,4’ en la salida PDF y NVDA/MathCAT verbalizará «*tres coma cuatro periódico*» y $\$ \backslash \text{spnum}[\text{read-period-sep=punto}]\{3;4\}$ imprimirá ‘3.4’ en la salida PDF y NVDA/MathCAT verbalizará «*tres punto cuatro periódico*»

- Esta llave es necesaria SOLO cuando se utilizan «*decimales periódicos puros*», es decir, cuando no se ha incluido en el $\{ \langle \text{argumento} \rangle \}$ una *coma* ‘,’ o un *punto* ‘.’ antes del signo de separación ‘;’ periódica.

notation-sym = $\{ \langle \text{times} \mid \text{cdot} \rangle \}$

default: *cdot*

Establece el *símbolo* que se utilizará para imprimir la «*notación científica*» al usar ‘e’ o ‘E’. Si se establece en *times* imprimirá ‘x’ en la salida PDF y NVDA/MathCAT verbalizará «*por*»; si se establece en *cdot* imprimirá ‘.’ en la salida PDF y NVDA/MathCAT verbalizará «*por*». Por ejemplo $\$ \backslash \text{spnum}\{3,14\text{e}3\}$ imprimirá ‘3,14 · 10³’ en la salida PDF y NVDA/MathCAT verbalizará «*tres coma catorce por diez al cubo*» y $\$ \backslash \text{spnum}[\text{notation-sym=times}]\{3,14\text{e}3\}$ imprimirá ‘3,14 × 10³’ en la salida PDF y NVDA/MathCAT verbalizará «*tres coma catorce por diez al cubo*».

- Decir «*notación científica*» aquí es un poco exagerado, en realidad es «*multiplicación por potencias de 10*», no se verifica si la entrada cumple o no con la condición de estar escrita en «*notación científica*».

3.2 El comando \spunit

$\backslash \text{spunit}$	$\backslash \text{spunit}\{ \langle \text{unidad} \rangle \}$	$\backslash \text{spunit}[\langle \text{key} = \text{val} \rangle]\{ \langle \text{número unidad} \cdot \text{unidad} \rangle \}$
	$\backslash \text{spunit}[\langle \text{key} = \text{val} \rangle]\{ \langle \text{unidad} \rangle \}$	$\backslash \text{spunit}[\langle \text{key} = \text{val} \rangle]\{ \langle \text{número unidad} \cdot \text{unidad} / \text{unidad} \rangle \}$
	$\backslash \text{spunit}[\langle \text{key} = \text{val} \rangle]\{ \langle \text{número unidad} \rangle \}$	$\backslash \text{spunit}[\langle \text{key} = \text{val} \rangle]\{ \langle \text{número unidad} \cdot \text{unidad} / \text{unidad} \cdot \text{unidad} \rangle \}$

El comando $\backslash \text{spunit}$ trabaja en *modo matemático* y recibe el *argumento obligatorio* $\{ \langle \text{unidad} \rangle \}$, el cual puede contener una «*parte numérica*» opcional la cual se procesa internamente por el comando $\backslash \text{spnum}$ al inicio del $\{ \langle \text{argumento} \rangle \}$ seguido de una $\{ \langle \text{unidad} \rangle \}$ o un $\{ \langle \text{alias} \rangle \}$ valido descritos en §3.2.2 o una $\{ \langle \text{unidad} \rangle \}$ creada por el comando $\backslash \text{spnewunit}$ (§3.2.3) o un $\{ \langle \text{alias} \rangle \}$ creado por el comando $\backslash \text{spunitalias}$ (§3.2.4).

Si no se proporciona el *argumento opcional* $[\langle \text{key} = \text{val} \rangle]$ y se ejecuta $\$ \backslash \text{spunit}\{23 \text{ m}\}$ se imprimirá ‘23m’ en la salida PDF y NVDA/MathCAT verbalizará «*veintitrés metros*».

La multiplicación entre las $\{ \langle \text{unidades} \rangle \}$ se realiza mediante un *asterisco* ‘*’ y las potencias de estas se indican con un *circunflejo seguido del exponente entre llaves* ‘^ $\{ \langle \text{exponente} \rangle \}$ ’. Por ejemplo si se ejecuta $\$ \backslash \text{spunit}\{23 \text{ m}^* \text{s}^{\wedge}\{2\}\}$ se imprimirá ‘23ms²’ en la salida PDF y NVDA/MathCAT verbalizará «*veintitrés metros segundos al cuadrado*».

Se pueden usar *fracciones lineales* con las $\{ \langle \text{unidades} \rangle \}$ utilizando una «*única barra*» ‘/’ para la separación entre las $\{ \langle \text{unidades} \rangle \}$ del numerador y el denominador, teniendo presente que el denominador NO puede contener números de ningún tipo. Por ejemplo si se ejecuta $\$ \backslash \text{spunit}\{23 \text{ m} / \text{s}^{\wedge}\{2\}\}$ se imprimirá ‘23m/s²’ en la salida PDF y NVDA/MathCAT verbalizará «*veintitrés metros por segundos al cuadrado*».

- Solo imprimiremos *fracciones lineales*, si se requiere otra forma fraccionaria para explicaciones y otros usos se debe usar el comando por separado. El $\{ \langle \text{argumento} \rangle \}$ puede contener espacios matemáticos validos, pero, NO puede contener comandos con o sin argumentos como $\backslash \text{textstyle}$, $\backslash \text{mathrm}\{ \langle \text{\#1} \rangle \}$ o $\backslash \text{mathbf}\{ \langle \text{\#1} \rangle \}$ por ejemplo, NO puede contener más de una «*única barra*» ‘/’ y NO puede contener números en el denominador. Cualquier entrada fuera de esta sintaxis devolverá un “error” deteniendo la compilación del documento.

3.2.1 Llaves para \spunit

El comando $\backslash \text{spunit}$ posee las $\langle \text{meta-keys} \rangle$ *cifras-sep*, *read-period-sep* y *notation-sym* pasadas al comando $\backslash \text{spnum}$ (§3.1.1).

print-cdot = $\{ \langle \text{true} \mid \text{false} \rangle \}$

default: *false*

Establece si se *imprime* $\backslash \text{cdot}$ ‘.’ o un *espacio fino* ‘\,’ para la multiplicación de $\{ \langle \text{unidades} \rangle \}$. Si se establece en *true*, se imprimirá ‘.’ entre las $\{ \langle \text{unidades} \rangle \}$ en la salida PDF; si se establece en *false* se imprimirá un *espacio fino* ‘\,’ entre las $\{ \langle \text{unidades} \rangle \}$ en la salida PDF.

read-cdot = $\{ \langle \text{true} \mid \text{false} \rangle \}$

default: *false*

Establece la *forma* en la que NVDA/MathCAT verbalizará la multiplicación de $\{ \langle \text{unidades} \rangle \}$. Si se establece en *true*, NVDA/MathCAT verbalizará «*por*» entre las $\{ \langle \text{unidades} \rangle \}$; si se establece en *false*, NVDA/MathCAT no verbalizará nada entre las $\{ \langle \text{unidades} \rangle \}$.

read-slash = $\{ \langle \text{por} \mid \text{partido-por} \rangle \}$

default: *por*

Establece la *forma* en la que NVDA/MathCAT verbalizará la «*barra*» ‘/’ de separación entre el numerador y el denominador. Si se establece en *por*, NVDA/MathCAT verbalizará «*por*»; si se establece en *partido-por*, NVDA/MathCAT verbalizará «*partido por*».

3.2.2 Argumentos para \spunit

Resumen de $\{ \langle \text{unidades} \rangle \}$ y $\{ \langle \text{alias} \rangle \}$ soportados por $\backslash \text{spunit}$.

Entrada	Alias soportados	Nombre unidad	Salida	NVDA/MathCAT
m	metro, metros	Metro	m	«metro»
g	gramo, gramos	Gramo	g	«gramo»
s	segundo, segundos	Segundo (Tiempo)	s	«segundo (Tiempo)»
l	litro, l-litro	Litro (minúscula)	l	«litro (minúscula)»
h	hora, horas	Hora	h	«hora»
d	dia, dias	Día	d	«día»
A	amperio, ampere	Amperio	A	«amperio»
V	voltio, volt	Voltio	V	«voltio»
W	vatio, watt	Vatio	W	«vatio»
J	julio, joule	Julio	J	«julio»
N	newton	Newton	N	«newton»
Pa	pascal	Pascal	Pa	«pascal»
Hz	hercio, hertz	Hercio	Hz	«hercio»
Ω	ohmio, ohm, Omega	Ohmio	Ω	«Ohmio»
F	faradio	Faradio	F	«Faradio»
C	culombio	Culombio	C	«Culombio»
K	kelvin, Kelvin	Kelvin	K	«Kelvin»
°C	celsius, degC, °C	Grados Celsius	°C	«Grados Celsius»
°F	fahrenheit, degF, °F	Grados Fahrenheit	°F	«Grados Fahrenheit»
cal	caloria, calorías	Caloría	cal	«Caloría»
b	bit	Bit	b	«Bit»
B	byte, bytes	Byte	B	«Byte»
in	pulgada, pulgadas	Pulgada	in	«Pulgada»
ft	pie, pies	Pie	ft	«Pie»
yd	yarda, yardas	Yarda	yd	«Yarda»
mi	milla, millas	Milla	mi	«Milla»
lb	libra, libras	Libra	lb	«Libra»
oz	onza, onzas	Onza	oz	«Onza»
gal	galon, galones	Galón	gal	«Galón»
Å	angstrom	Angstrom	Å	«Angstrom»
μ	micro	Micro (Prefijo)	μ	«Micro (Prefijo)»
Ω	mho	Mho / Siemens	Ω	«Mho / Siemens»
'	minmin	Minuto (Arco)	'	«Minuto (Arco)»
"	segseg	Segundo (Arco)	"	«Segundo (Arco)»
u	u-masa	Unidad masa atómica	u	«Unidad masa atómica»
Unidades directas (sin alias):				
L	-	Litro (Mayúscula)	L	«Litro (Mayúscula)»
ℓ	-	Litro (Cursiva)	ℓ	«Litro (Cursiva)»
w	-	Vatio (Minúscula)	w	«Vatio (Minúscula)»
°K	-	Grado Kelvin	°K	«Grado Kelvin»
Bq	-	Becquerel	Bq	«Becquerel»
Da	-	Dalton	Da	«Dalton»
Gy	-	Gray	Gy	«Gray»
S	-	Siemens	S	«Siemens»
Sv	-	Sievert	Sv	«Sievert»
T	-	Tesla	T	«Tesla»
Wb	-	Weber	Wb	«Weber»
atm	-	Atmósfera	atm	«Atmósfera»
au	-	Unidad astronómica	au	«Unidad astronómica»
bar	-	Bar	bar	«Bar»
cd	-	Candela	cd	«Candela»
dB	-	Decibelio	dB	«Decibelio»
dyn	-	Dina	dyn	«Dina»
eV	-	Electronvoltio	eV	«Electronvoltio»
erg	-	Ergio	erg	«Ergio»
ha	-	Hectárea	ha	«Hectárea»
kat	-	Katal	kat	«Katal»
kg	-	Kilogramo	kg	«Kilogramo»
km	-	Kilómetro	km	«Kilómetro»
lm	-	Lumen	lm	«Lumen»

Continúa en la siguiente página...

Continuación de la tabla 1

Entrada	Alias soportados	Nombre unidad	Salida	MathCAT
lx	-	Lux	lx	« <i>Lux</i> »
cm	-	Centímetro	cm	« <i>Centímetro</i> »
min	-	Minuto (Tiempo)	min	« <i>Minuto (Tiempo)</i> »
mm	-	Milímetro	mm	« <i>Milímetro</i> »
mol	-	Mol	mol	« <i>Mol</i> »
pc	-	Pársec	pc	« <i>Pársec</i> »
rad	-	Radián	rad	« <i>Radián</i> »
rpm	-	Rev. por minuto	rpm	« <i>Rev. por minuto</i> »
sr	-	Estereorradián	sr	« <i>Estereorradián</i> »
t	-	Tonelada	t	« <i>Tonelada</i> »
a	-	Año / Área (are)	a	« <i>Año / Área (are)</i> »
y	-	Año (Year)	y	« <i>Año (Year)</i> »
°	-	Grado (Arco)	°	« <i>Grado (Arco)</i> »

3.2.3 El comando \spnewunit

```
\spnewunit \spnewunit{\langle nueva unidad \rangle}{\langle verbalización NVDA \rangle}
```

El comando `\spnewunit` trabaja en *modo texto* y recibe dos *argumentos obligatorios*: $\langle \textit{nueva unidad} \rangle$ que establece lo que se imprimirá en la salida PDF y $\langle \textit{verbalización NVDA} \rangle$ que establece como NVDA/MathCAT verbalizará la $\langle \textit{nueva unidad} \rangle$.

La {*nueva unidad*} no debe estar registrada como {*unidad*} o {*alias*} para \spunit descritos en §3.2.2. Si la {*nueva unidad*} ya está registrada se devolverá un “error” y se detendrá la compilación del documento.

La declaración de la $\langle\textit{nueva unidad}\rangle$ es «global» y se almacena en el módulo `spintent.lua` como una $\langle\textit{unidad}\rangle$ válida para pasar como $\langle\textit{argumento}\rangle$ a `\spunit`. Por ejemplo: `\spnewunit{px}{píxeles}` creará la $\langle\textit{nueva unidad}\rangle$ ‘px’ y cuando se utilice `\spunit{100 px}` se imprimirá ‘100px’ en la salida PDF y NVDA/MathCAT verbalizará «cien píxeles».

3.2.4 El comando \spunitalias

`\spunitalias \spunitalias{nuevo alias}{expresión de unidades}`

El comando `\spunitalias` trabaja en *modo texto* y recibe dos *argumentos obligatorios*: $\{ \textit{nuevo alias} \}$ que establece es el nuevo $\{ \textit{alias} \}$ de $\{ \textit{argumento} \}$ para `\spunit` y $\{ \textit{expresión de unidades} \}$ que puede contener multiplicación o división de $\{ \textit{unidades} \}$ ya registradas que se imprimirán en la salida PDF.

El `{\nuevo alias}` no debe estar registrado como `{\unidad}` o `{\alias}` para `\spunit` descritos en §3.2.2 o estar definido como `{\unidad}` por medio del comando `\spnewunit`. Si el `{\nuevo alias}` ya está registrado se devolverá un “error” y se detendrá la compilación del documento.

La declaración de un $\{\langle \text{nuevo alias} \rangle\}$ es «global» y se almacena en el módulo `spintent`. Lua como una $\{\langle \text{alias} \rangle\}$ valido para pasar como $\{\langle \text{argumento} \rangle\}$ a `\spunit`. Por ejemplo: `\spunitalias{caudal}{m^3/s}` creará el $\{\langle \text{alias} \rangle\}$ ‘caudal’ y cuando se utilice `\spunit{50 caudal}` se imprimirá ‘50 m³/s’ en la salida PDF y **NVDA/MathCAT** verbalizará «cincuenta metros al cubo por segundos».

3.3 El comando \spmoney

```
\spmoney \spmoney{< cifra >}
\spmoney{< cifra moneda >}
\spmoney{< key = val >}{< cifra moneda >}
```

El comando `\spmoney` trabaja en *modo matemático* y recibe el *argumento obligatorio* $\{ \langle \text{cifra} \rangle \}$ que contiene la «cifra numérica» positiva o negativa la se cual procesa internamente por el comando `\sprnum` al inicio del $\{ \langle \text{argumento} \rangle \}$ acompañada de un «símbolo de moneda» o «alias de moneda» opcional descritos en §3.3.2.

Si no se proporciona el *argumento opcional* `[key = val]` y se ejecuta `\$spmoney{500}` se imprimirá ‘\$500’ en la salida PDF y **NVDA/MathCAT** verbalizará «quinientos pesos».

El comportamiento de `\spmoney` depende de como se pase el *argumento obligatorio* que tiene prioridad por sobre los valores por defecto. Por ejemplo: `\spmoney{500 dólares}` imprimirá ‘\$500’ en la salida PDF y `NVDA/MathCAT` verbalizará «*quinientos dolares*» aunque la moneda por defecto sean pesos.

Si se pasa un «*alias de moneda*» del tipo ISO (código de divisas) como CLP o USD (mayúsculas) se modifica la verbalización en NVDA/MathCAT agregando el país y se ajusta la salida PDF al estandar ISO. Por ejemplo: `\$spmoney{500 CLP}` imprimirá ‘CLP500’ en la salida PDF y NVDA/MathCAT verbalizará «*quinientos pesos chilenos*».

Si se pasa un «*alias de moneda*» del tipo ISO (código de divisas) como `clp` o `usd` (minúsculas) se modifica la verbalización en **NVDA/MathCAT** agregando el país, pero, NO se modifica la salida PDF imprimiendo el

«*símbolo de moneda*» correspondiente. Por ejemplo: `\$ \spmoney{500 c\p}` imprimirá '\$500' en la salida PDF y NVDA/MathCAT verbalizará «quinientos pesos chilenos»

Pasar de manera directa en el *argumento obligatorio* un «*símbolo de moneda*» como '\$', '€', '£' o un «*alias de moneda*» como peso, euro o libra produce la salida PDF y verbalización estándar esperada para NVDA/MathCAT.

- La posición del «*símbolo de moneda*» a la izquierda o a la derecha de la «*cifra numérica*» y la verbalización para NVDA/MathCAT se manejan desde el módulo Lua `spintent.lua` y no se proveen llaves para modificar este comportamiento.
- El «*símbolo de moneda*» es tomado de la fuente tipográfica definida para el *modo matemático* al momento de ejecutar el comando, se debe tener presente en caso de querer usar un «*símbolo de moneda*» poco habitual que esté soportada por `spintent`. El `{\argumento}` puede contener espacios matemáticos válidos, pero, NO puede contener comandos con o sin argumentos como `\textstyle`, `\mathrm{\#1}` o `\mathbf{\#1}` por ejemplo. La coma ',' o el punto '.' como *separador decimal* solo pueden aparecer una «única vez». Cualquier entrada fuera de esta sintaxis devolverá un "error" deteniendo la compilación del documento.

3.3.1 Llaves para \spmoney

El comando `\spmoney` posee la *meta-key* `cifras-sep` pasada al comando `\spnum` (§3.1.1).

`currency = {\moneda | alias}` default: peso

Establece el *símbolo de moneda* por defecto que se imprimirá en la salida PDF y cómo lo verbalizará NVDA/MathCAT cuando SOLO se pasa una «*cifra numérica*» como `{\argumento}` al comando `\spmoney`. El «*símbolo de moneda*» o el «*alias de moneda*» pasado a esta llave debe ser alguno de los descritos en §3.3.2.

`sub-units = {\true | false}` default: false

Establece si se usarán *subunidades* de la moneda en curso para la verbalización en NVDA/MathCAT. La moneda en curso debe aceptar *subunidades*. Por ejemplo: `\$ \spmoney[sub-units=true]{100,5 dolares}` imprimirá '\$100,5' en la salida PDF y NVDA/MathCAT verbalizará «*cient dólares con cincuenta centavos*».

`dolar-math = {\true | false}` default: true

Establece si el *símbolo* '\$' se imprimirá en *modo texto* o en *modo matemático*. Si se establece en `true`, se usará la fuente tipográfica definida para el *modo matemático* en curso para imprimir '\$' en la salida PDF; si se establece en `false`, se usará la fuente tipográfica definida para el *modo texto* para imprimir '\$' en la salida PDF.

3.3.2 Monedas y alias para \spmoney

Resumen de los `{\símbolos de moneda}` y `{\alias de moneda}` soportados como `{\argumento}` para `\spmoney`.

3.4 El comando \spshort

`\spshort` `{\abreviaturas | acrónimos | números ordinales}`
`\spshort[key = val]` `{\abreviaturas | acrónimos | números ordinales}`
`\spshort[read-arg={verbalización del argumento para NVDA}]` `{\comandos LATEX}`

El comando `\spshort` trabaja *solo en modo texto* y recibe un *argumento obligatorio* del tipo `{\abreviaturas}`, `{\acrónimos}` o `{\números ordinales}` descritos en §3.4.2.

Si no se proporciona el *argumento opcional* `[key = val]` y se ejecuta `\spshort{dr.a}` se imprimirá 'Dr.^a' en la salida PDF y NVDA/MathCAT verbalizará «*doctora*»; si se ejecuta `\spshort{onu}` se imprimirá 'ONU' en la salida PDF y NVDA/MathCAT verbalizará «*organización de las naciones unidas*» y si se ejecuta `\spshort{mineduc}` se imprimirá 'Mineduc' en la salida PDF y NVDA/MathCAT verbalizará «*ministerio de educación*».

Para la escritura de `{\números ordinales}` se puede ejecutar `\spshort{1.a}` que imprimirá '1.^a' en la salida PDF y NVDA/MathCAT verbalizará «*primera*»; si se ejecuta `\spshort{1.o}` imprimirá '1.^o' en la salida PDF y NVDA/MathCAT verbalizará «*primero*».

Para crear una nueva `{\abreviatura}`, `{\acrónimo}` o `{\número ordinal}` que no se encuentre registrada por `spintent` dentro del módulo Lua `spintent.lua` descritas en §3.4.2 se debe utilizar la llave `read-arg` y definir esta dentro del *argumento obligatorio* que puede contener `{\comandos LATEX}` en *modo texto* los cuales se imprimirán en salida PDF.

- Para la escritura de `{\números ordinales}` se debe tener cuidado y no confundir la «*letra o voladita*» 'º' con el símbolo de grado '°'. Si la fuente tipográfica del *modo texto* está utilizando `Numbers=OldStyle` los números pueden quedar muy separados de la «*letra voladita*», la solución para esto es agregar `\addfontfeatures{Numbers=Lining}` dentro de un grupo junto al comando: `{\addfontfeatures{Numbers=Lining}\spshort{1.o}}`.
- El comando `\spshort` está pensado como un reemplazo para *shorthands* y el comando `\sptext` provistos por el paquete `babel` en español compatible con *tagged* PDF. Es de las pocas utilidades que provee el paquete `spintent` que trabaja *solo en modo texto*. Internamente utiliza la llave `E` (*expansion text*) proveída por el paquete `tagpdf`[29].

Entrada	Alias	Nombre	Salida	NVDA/MathCAT
\$	peso, pesos	peso	\$	«peso / pesos»
€	euro, euros	euro	€	«euro / euros»
£	libra, libras	libra	£	«libra / libras»
¥	yen, yenes	yen	¥	«yen / yenes»
¥	yuan, yuanes	yuan	¥	«yuan / yuanes»
₩	won, wons	won	₩	«won / wons»
₪	shekel, shekels, sequel, sequeles	séquel	₪	«séquel / séqueles»
₹	rupia, rupias	rupia	₹	«rupia / rupias»
₽	rublo, rublos	rublo	₽	«rublo / rublos»
₺	lira, liras	lira	₺	«lira / liras»
₾	grivna, grivnas	grivna	₾	«grivna / grivnas»
¢	centavo, centavos	centavo	¢	«centavo / centavos»

Divisas en formato ISO (Mayúsculas)

CLP	—	peso chileno	CLP	«peso chileno / pesos chilenos»
MXN	—	peso mexicano	MXN	«peso mexicano / pesos mexicanos»
USD	—	dólar	USD	«dólar estadounidense / dólares estadounidenses»
EUR	—	euro	EUR	«euro / euros»
GBP	—	libra	GBP	«libra / libras»
JPY	—	yen	JPY	«yen / yenes»
CNY	—	yuan	CNY	«yuan / yuanes»
KRW	—	won	KRW	«won / wons»
ILS	—	séquel	ILS	«séquel / séqueles»
INR	—	rupia	INR	«rupia / rupias»
RUB	—	rublo	RUB	«rublo / rublos»
TRY	—	lira	TRY	«lira / liras»
UAH	—	grivna	UAH	«grivna / grivnas»

Códigos ISO en minúsculas

clp	—	peso chileno	\$	«peso chileno / pesos chilenos»
mxn	—	peso mexicano	\$	«peso mexicano / pesos mexicanos»
usd	dolar, dolares	dólar	\$	«dólar estadounidense / dólares estadounidenses»
eur	—	euro	€	«euro / euros»
gbp	—	libra	£	«libra / libras»
jpy	—	yen	¥	«yen / yenes»
cny	—	yuan	¥	«yuan / yuanes»
krw	—	won	₩	«won / wons»
ils	—	séquel	₪	«séquel / séqueles»
inr	—	rupia	₹	«rupia / rupias»
rub	—	rublo	₽	«rublo / rublos»
try	—	lira	₺	«lira / liras»
uah	—	grivna	₾	«grivna / grivnas»

3.4.1 Llaves para \spshort

`read-arg = { <verbalización NVDA> }` default: no usado

Establece la *verbalización del argumento* { <comandos $\LaTeX$ > } de la nueva { <abreviatura> }, { <acrónimo> } o { <número ordinal> } para NVDA. El valor de esta llave *siempre* debe pasarse entre ‘{ }’. Por ejemplo: `\spshort[read-arg={octavo}]{8.\textsuperscript{\emph{avo}}}` creará una nueva { <abreviatura> } que imprimirá ‘8.^{avo}’ en la salida PDF y NVDA verbalizará «octavo».

`small-caps = { <true | false> }` default: false

Establece si se usarán *versalitas* para imprimir { <acrónimos> } y para la «letra volada» en { <abreviaturas> } y { <números ordinales> } respetando las reglas de la RAE para el uso de estas. Si se establece en `true`, las abreviaturas con mayúscula como EE. UU. y los acrónimos de cinco o más letras como Unicef no se verán afectados; si se establece en `false`, no se aplicarán *versalitas*. Por ejemplo: `\spshort[small-caps=true]{dr.a}` imprimirá ‘Dr.^a’ en la salida PDF y NVDA verbalizará «doctora».

3.4.2 Argumentos para \spshort

El cuadro 2 resume las abreviaturas, expresiones latinas y títulos registrados en el módulo Lua `spintent.lua`.

Cuadro 2: Abreviaturas soportadas

Entrada	Alias / Variantes	Salida PDF	Lectura NVDA
Abreviaturas con letras voladas			
dr.a	dr. ^a	Dr. ^a	«doctora»
dr.as	—	Dr. ^{as}	«doctoras»
prof.a	—	Prof. ^a	«profesora»
d.a	d. ^a	D. ^a	«doña»
m.a	—	M. ^a	«maría»
n.o	n. ^o	N. ^o	«número»
n.os	—	N. ^{os}	«números»
c.ia	—	C. ^{ia}	«compañía»
Abreviaturas comunes y expresiones latinas			
pág.	pag.	pág.	«página»
vol.	—	vol.	«volumen»
etc.	—	etc.	«etcétera»
et al.	et. al.	et al.	«y otros»
ibíd.	ibid.	ibíd.	«ibídem»
op. cit.	op.cit.	op. cit.	«obra citada»
loc. cit.	loc.cit.	loc. cit.	«lugar citado»
v. gr.	v.gr.	v. gr.	«verbigracia»
i. e.	i.e.	i. e.	«esto es»
e. g.	e.g.	e. g.	«por ejemplo»
p. ej.	p.ej.	p. ej.	«por ejemplo»
Títulos, profesiones y cargos			
sr.	—	Sr.	«señor»
sra.	—	Sra.	«señora»
srta.	—	Srta.	«señorita»
dr.	—	Dr.	«doctor»
dra.	—	Dra.	«doctora»
dres.	—	Dres.	«doctores»
dras.	—	Dras.	«doctoras»
prof.	—	Prof.	«profesor»
profa.	—	Profa.	«profesora»
profs.	—	Profs.	«profesores»
ing.	—	Ing.	«ingeniero»
ings.	—	Ings.	«ingenieros»
lic.	—	Lic.	«licenciado»
v. b.	v.b.	V. B.	«visto bueno»
Abreviaturas compuestas e institucionales			
s. a.	s.a.	S. A.	«sociedad anónima»
ee. uu.	ee.uu.	EE. UU.	«estados unidos»
dd. hh.	dd.hh.	DD. HH.	«derechos humanos»
jj. oo.	jj.oo.	JJ. OO.	«juegos olímpicos»

Continúa en la siguiente página...

Cuadro 2: (Continuación - Abreviaturas soportadas)

Entrada	Alias / Variantes	Salida PDF	Lectura NVDA
ff. aa.	ff.aa.	FF. AA.	«fuerzas armadas»
rr. ee.	rr.ee.	RR. EE.	«relaciones exteriores»
cc. aa.	cc.aa.	CC. AA.	«comunidades autónomas»
p. d.	p.d.	P. D.	«posdata»
a. c.	a.c.	a. C.	«antes de Cristo»
d. c.	d.c.	d. C.	«después de Cristo»
d. n. i.	d.n.i.	D. N. I.	«documento nacional de identidad»
r. u. t.	r.u.t.	R. U. T.	«rol único tributario»
r. u. n.	r.u.n.	R. U. N.	«rol único nacional»

El cuadro 3 detalla las siglas y acrónimos registrados en el módulo Lua `spintent.lua`.

Cuadro 3: Siglas y acrónimos soportados

Entrada	Alias / Variantes	Salida PDF	Lectura NVDA
Siglas (Deletreo o lectura expandida)			
dni	—	DNI	«documento nacional de identidad»
rut	—	RUT	«rol único tributario»
run	—	RUN	«rol único nacional»
onu	—	ONU	«organización de las naciones unidas»
rae	—	RAE	«real academia española»
ong	ongs	ONG	«organización no gubernamental»
urss	—	URSS	«unión de repúblicas socialistas soviéticas»
oea	—	OEA	«organización de los estados americanos»
oms	—	OMS	«organización mundial de la salud»
fmi	—	FMI	«fondo monetario internacional»
bid	—	BID	«banco interamericano de desarrollo»
otan	—	OTAN	«organización del tratado del atlántico norte»
ue	—	UE	«unión europea»
ru	—	RU	«reino unido»
Acrónimos silábicos			
unicef	—	Unicef	«fondo de las naciones unidas para la infancia»
unesco	—	Unesco	«organización de las naciones unidas para la educación, la ciencia y la cultura»
mercosur	—	Mercosur	«mercado común del sur»
cepal	—	Cepal	«comisión económica para américa latina»
mineduc	—	Mineduc	«ministerio de educación»
conicet	—	Conicet	«consejo nacional de investigaciones»

El cuadro 4 muestra ejemplos de números ordinales registrados en el módulo Lua `spintent.lua`.

Cuadro 4: Números ordinales soportados

Entrada	Alias / Variantes	Salida PDF	Lectura NVDA
Sufijos para ordinales (Ejemplos)			
1.a	1. ^a	1. ^a	«primera»
2.o	2. ^o	2. ^o	«segundo»
3.er	—	3. ^{er}	«tercer»
4.os	—	4. ^{os}	«cuartos»
5.as	—	5. ^{as}	«quintas»

4 Utilidades generales

Además de los «cuatro grandes» y sus asociados, el paquete **spintent** provee tres utilidades de uso general: `\spsiglo` para la escritura y lectura de siglos, `\sptime` para la escritura y lectura de tiempo y `\spdate` para la escritura y lectura de fechas.

El comando `\spsiglo`

```
\spsiglo \spsiglo{<siglo>}
\spsiglo[<key = val>]{<siglo>}
```

El comando `\spsiglo` trabaja en *modo texto* y en *modo matemático*, recibe el *argumento obligatorio* `{<siglo>}`, el cual puede ser un *número arábigo* ‘21’ o un *número romano* ‘XXI’ mayor que ‘0’ y menor que ‘4000’.

Si no se proporciona el *argumento opcional* `[<key = val>]` y se ejecuta `\spsiglo{21}` o `\spsiglo{XXI}` se imprimirá ‘xxi’ en la salida PDF y NVDA/MathCAT verbalizará «veintiuno».

- El *argumento obligatorio* `{<siglo>}` «siempre» es convertido a *número romano en versalitas* y es verbalizado por NVDA/MathCAT como *número arábigo*. Esta es la única utilidad que provee el paquete **spintent** que trabaja en *modo texto* y en *modo matemático*. Internamente utiliza la llave `alt` proveída por el paquete **tagpdf** en *modo texto* y usa `\MathMLintent` para el *modo matemático*.

Llaves para `\spsiglo`

```
print-era = {<ac | dc | none>} default: none
```

Establece si se *imprime* o no la abreviatura después de imprimir el argumento `{<siglo>}`. Si se establece en *ac*, `\spsiglo[print-era=ac]{XXI}` imprimirá ‘xi a. C.’ en la salida PDF y NVDA/MathCAT verbalizará «veintiuno antes de cristo»; si se establece en *dc*, `\spsiglo[print-era=dc]{XXI}` imprimirá ‘xi d. C.’ en la salida PDF y NVDA/MathCAT verbalizará «veintiuno después de cristo»; si se establece en *none*, `\spsiglo[print-era=none]{XXI}` imprimirá ‘xi’ en la salida PDF y NVDA/MathCAT verbalizará «veintiuno».

El comando `\sptime`

```
\sptime \sptime{<horas : minutos>}
\sptime{<horas : minutos : segundos>}
```

El comando `\sptime` trabaja en *modo matemático* y recibe el *argumento obligatorio* `{<horas : minutos>}` o `{<horas : minutos : segundos>}`. Por ejemplo: `\sptime{23:45}` imprimirá ‘23:45’ en *modo texto* la salida PDF y NVDA/MathCAT verbalizará «las veintitrés y cuarenta y cinco minutos».

El comando `\spdate`

```
\spdate \spdate{<DD/MM/YYYY>} \spdate{<DD-MM-YYYY>}
\spdate{<YYYY/MM/DD>} \spdate{<YYYY-MM-DD>}
```

El comando `\spdate` trabaja en *modo matemático* y recibe el *argumento obligatorio* `{<DD/MM/YYYY>}`, `{<DD-MM-YYYY>}` o `{<YYYY-MM-DD>}`. Por ejemplo: `\spdate{1981-01-02}` y `\spdate{02/01/1981}` imprimirán en *modo texto* la salida PDF ‘1981-01-02’ y ‘02/01/1981’ respectivamente y en ambos casos, NVDA/MathCAT verbalizará «dos de enero de mil novecientos ochenta y uno».

- Si se pasa el argumento `{<YYYY/MM/DD>}`, internamente se convertirá en `{<DD/MM/YYYY>}` y se imprimirá de esa forma por compatibilidad con NVDA/MathCAT.

5 Utilidades para símbolos

El paquete **spintent** provee tres comandos `\spread`, `\spdsym` y `\spmsym` para trabajar con «*símbolos matemáticos*» descritos en esta sección. Están pensando desde el punto de vista docente, para situaciones donde la lectura de un «*símbolo*» (acorde al contexto) es relevante.

- A diferencia de las otras utilidades implementados por **spintent** (que analizan y dan formato a la salida), estos no procesan los «*símbolos matemáticos*», solo afectan la *forma* (semántica) en que serán verbalizados por NVDA/MathCAT.

El comando `\spread`

```
\spread \spread{<verbalización NVDA>}{<símbolo matemático>}
```

El comando `\spread` trabaja en *modo matemático* y recibe dos *argumentos obligatorios*: El primer *argumento* `{<verbalización NVDA>}` establece la «*forma*» en la cual NVDA/MathCAT verbalizará al segundo *argumento* `{<símbolo matemático>}`. Por ejemplo: `\spread{es-paralela}{\parallel}` imprimirá ‘||’ en la salida PDF y NVDA/MathCAT verbalizará «es paralela»; `\spread{es-paralelo}{\parallel}` imprimirá ‘||’ en la salida PDF y NVDA/MathCAT verbalizará «es paralelo».

El comando `\spdsym`

```
\spdsym \spdsym
\spdsym[⟨key = val⟩]
```

El comando `\spdsym` trabaja en *modo matemático* y recibe el *argumento opcional* [`⟨key = val⟩`] mediante el cual se establece la «forma» en la que se imprimirá el *símbolo de división* y como se verbalizará en NVDA/MathCAT.

Si no se proporciona el *argumento opcional* [`⟨key = val⟩`], `\spdsym` imprimirá ‘ $\div$ ’ en la salida PDF y NVDA/MathCAT verbalizará «dividido».

Llaves para `\spdsym`

`prefix = {⟨prefijo⟩}` default: no usado

Establece el *prefijo* que será verbalizado por NVDA/MathCAT «antes» de verbalizar el {⟨valor⟩} establecido por la llave `read-sym`. El valor de esta llave *siempre* debe pasarse entre llaves ‘{ }’.

`suffix = {⟨sufijo⟩}` default: no usado

Establece el *sufijo* que será verbalizado por NVDA/MathCAT «después» de verbalizar el {⟨valor⟩} establecido por la llave `read-sym`. El valor de esta llave *siempre* debe pasarse entre llaves ‘{ }’.

`set-sym = {⟨old-style | two-dots | slash⟩}` default: two-dots

Establece el *símbolo de división* que se imprimirá en la salida PDF. Si se establece en `old-style`, imprimirá ‘ $\div$ ’; si se establece en `two-dots`, imprimirá ‘ $\ddots$ ’ y si se establece `slash`, imprimirá ‘/’.

`read-sym = {⟨dividido | dividido-por | dividido-entre⟩}` default: dividido

Establece la *forma* con la cual NVDA/MathCAT verbalizará el *símbolo de división*. Si se establece en `dividido`, NVDA/MathCAT verbalizará «dividido»; si se establece en `dividido-por`, NVDA/MathCAT verbalizará «dividido por» y si se establece en `dividido-entre`, NVDA/MathCAT verbalizará «dividido entre».

El comando `\spmsym`

```
\spmsym \spmsym
\spmsym[⟨key = val⟩]
```

El comando `\spmsym` trabaja en *modo matemático* y recibe el *argumento opcional* [`⟨key = val⟩`] mediante el cual se establece la «forma» en la que se imprimirá el *símbolo de multiplicación* y como se verbalizará en NVDA/MathCAT.

Si no se proporciona el *argumento opcional* [`⟨key = val⟩`], `\spmsym` imprimirá ‘ $\cdot$ ’ en la salida PDF y NVDA/MathCAT verbalizará «por».

Llaves para `\spmsym`

`prefix = {⟨prefijo⟩}` default: no usado

Establece el *prefijo* que será verbalizado por NVDA/MathCAT «antes» de verbalizar el {⟨valor⟩} establecido por la llave `read-sym`. El valor de esta llave *siempre* debe pasarse entre llaves ‘{ }’.

`suffix = {⟨sufijo⟩}` default: no usado

Establece el *sufijo* que será verbalizado por NVDA/MathCAT «después» de verbalizar el {⟨valor⟩} establecido por la llave `read-sym`. El valor de esta llave *siempre* debe pasarse entre llaves ‘{ }’.

`set-sym = {⟨cross | dot | asterisk⟩}` default: dot

Establece el *símbolo de multiplicación* que se imprimirá en la salida PDF. Si se establece en `cross`, imprimirá ‘ $\times$ ’; si se establece en `dot`, imprimirá ‘ $\cdot$ ’ y si se establece en `asterisk`, imprimirá ‘ $\ast$ ’.

`read-sym = {⟨por | multiplicado-por | veces⟩}` default: por

Establece la *forma* con la cual NVDA/MathCAT verbalizará el *símbolo de multiplicación*. Si se establece en `por`, NVDA/MathCAT verbalizará «por»; si se establece en `multiplicado-por`, NVDA/MathCAT verbalizará «multiplicado por» y si se establece en `veces`, NVDA/MathCAT verbalizará «veces».

`not-print = {⟨true | false⟩}` default: false

Deshabilita la *impresión* del *símbolo de multiplicación*. Cuando se establece esta llave, NVDA/MathCAT verbalizará el *símbolo de multiplicación* acorde al valor establecido por la llave `read-sym`, pero, no se imprimirá este en la salida PDF.

6 Utilidades para números

El paquete `spintent` provee utilidades para trabajar con números descritas en esta sección.

El comando `\spdiv`

```
\spdiv {<dividendo>}{<divisor>}
\spdiv[<key = val>]{<dividendo>}{<divisor>}
```

El comando `\spdiv` trabaja en *modo matemático* y recibe dos *argumentos obligatorios* `{<dividendo>}` y `{<divisor>}`.

Si no se proporciona el *argumento opcional* `[<key = val>]`, `\spdiv{20}{26}` imprimirá ‘20 : 26’ en la salida PDF y NVDA/MathCAT verbalizará «veinte dividido veinte y seis».

Llaves para `\spdiv`

El comando `\spdiv` posee las *meta-keys* `cifras-sep`, `read-sign`, `read-period-sep` y `notation-sym` pasadas al comando `\spnum` (§3.1.1) junto a las *meta-keys* `set-sym` y `read-sym` pasadas al comando `\spdsym` (pág. 12).

`wrap-parens = {<true | false>}` default: false

Establece si se *imprimen* paréntesis redondos ‘(...)’ alrededor de los *argumentos* `{<dividendo>}` y `{<divisor>}` pasados a `\spdiv`. Si se establece en `true`, se imprimirán paréntesis redondos alrededor de ambos *argumentos*; si se establece en `false`, no se imprimirán los paréntesis.

`read-parens = {<true | false>}` default: true

Establece si NVDA/MathCAT verbalizará los paréntesis redondos ‘(...)’ alrededor de los *argumentos* `{<dividendo>}` y `{<divisor>}` pasados a `\spdiv`. Si se establece en `true`, NVDA/MathCAT verbalizará la lectura de los paréntesis; si se establece en `false`, NVDA/MathCAT no los verbalizará.

El comando `\spmul`

```
\spmul {<factor>}{<factor>}
\spmul[<key = val>]{<factor>}{<factor>}
```

El comando `\spmul` trabaja en *modo matemático* y recibe dos *argumentos obligatorios* `{<factor>}` y `{<factor>}`.

Si no se proporciona el *argumento opcional* `[<key = val>]`, `\spmul{20}{26}` imprimirá ‘20 · 26’ en la salida PDF y NVDA/MathCAT verbalizará «veinte por veinte y seis».

Llaves para `\spmul`

El comando `\spmul` posee las *meta-keys* `cifras-sep`, `read-sign`, `read-period-sep` y `notation-sym` pasadas al comando `\spnum` (§3.1.1) junto a las *meta-keys* `set-sym` y `read-sym` pasadas al comando `\spmsym` (pág. 12).

`wrap-parens = {<true | false>}` default: false

Establece si se *imprimen* paréntesis redondos ‘(...)’ alrededor de los *argumentos* `{<factor>}` y `{<factor>}` pasados a `\spmul`. Si se establece en `true`, se imprimirán paréntesis redondos alrededor de ambos *argumentos*; si se establece en `false`, no se imprimirán los paréntesis.

`read-parens = {<true | false>}` default: true

Establece si NVDA/MathCAT verbalizará los paréntesis redondos ‘(...)’ alrededor de los *argumentos* `{<factor>}` y `{<factor>}` pasados a `\spmul`. Si se establece en `true`, NVDA/MathCAT verbalizará la lectura de los paréntesis; si se establece en `false`, NVDA/MathCAT no los verbalizará.

El comando `\spnD`

```
\spnD {<número natural>}
\spnD[<key = val>]{<número natural>}
```

El comando `\spnD` trabaja en *modo matemático* y recibe el *argumento delimitado por paréntesis* `(<número natural>)`, por ejemplo: ‘(6)’. Si se usa sin `(<argumento>)`, `\spnD` imprimirá ‘D(n)’ en la salida PDF y NVDA/MathCAT verbalizará «divisores de n».

Si se usa con `(<argumento>)`, por ejemplo, `\spnD(6)`, imprimirá ‘D(6)’ en la salida PDF y NVDA/MathCAT verbalizará «divisores de seis».

Llaves para `\spnD`

`prefix = {<prefijo>}` default: no usado

Establece el *prefijo* que será verbalizado por NVDA/MathCAT «antes» de verbalizar `\spnD(<argumento>)`. El valor de esta llave *siempre* debe pasarse entre llaves ‘{ }’.

`result = {<true | false>}` default: false

Establece si se *imprime* el resultado de calcular los divisores del `(<número natural>)` pasado a `\spnD`. Si se establece en `true`, `\spnD[result=true](6)` imprimirá ‘D(6) = {1,2,3,6}’ en la salida PDF y NVDA/MathCAT verbalizará «los divisores de seis son uno, dos, tres y seis»; si se establece en `false`, no se imprimirá en la salida PDF el resultado de calcular los divisores.

`result-num = {<número natural>}` default: 5

Establece la *cantidad* de números que se mostrarán en la salida PDF al activar `result=true`. Si se establece en `2`, solo se imprimirán dos valores en la salida PDF; si no se establece, se imprimirán como máximo `5` valores y luego se añadirá ‘...’ en la salida PDF.

`sample-arg = {⟨true | false⟩}` default: *false*

Desactiva la *validación* interna del (`⟨argumento⟩`) pasado a `\spnD`, permitiendo usar ejemplos «no numéricos» como entrada. Si se establece en `true`, `\spnD[sample-arg=true](m)` imprimirá ‘D(*m*)’ en la salida PDF y NVDA/MathCAT verbalizará «divisores de *m*»; si se establece en `false`, el (`⟨argumento⟩`) debe ser un número natural, en caso contrario se devolverá un “error”.

`silent-cmd = {⟨true | false⟩}` default: *false*

Establece si se *silencia* la verbalización para NVDA/MathCAT de `\spnD(⟨argumento⟩)` cuando se establece `sample-arg=true` o cuando `\spnD` se usa sin (`⟨argumento⟩`). Si se establece en `true` NVDA/MathCAT NO lo verbalizará; si se establece en `false`, NVDA/MathCAT SÍ lo verbalizará.

El comando `\spnM`

```
\spnM \spnM
\spnM(⟨número natural⟩)
\spnM[key = val](⟨número natural⟩)
```

El comando `\spnM` trabaja en *modo matemático* y recibe el *argumento delimitado por paréntesis* (`⟨número natural⟩`), por ejemplo: ‘(6)’. Si se usa sin (`⟨argumento⟩`), `\spnM` imprimirá ‘M(*n*)’ en la salida PDF y NVDA/MathCAT verbalizará «múltiplos de *n*».

Si se usa con (`⟨argumento⟩`), por ejemplo, `\spnM(6)`, imprimirá ‘M(6)’ en la salida PDF y NVDA/MathCAT verbalizará «múltiplos de seis».

Llaves para `\spnM`

`prefix = {⟨prefijo⟩}` default: *no usado*

Establece el *prefijo* que será verbalizado por NVDA/MathCAT «antes» de verbalizar `\spnM(⟨argumento⟩)`. El valor de esta llave *siempre* debe pasarse entre llaves ‘{ }’.

`result = {⟨true | false⟩}` default: *false*

Establece si se *imprime* el resultado de calcular los *múltiplos* del (`⟨número natural⟩`) pasado a `\spnM`. Si se establece en `true`, `\spnM[result=true](6)` imprimirá ‘M(6) = {6, 12, 18, 24, 30, ...}’ en la salida PDF y NVDA/MathCAT verbalizará «los múltiplos de seis son seis, doce, dieciocho, veinticuatro, treinta puntos suspensivos»; si se establece en `false`, no se imprimirá en la salida PDF el resultado de calcular los *múltiplos*.

`result-num = {⟨número natural⟩}` default: *5*

Establece la *cantidad* de números que se mostrarán en la salida PDF al activar `result=true`. Si se establece en `2`, solo se imprimirán dos valores en la salida PDF seguidos de ‘...’; si no se establece, se imprimirán por defecto los `5` primeros valores y luego se añadirá ‘...’ en la salida PDF.

`sample-arg = {⟨true | false⟩}` default: *false*

Desactiva la *validación* interna del (`⟨argumento⟩`) pasado a `\spnM`, permitiendo usar ejemplos «no numéricos» como entrada. Si se establece en `true`, `\spnM[sample-arg=true](m)` imprimirá ‘M(*m*)’ en la salida PDF y NVDA/MathCAT verbalizará «múltiplos de *m*»; si se establece en `false`, el (`⟨argumento⟩`) debe ser un número natural, en caso contrario se devolverá un “error”.

`silent-cmd = {⟨true | false⟩}` default: *false*

Establece si se *silencia* la verbalización en NVDA/MathCAT de `\spnM(⟨argumento⟩)` cuando se establece `sample-arg=true` o cuando `\spnM` se usa sin (`⟨argumento⟩`). Si se establece en `true` NVDA/MathCAT NO lo verbalizará; si se establece en `false` NVDA/MathCAT lo verbalizará.

El comando `\spmcD`

```
\spmcD \spmcD
\spmcD(⟨números separados por coma⟩)
\spmcD[key = val](⟨números separados por coma⟩)
```

El comando `\spmcD` trabaja en *modo matemático* y recibe el *argumento delimitado por paréntesis* (`⟨números separados por coma⟩`), el cual debe ser una lista de *números naturales*, por ejemplo: ‘(2, 3, 6)’. Si se usa sin (`⟨argumento⟩`) `\spmcD` imprimirá ‘mcd’ en la salida PDF, y NVDA/MathCAT verbalizará «máximo común divisor» o «*m c d*», acorde al {`⟨valor⟩`} establecido por la llave `read-cmd`.

Si se usa con (`⟨argumento⟩`) `\spmcD(2, 3, 6)` imprimirá ‘mcd(2,3,6)’ en la salida PDF, y NVDA/MathCAT verbalizará «máximo común divisor entre dos, tres y seis» o «*m c d entre dos, tres y seis*» acorde al {`⟨valor⟩`} establecido por la llave `read-cmd`.

Llaves para `\spgcd`

<code>upper-case = {⟨true false⟩}</code>	default: <i>false</i>
Establece si se usarán <i>mayúsculas</i> o <i>minúsculas</i> en la salida PDF para el comando <code>\spgcd</code> . Si se establece en <i>true</i> , se imprimirá ‘MCD’ en la salida PDF; si se establece en <i>false</i> , se imprimirá ‘gcd’ en la salida PDF.	
<code>read-cmd = {⟨terse clear⟩}</code>	default: <i>clear</i>
Establece la <i>forma</i> en la que NVDA/MathCAT verbalizará el comando <code>\spgcd</code> . Si se establece en <i>terse</i> , NVDA/MathCAT verbalizará « <i>m c d</i> »; si se establece en <i>clear</i> , NVDA/MathCAT verbalizará « <i>máximo común divisor</i> ».	
<code>prefix = {⟨prefijo⟩}</code>	default: <i>no usado</i>
Establece el <i>prefijo</i> que será verbalizado por NVDA/MathCAT «antes» de verbalizar el valor establecido por la llave <code>read-cmd</code> . El valor de esta llave <i>siempre</i> debe pasarse entre llaves ‘{ }’.	
<code>result = {⟨true false⟩}</code>	default: <i>false</i>
Establece si se <i>imprime</i> el resultado de calcular el « <i>máximo común divisor</i> » del (⟨ <i>argumento</i> ⟩) pasado a <code>\spgcd</code> . Si se establece en <i>true</i> : <code>\spgcd[result=true](2, 3, 6)</code> imprimirá ‘gcd(2,3,6) = 1’ en la salida PDF y NVDA/MathCAT verbalizará « <i>máximo común divisor entre dos, tres y seis es igual a uno</i> »; si se establece en <i>false</i> no se imprimirá en la salida PDF el resultado de calcular el « <i>máximo común divisor</i> ».	
<code>sample-arg = {⟨true false⟩}</code>	default: <i>false</i>
Desactiva la <i>validación</i> interna del (⟨ <i>argumento</i> ⟩) pasado a <code>\spgcd</code> , permitiendo usar ejemplos «no numéricos» como entrada. Si se establece en <i>true</i> : <code>\spgcd[sample-arg=true](m, n)</code> , imprimirá ‘gcd(<i>m</i> , <i>n</i>)’ en la salida PDF y NVDA/MathCAT verbalizará « <i>máximo común divisor entre m y n</i> »; si se establece en <i>false</i> el (⟨ <i>argumento</i> ⟩) debe ser una lista de <i>números naturales</i> , en caso contrario se devolverá un “error”.	
<code>silent-cmd = {⟨true false⟩}</code>	default: <i>false</i>
Establece si se <i>silencia</i> la verbalización en NVDA/MathCAT de <code>\spgcd(⟨argumento⟩)</code> cuando se establece <code>sample-arg=true</code> o cuando <code>\spgcd</code> se usa sin (⟨ <i>argumento</i> ⟩). Si se establece en <i>true</i> NVDA/MathCAT NO lo verbalizará; si se establece en <i>false</i> , NVDA/MathCAT SÍ lo verbalizará.	

El comando `\spmcn`

```

\spmcn \spmcn
\spmcn(⟨números separados por coma⟩)
\spmcn[⟨key = val⟩](⟨números separados por coma⟩)

```

El comando `\spmcn` trabaja en *modo matemático* y recibe el *argumento delimitado por paréntesis* (⟨*números separados por coma*⟩), el cual debe ser una lista de *números naturales*, por ejemplo: ‘(2, 3, 6)’. Si se usa sin (⟨*argumento*⟩) `\spmcn` imprimirá ‘mcn’ en la salida PDF, y NVDA/MathCAT verbalizará «*mínimo común múltiplo*» o «*m c m*», acorde al {⟨*valor*⟩} establecido por la llave `read-cmd`.

Si se usa con (⟨*argumento*⟩) `\spmcn(2, 3, 6)` imprimirá ‘mcn(2,3,6)’ en la salida PDF, y NVDA/MathCAT verbalizará «*mínimo común múltiplo entre dos, tres y seis*» o «*m c m entre dos, tres y seis*» acorde al {⟨*valor*⟩} establecido por la llave `read-cmd`.

Llaves para `\spmcn`

<code>upper-case = {⟨true false⟩}</code>	default: <i>false</i>
Establece si se usarán <i>mayúsculas</i> o <i>minúsculas</i> en la salida PDF para el comando <code>\spmcn</code> . Si se establece en <i>true</i> se imprimirá ‘MCM’ en la salida PDF; si se establece en <i>false</i> se imprimirá ‘mcn’ en la salida PDF.	
<code>read-cmd = {⟨terse clear⟩}</code>	default: <i>clear</i>
Establece la <i>forma</i> en la que NVDA/MathCAT verbalizará el comando <code>\spmcn</code> . Si se establece en <i>terse</i> , NVDA/MathCAT verbalizará « <i>m c m</i> »; si se establece en <i>clear</i> , NVDA/MathCAT verbalizará « <i>mínimo común múltiplo</i> ».	
<code>prefix = {⟨prefijo⟩}</code>	default: <i>no usado</i>
Establece el <i>prefijo</i> que será verbalizado por NVDA/MathCAT «antes» de verbalizar el valor establecido por la llave <code>read-cmd</code> . El valor de esta llave <i>siempre</i> debe pasarse entre llaves ‘{ }’.	
<code>result = {⟨true false⟩}</code>	default: <i>false</i>
Establece si se <i>imprime</i> el resultado de calcular el « <i>mínimo común múltiplo</i> » del (⟨ <i>argumento</i> ⟩) pasado a <code>\spmcn</code> . Si se establece en <i>true</i> : <code>\spmcn[result=true](2, 3, 6)</code> imprimirá ‘mcn(2,3,6) = 6’ en la salida PDF y NVDA/MathCAT verbalizará « <i>mínimo común múltiplo entre dos, tres y seis es igual a seis</i> »; si se establece en <i>false</i> no se imprimirá en la salida PDF el resultado de calcular el « <i>mínimo común múltiplo</i> ».	
<code>sample-arg = {⟨true false⟩}</code>	default: <i>false</i>
Desactiva la <i>validación</i> interna del (⟨ <i>argumento</i> ⟩) pasado a <code>\spmcn</code> , permitiendo usar ejemplos «no numéricos» como entrada. Si se establece en <i>true</i> : <code>\spmcn[sample-arg=true](m, n)</code> , imprimirá ‘mcn(<i>m</i> , <i>n</i>)’ en la salida PDF y NVDA/MathCAT verbalizará « <i>mínimo común múltiplo entre m y n</i> »; si se establece en <i>false</i> el (⟨ <i>argumento</i> ⟩) debe ser una lista de <i>números naturales</i> , en caso contrario se devolverá un “error”.	
<code>silent-cmd = {⟨true false⟩}</code>	default: <i>false</i>

Establece si se *silencia* la verbalización en NVDA/MathCAT de `\spmc`(*argumento*) cuando se establece `sample-arg=true` o cuando `\spmc` se usa sin (*argumento*). Si se establece en `true` NVDA/MathCAT NO lo verbalizará; si se establece en `false` NVDA/MathCAT SÍ lo verbalizará.

El comando `\spnZ`

```
\spnZ {<número entero>}
\spnZ[<key = val>]{<número entero>}
```

El comando `\spnZ` trabaja en *modo matemático* y recibe el *argumento* obligatorio `{<número entero>}` el cual se procesa internamente bajo el comando `\spnum`. Este comando está dedicado exclusivamente al manejo de «*números enteros*», y añade algunas utilidades prácticas para el demarcado e interacción con NVDA/MathCAT.

Llaves para `\spnZ`

El comando `\spnZ` posee las *meta-keys* `cifras-sep` y `read-sign` pasadas al comando `\spnum` (§3.1.1).

`wrap-parens = {<true | false>}` default: *false*

Establece si se *imprimen* paréntesis redondos ‘(…)’ alrededor del argumento `{<número entero>}` pasado a `\spnZ`. Si se establece en `true`, se imprimirán paréntesis redondos alrededor del `{<número entero>}`; si se establece en `false`, no se imprimirán los paréntesis.

`read-parens = {<true | false>}` default: *false*

Establece si NVDA/MathCAT *verbalizará* los paréntesis redondos ‘(…)’ alrededor del *argumento* `{<número entero>}` pasado a `\spnZ`. Si se establece en `true`, NVDA/MathCAT verbalizará la lectura del paréntesis ‘(…)»; si se establece en `false`, NVDA/MathCAT no los verbalizará.

El comando `\spmQ`

```
\spmQ {<número entero>}{<numerador>}{<denominador>}
\spmQ[<key = val>]{<número entero>}{<numerador>}{<denominador>}
```

El comando `\spmQ` trabaja en *modo matemático* y recibe *tres argumentos* obligatorios: `{<número entero>}`, `{<numerador>}` y `{<denominador>}`, los procesa internamente y devuelve un «*número mixto*».

Si no se proporciona el *argumento opcional* `[<key = val>]`, `\spmQ{2}{5}{7}` imprimirá ‘ $2\frac{5}{7}$ ’ en la salida PDF y NVDA/MathCAT verbalizará «*dos y cinco séptimos*».

- En el caso de las fracciones, y en especial con los «*números mixtos*», no se pueden manejar muchas cosas desde el lado de **spintent**. Por ejemplo, la entrada `$3\frac{5}{7}$` con *tagged* PDF se verbaliza por NVDA/MathCAT como «*tres más cinco séptimos*», lo cual no está mal, pero pedagógicamente hablando no es del todo correcto.

Llaves para `\spmQ`

`join-word = {<entero | enteros | y>}` default: *y*

Establece la *forma* en la que NVDA/MathCAT verbaliza la conexión entre el entero y la fracción que forman el número mixto. Si se establece en `entero`, al usar `\spmQ[join-word=entero]{1}{5}{7}` imprimirá ‘ $1\frac{5}{7}$ ’ en la salida PDF, y NVDA/MathCAT verbalizará «*un entero cinco séptimos*»; si se establece en `enteros`, al usar `\spmQ[join-word=enteros]{2}{5}{7}` imprimirá ‘ $2\frac{5}{7}$ ’ en la salida PDF, y NVDA/MathCAT verbalizará «*dos enteros cinco séptimos*»; si se establece en `y`, al usar `\spmQ[join-word=y]{2}{5}{7}` imprimirá ‘ $2\frac{5}{7}$ ’ en la salida PDF, y NVDA/MathCAT verbalizará «*dos y cinco séptimos*».

7 Utilidades para geometría

El paquete **spintent** provee utilidades para trabajar con geometría (plana), que describiremos en esta sección.

El comando `\spang`

```
\spang {<grados>}
\spang {<grados : minutos>}
\spang {<grados : minutos : segundos>}
```

El comando `\spang` trabaja en *modo matemático* y recibe el *argumento obligatorio* `{<grados>}`, `{<grados : minutos>}` o `{<grados : minutos : segundos>}`. Los procesa internamente bajo los comandos `\spnum` y `\spunit` para imprimir y verbalizar el «*ángulo sexagesimal*». Por ejemplo, `\spang{45:30:15}` imprimirá ‘45°30’15”’ en la salida PDF y NVDA/MathCAT verbalizará «*cuarenta y cinco grados treinta minutos quince segundos*».

8 Referencias

- [1] ROBERTSON, WILL . “unicode-math - Unicode mathematics support for X_YTeX and LuaTeX”. Available from CTAN, <https://www.ctan.org/pkg/unicode-math>, 2023.
- [2] KRUEGER, MARCEL. “LuaMML - Automatically generate MathML from LuaTeX math mode material”. Available from CTAN, <https://ctan.org/pkg/luamml>, 2026.
- [3] The L^AT_EX Project. “fontspec - Advanced font selection for X_YL^AT_EX and LuaTeX”. Available from CTAN, <https://www.ctan.org/pkg/fontspec>, 2025.
- [4] VOß, HERBERT. “libertinus-otf - Support for Libertinus OpenType”. Available from CTAN, <https://www.ctan.org/pkg/libertinus-otf>, 2025.
- [5] FLIPO, DANIEL. “erewhon-math - Utopia based OpenType Math font”. Available from CTAN, <https://ctan.org/pkg/erewhon-math>, 2026.
- [6] HOFSTRA, SILKE. “sourcecodepro - Use SourceCodePro with T_EX(-like) systems”. Available from CTAN, <https://ctan.org/pkg/sourcecodepro>, 2025.
- [7] GONZÁLEZ, PABLO. “scontents - Stores L^AT_EX contents in memory or files”. Available from CTAN, <https://www.ctan.org/pkg/scontents>, 2026.
- [8] GONZÁLEZ, PABLO. “enumext - Enumerate exercise sheets”. Available from CTAN, <https://www.ctan.org/pkg/enumext>, 2026.
- [9] BRAAMS, JOHANNES AND BEZOS, JAVIER. “babel - Multilingual support for L^AT_EX, LuaL^AT_EX, X_YL^AT_EX, and Plain T_EX”. Available from CTAN, <https://www.ctan.org/pkg/babel>, 2026.
- [10] BEZOS, JAVIER. “babel-spanish - Babel support for Spanish”. Available from CTAN, <https://www.ctan.org/pkg/babel-spanish>, 2026.
- [11] NV ACCESS. “NVDA - NonVisual Desktop Access screen reader”. Available from NV ACCESS, <https://www.nvaccess.org/>, 2026.
- [12] SOIFFER, NEIL. “MathCAT - Math to speech and braille translation”. Available from GITHUB, <https://nsoiffer.github.io/MathCAT/>, 2026.
- [13] ADOBE INC. “Adobe Acrobat Reader - Reliable PDF viewer”. Available from ADOBE, <https://get.adobe.com/reader/>, 2026.
- [14] ISO. “ISO 32000-2:2020 - Portable Document Format (PDF 2.0)”. Available from PDF ASSOCIATION, <https://pdfa.org/resource/iso-32000-2/>, 2020.
- [15] PDF ASSOCIATION. “WTPDF - Well-Tagged PDF specification”. Available from PDFA, <https://pdfa.org/wtpdf/>, 2024.
- [16] PDF ASSOCIATION. “PDF 2.0 Errata Collection 3”. Available from PDFA, <https://pdfa.org/PDF-2-0-errata-collection-3-now-available/>, 2026.
- [17] L^AT_EX PROJECT LWG. “Best Practice Guide: Math in PDF”. Available from PDFA, <https://pdfa.org/resource/best-practice-guide-math-in-PDF/>, 2026.
- [18] JOHNSON, DUFF. “PDF 2.0: New Features, Real-World Impact”. Available from PDFA, <https://pdfa.org/PDF-2-0-new-features-real-world-impact/>, 2026.
- [19] The L^AT_EX Project. “LaTeX Tagged PDF Project - Documentation and resources”. Available from L^AT_EX PROJECT, <https://latex3.github.io/tagging-project/>, 2026.
- [20] VERAPDF CONSORTIUM. “veraPDF - Industry supported PDF/A and PDF/UA validator”. Available from VERAPDF, <https://verapdf.org/>, 2026.
- [21] DUAL LAB. “ngPDF - Next Generation PDF demonstrator and validator”. Available from NGPDF, <https://ngpdf.com/>, 2026.
- [22] BERRY, KARL. “L^AT_EX 2_ε: An Unofficial Reference Manual”. Available from CTAN, <https://ctan.org/pkg/latex2e-help-texinfo>, 2026.
- [23] The L^AT_EX Project. “Extensive support for hypertext in L^AT_EX”. Available from CTAN, <https://www.ctan.org/pkg/hyperref>, 2026.
- [24] The L^AT_EX Project. “The expl3 package”. Available from CTAN, <https://www.ctan.org/pkg/l3kernel>, 2026.

- [25] The $\text{\LaTeX}$ Project. “The $\text{\LaTeX}$ 3 Interfaces”. Available from CTAN, <https://www.ctan.org/pkg/l3kernel>, 2026.
- [26] The $\text{\LaTeX}$ Project. “The $\text{\LaTeX}$ 2 ϵ sources”. Available from CTAN, <https://ctan.org/tex-archive/macros/latex/base>, 2026.
- [27] The $\text{\LaTeX}$ Project. “ $\text{\LaTeX}$ News, Issue 41, June 2025”. Available from CTAN, <https://ctan.org/tex-archive/macros/latex/base>, 2025.
- [28] The $\text{\LaTeX}$ Project. “ $\text{\LaTeX}$ for authors current version”. Available from CTAN, <https://ctan.org/pkg/latex-base>, 2026.
- [29] FISCHER, ULRIKE. “tagpdf – $\text{\LaTeX}$ kernel code for PDF tagging”. Available from CTAN, <https://www.ctan.org/pkg/tagpdf>, 2026.
- [30] The $\text{\LaTeX}$ Project. “latex-lab – $\text{\LaTeX}$ laboratory”. Available from CTAN, <https://www.ctan.org/pkg/latex-lab>, 2026.

9 Change history

v0.95 (ctan), 2026-08-11 – First public release.

10 Índice de la Documentación

Los números en cursiva indican las páginas donde se describe la entrada correspondiente.

C

Document class:

article 1

book 1

letter 1

report 1

Commands provide by **spintent**:

\setspintent 3

\spang 16

\spdate 11

\spdiv 13

\spdsym 12

\spmQ 16

\spmcd 14

\spmcm 15

\spmoney 6

\spmsym 12

\spmul 13

\spnD 13

\spnM 14

\spnZ 16

\spnewunit 6

\spnum 3

\spread 11

\spshort 7

\psiglo 11

\sptime 11

\spunitalias 6

\spunit 4

K

Keys for \spdiv provide by **spintent**:

cifras-sep 13

notation-sym 13

read-parens 13

read-period-sep 13

read-sign 13

read-sym 13

set-sym 13

wrap-parens 13

Keys for \spdsym provide by **spintent**:

prefix 12

read-sym 12

set-sym 12

suffix 12

Keys for \spmcd provide by **spintent**:

prefix 15

read-cmd 15

result 15

sample-arg 15

silent-cmd 15

upper-case 15

Keys for \spmcm provide by **spintent**:

prefix 15

read-cmd 15

result 15

sample-arg 15

silent-cmd 15

upper-case 15

Keys for \spmoney provide by **spintent**:

cifras-sep 7

currency 7

dolar-math 7

sub-units 7

Keys for \spmQ provide by **spintent**:

join-word 16

Keys for \spmsym provide by **spintent**:

not-print 12

prefix 12

read-sym 12

set-sym 12

suffix 12

Keys for \spmul provide by **spintent**:

cifras-sep 13

notation-sym 13

read-parens 13

read-period-sep 13

read-sign 13

read-sym 13

set-sym 13

wrap-parens 13

Keys for \spnD provide by **spintent**:

prefix 13

result-num 13

result 13

sample-arg 14

silent-cmd 14

Keys for \spnM provide by **spintent**:

prefix 14

result-num 14

result 14

sample-arg 14

silent-cmd 14

Keys for \spnum provide by **spintent**:

cifras-sep 3

notation-sym 4

read-period-sep 3

read-sign 3

Keys for \spnZ provide by **spintent**:

cifras-sep 16

read-parens 16

read-sign 16

wrap-parens 16

Keys for \spshort provide by **spintent**:

read-arg 9

small-caps 9

Keys for \psiglo provide by **spintent**:

print-era 11

Keys for \spunit provide by **spintent**:

cifras-sep 4

notation-sym 4

print-cdot 4

read-cdot 4

read-period-sep 4

read-slash 4

M

Lua module provide by **spintent**:

spintent.lua 6, 7, 9, 10

©2026 by Pablo González L

20 / 77

P	
Packages:	
babel-spanish	2
babel	2, 7
documentmetadata-support	1
enumext	1
expl3	3
fontspec	2
fourier-otf	2
hyperref	2
l3keys	3
latex-lab-mathintent	1
libertinus-otf	2
luamml	1
scontents	1
sourcecodepro	2
tagpdf	1, 7, 11
unicode-math	1

11 Implementation

The most recent publicly released version of `spintent` is available at CTAN: <https://www.ctan.org/pkg/spintent>. While general feedback via email is welcomed, specific bugs or feature requests should be reported through the issue tracker: <https://github.com/pablgonz/spintent/issues>.

- The documentation presented here is far from professional, it contains a lot of obvious information that to the eye of a TeXpert are superfluous, but, after so many years developing this project is the only way to remember what does what.

11.1 General conventions

Functions of the form `\__spintent_luafun_some:n` or `\__spintent_luafun_some:nn` are defined (created) in the Lua module `spintent.lua` and only their variants are declared in `expl3`.

Similarly, variables of the form `\l__spintent_luaset_some_tl` or `\l__spintent_luaset_some_str` are defined in the Lua module `spintent.lua` and only declared in `expl3`.

All variables and functions defined in this package are private and are NOT intended to work or be used by another package or module.

11.2 Initial set up

Start the DocStrip guards.

```
1 (*package)
```

Identify the internal prefix (L^AT_EX3 DocStrip convention) for l3doc class.

```
2 (@@=spintent)
```

11.3 Declaration of the package

First we will make sure we have a minimum (super updated) version of L^AT_EX to work correctly. **NeedsTeX-Format**

```
3 \NeedsTeXFormat{LaTeX2e}[2026-11-01]
```

Now declare the `spintent` package.

```
4 \ProvidesExplPackage {spintent} {2026-08-11} {0.95} {Spanish parse intents}
```

We check if the requirements for running the `spintent` package are met; that is: `\DocumentMetadata` must be active, Lua_TE_X must be running, and the `unicode-math` or `lua-unicode-math` package must be loaded. If neither is present, we will load `unicode-math`. If the conditions cannot be met, we will return a “fatal error”.

```
5 \IfDocumentMetadataF
6 {
7   \sys_if_engine luatex:TF
8   {
9     \msg_new:nnn { spintent } { package-not-load }
10    {
11      The~'#1'~package~will~be~loaded~as~a~dependency.
12    }
13    \IfPackageLoadedF { unicode-math }
14    {
15      \IfPackageLoadedF { lua-unicode-math }
16      {
17        \msg_info:nnn { spintent } { package-not-load } { unicode-math }
18        \RequirePackage{unicode-math}
19      }
20    }
21    \msg_new:nnn { spintent } { env-success }
22    { Everything~looks~correct,~spintent~will~make~the~intent! }
23    \msg_info:nn { spintent } { env-success }
24  }
25  {
26    \msg_new:nnnn { spintent } { fatal-engine-requires-luatex }
27    { The~spintent~package~requires~LuaLaTeX. }
28    { This~package~relies~on~Lua~module.~Compile~with~lualatex. }
29    \msg_fatal:nn { spintent } { fatal-engine-requires-luatex }
30  }
31 }
32 {
33   \msg_new:nnnn { spintent } { fatal-metadata-missing }
34   { \token_to_str:N \DocumentMetadata \c_space_tl is~missing. }
35   {
36     The~spintent~package~requires~tagged~PDF~active.\\
37     You~must~declare~\token_to_str:N \DocumentMetadata { tagging=on } ~
38   }
39   \msg_fatal:nn { spintent } { fatal-metadata-missing }
40 }
```

11.4 Some kernel variants

Non-standard kernel variants.

```
\str_if_eq:nVTF
\str_if_eq:nVF
\str_uppercase:V
\str_lowercase:V
\seq_set_from_clist:Ne
41 \cs_generate_variant:Nn \str_if_eq:nnTF { V }
42 \cs_generate_variant:Nn \str_if_eq:nnF { V }
43 \cs_generate_variant:Nn \str_uppercase:n { V }
44 \cs_generate_variant:Nn \str_lowercase:n { V }
45 \cs_generate_variant:Nn \seq_set_from_clist:Nn { Ne }
```

(End of definition for `\str_if_eq:nVTF` and others.)

11.5 Loading the Lua module and functions

Now we load the Lua module `spintent.lua` and generate the variants for the functions `\__spintent_luafun_spunit_lookup_alias:n` used by `\spunit` (§11.13), `\__spintent_luafun_spmoney_lookup_data:V` used by `\spmoney` (§11.15), `\__spintent_luafun_spmQ_scale_int:Vn` used by `\spmQ` (§11.28) and `\__spintent_luafun_clean_split_and_set:V` used by `\spnum` (§11.12) defined in it.

```
46 \lua_load_module:n { spintent.lua }
47 \cs_generate_variant:Nn \__spintent_luafun_spunit_lookup_alias:n { V }
48 \cs_generate_variant:Nn \__spintent_luafun_spmoney_lookup_data:n { V }
49 \cs_generate_variant:Nn \__spintent_luafun_spmQ_scale_int:nn { VV, Vn }
50 \cs_generate_variant:Nn \__spintent_luafun_clean_split_and_set:n { V }
```

(End of definition for `\__spintent_luafun_spunit_lookup_alias:V` and others.)

11.6 Internal copy of the unicode invisible separator

The function `\__spintent_invisible_sep:` is an internal copy of the Unicode invisible separator U-2063 that uses `\luamml_annotate:en` from the package `luamml` [2] and let `<mo>...</mo>` in the `MathML` structure embedded in the PDF output.

```
51 \cs_new_protected:Nn \__spintent_invisible_sep:
52 {
53   \luamml_annotate:en
54   {
55     core = {[0] = 'mo', intent=':silent', "^^^2063"}
56   }
57   {
58     \latelua{}
59   }
60 }
```

The token list constant `\c__spintent_invisible_sep_tl` is an internal copy of the Unicode invisible separator U-2063 using the primitive `\Umathchardef` from `LuaTeX` which leaves `<mi>...</mi>` in the structure `MathML` embedded in the output PDF.

```
61 \hook_gput_code:nnn { begindocument } { spintent }
62 {
63   \Umathchardef \c__spintent_invisible_sep_tl = "0 "0 "2063
64 }
```

Both copies are used for different purposes, the function `\__spintent_invisible_sep:` will be used to force the start of a `<mrow>` in the structure `MathML`, the token list constant `\c__spintent_invisible_sep_tl` will be used to inject text into the structure `MathML` when we use `\MathMLintent{<text>}`.

(End of definition for `\__spintent_invisible_sep:` and `\c__spintent_invisible_sep_tl`.)

11.7 Normalize argument and affixes

We normalize the `{<argument>}` to support affixes (prefixes/suffixes) by trimming leading and trailing spaces, converting internal spaces to hyphens, collapsing consecutive hyphens, and removing leading or trailing hyphens.

```
\__spintent_collapse_hyphens:N
65 \cs_new_protected:Nn \__spintent_collapse_hyphens:N
66 {
67   \tl_if_in:NnT #1 { -- }
68   {
69     \tl_replace_all:Nnn #1 { -- } { - }
70     \__spintent_collapse_hyphens:N #1
71   }
72 }
```

(End of definition for `\__spintent_collapse_hyphens:N`.)

```

\__spintent_sanitize_affix:Nn
\__spintent_sanitize_affix:cn

```

The function `\__spintent_sanitize_affix:Nn` checks if the $\langle argument \rangle$ passed in ‘#2’ is *blank*. If so, it clears the target token list variable passed in ‘#1’. Otherwise, trimming bounds, converting spaces ‘ ’ to hyphens ‘-’, collapsing multiple hyphens, and safely stripping leading/trailing hyphens. For example, if you pass a target variable and raw text:

```

\__spintent_sanitize_affix:Nn \l__spintent_spsomesym_prefix_tl{ foo bar baz }

```

The function will “store” the normalized string foo-bar-baz inside `\l__spintent_spsomesym_prefix_tl`, ready to be conditionally appended.

```

73 \cs_new_protected:Nn \__spintent_sanitize_affix:Nn
74 {
75   \tl_if_blank:nTF {#2}
76   { \tl_clear:N #1 }
77   {
78     \tl_set:Nx #1 {#2}
79     \tl_trim_spaces:N #1
80     \tl_replace_all:Nnn #1 { ~ } { - }
81     \__spintent_collapse_hyphens:N #1
82     \tl_put_left:Nn #1 { \q_mark }
83     \tl_put_right:Nn #1 { \q_stop }
84     \tl_replace_all:Nnn #1 { \q_mark - } { \q_mark }
85     \tl_replace_all:Nnn #1 { - \q_stop } { \q_stop }
86     \tl_remove_all:Nn #1 { \q_mark }
87     \tl_remove_all:Nn #1 { \q_stop }
88   }
89 }
90 \cs_generate_variant:Nn \__spintent_sanitize_affix:Nn { cn }

```

(End of definition for `\__spintent_sanitize_affix:Nn`.)

11.8 The command `\setspintent`

The `\setspintent` command is used to configure $\langle keys \rangle$ “globally” for the utilities provided by `spintent`.

11.8.1 Internal variables and error messages for `\setspintent`

```

\l__spintent_setspintent_input_arg_cmd_tl
unknown-command-setup

```

The variable `\l__spintent_setspintent_input_arg_cmd_tl` will store the command or command name passed to the “first” $\langle argument \rangle$ ‘#1’ of the command `\setspintent`.

```

91 \tl_new:N \l__spintent_setspintent_input_arg_cmd_tl

```

Error message if the $\langle argument \rangle$ ‘#1’ passed to `\spintent` is a “non-existent” command.

```

92 \msg_new:nnnn { spintent } { unknown-command-setup }
93 {
94   Cannot~configure~unknown~command~'\exp_not:c{#1}'.
95 }
96 {
97   You~attempted~to~use~\string\setspintent{#1}{...},~but~the~
98   macro~\exp_not:c{#1}~is~not~defined.~Please~check~for~typos.
99 }

```

(End of definition for `\l__spintent_setspintent_input_arg_cmd_tl` and `unknown-command-setup`.)

```

\__spintent_setspintent_set_keys:nn
\__spintent_setspintent_set_keys:en

```

The function `\__spintent_setspintent_set_keys:nn` will check if the command passed as $\langle argument \rangle$ exists in ‘#1’ and set the $\langle keys \rangle$ passed as $\langle argument \rangle$ in ‘#2’ to the correct path; otherwise, it will return an error.

```

100 \cs_new_protected:Nn \__spintent_setspintent_set_keys:nn
101 {
102   \cs_if_exist:cTF { #1 }
103   {
104     \keys_set:nn { spintent / #1 } { #2 }
105   }
106   {
107     \msg_error:nnn { spintent } { unknown-command-setup } { #1 }
108   }
109 }
110 \cs_generate_variant:Nn \__spintent_setspintent_set_keys:nn { en }

```

(End of definition for `\__spintent_setspintent_set_keys:nn`.)

```

\__spintent_setspintent_parse_args:nn
\__spintent_setspintent_parse_args:Vn

```

The function `\__spintent_setspintent_parse_args:nn` will check if the argument passed in #1 begins with ‘\’ or is just a *command name*. If it begins with ‘\’, it will remove it using `\cs_to_str:N` and then execute `\__spintent_setspintent_set_keys:en`. Otherwise, i.e., if ‘\’ was not passed, it will directly execute `\__spintent_setspintent_set_keys:nn`.

```

111 \cs_new_protected:Nn \__spintent_setspintent_parse_args:nn

```

```

112 {
113     \tl_if_single:nTF { #1 }
114     {
115         \token_if_cs:NTF #1
116         {
117             \__spintent_setspintent_set_keys:en { \cs_to_str:N #1 } { #2 }
118         }
119         {
120             \__spintent_setspintent_set_keys:nn { #1 } { #2 }
121         }
122     }
123     {
124         \__spintent_setspintent_set_keys:nn { #1 } { #2 }
125     }
126 }
127 \cs_generate_variant:Nn \__spintent_setspintent_parse_args:nn { Vn }

```

(End of definition for `\__spintent_setspintent_parse_args:nn`.)

`\setspintent` Now we define the user command `\setspintent`.

```

128 \NewDocumentCommand \setspintent { m m }
129 {
130     \tl_if_blank:nTF { #1 }
131     {
132         \msg_error:nnn { spintent } { unknown-command-setup } { <empty> }
133     }
134     {
135         \tl_set:Nn \__spintent_setspintent_input_arg_cmd_tl { #1 }
136         \tl_trim_spaces:N \__spintent_setspintent_input_arg_cmd_tl
137         \__spintent_setspintent_parse_args:Vn \__spintent_setspintent_input_arg_cmd_tl { #2 }
138     }
139 }

```

(End of definition for `\setspintent`. This function is documented on page 3.)

11.9 Handling unknown keys

`\__spintent_command_name_str`
`unknown-choice`
`cmd-key-unknown`
`cmd-key-value-unknown`

We define the variable `\__spintent_command_name_str` which will *store* the “name” of each command within the implementation along with the messages `unknown-choice` used by *⟨keys⟩* that use `.choice:n` in their implementation and the messages `cmd-key-unknown`, `cmd-key-value-unknown` used by the *⟨key⟩* `unknown` in commands that use `[⟨key = val⟩]`.

```

140 \str_new:N \__spintent_command_name_str
141 \msg_new:nnn { spintent } { unknown-choice }
142 {
143     The~value~'#3'~for~'#1'~key~is~invalid~use~('#2').
144 }
145 \msg_new:nnnn { spintent } { cmd-key-unknown }
146 {
147     The~key~'#1'~is~unknown~by~command~
148     \c_backslash_str \__spintent_command_name_str \c_space_tl
149     and~is~being~ignored~\msg_line_context:.
150 }
151 {
152     The~command~\c_backslash_str \__spintent_command_name_str \c_space_tl
153     does~not~have~a~key~called~'#1'.\\
154     Check~that~you~have~spelled~the~key~name~correctly.
155 }
156 \msg_new:nnnn { spintent } { cmd-key-value-unknown }
157 {
158     The~key~'#1=#2'~is~unknown~by~command~
159     \c_backslash_str \__spintent_command_name_str \c_space_tl
160     and~is~being~ignored~\msg_line_context:.
161 }
162 {
163     The~command~\c_backslash_str \__spintent_command_name_str \c_space_tl
164     does~not~have~a~key~called~'#1'.\\
165     Check~that~you~have~spelled~the~key~name~correctly.
166 }

```

(End of definition for `\__spintent_command_name_str` and others.)


```

\__spintent_unknown_keys:n
\__spintent_unknown_keys:nn

```

The internal functions `\__spintent_unknown_keys:n` and `\__spintent_unknown_keys:nn` used by the *⟨key⟩* unknown.

```

167 \cs_new_protected:Nn \__spintent_unknown_keys_cmd:n
168   {
169     \exp_args:NV \__spintent_unknown_keys_cmd:nn \l_keys_key_str {#1}
170   }
171 \cs_new_protected:Nn \__spintent_unknown_keys_cmd:nn
172   {
173     \tl_if_blank:nTF {#2}
174       { \msg_error:nnn { spintent } { cmd-key-unknown } {#1} }
175       { \msg_error:nnnn { spintent } { cmd-key-value-unknown } {#1} {#2} }
176   }

```

(End of definition for `\__spintent_unknown_keys:n` and `\__spintent_unknown_keys:nn`.)

11.10 Definition of core base variables

```

\__spintent_luafun_clean_split_and_set:n
\__spintent_luafun_clean_split_and_set:v

```

The function `\__spintent_luafun_clean_split_and_set:n` defined in the Lua module `spintent.lua`, is common to several commands in this implementation. It analyzes, processes, cleans and splits the *{⟨argument⟩}* passed to commands using internal functions defined in the Lua module `spintent.lua`. It isolates signs, integer components, decimal separators, decimal parte, decimal periodic part, exponents and final physical “units”, assigning them directly to `expl3` variables.

This function assigns the states of the variables ‘`_str`’ or ‘`_tl`’ described below to `true`, `false`, `error` or stores information according to the role it fulfills within the implementation.

(End of definition for `\__spintent_luafun_clean_split_and_set:n`.)

```

\__spintent_luaset_sign_tl
\__spintent_luaset_part_int_tl
\__spintent_luaset_dec_sep_tl
\__spintent_luaset_part_dec_tl
\__spintent_luaset_part_period_tl
\__spintent_luaset_has_integer_str
\__spintent_luaset_has_natural_str

```

The variable `\__spintent_luaset_sign_tl` stores the explicit ‘+’ or ‘−’ sign if present. The variable `\__spintent_luaset_part_int_tl` stores the raw digit sequence of the “integer part”. The variable `\__spintent_luaset_dec_sep_tl` stores the matched ‘,’ or ‘.’ used as the decimal separator. The variable `\__spintent_luaset_part_dec_tl` stores the raw digits of the “finite decimal part” that follow the separator, and the variable `\__spintent_luaset_part_period_tl` stores the digits of the “infinite periodic decimal part” that follow a semicolon ‘;’.

The variable `\__spintent_luaset_has_integer_str` store the state `true` when it is *only* an “integer”. The variable `\__spintent_luaset_has_natural_str` store the state `true` when it is *only* an “natural number”.

```

177 \tl_new:N \__spintent_luaset_sign_tl
178 \tl_new:N \__spintent_luaset_part_int_tl
179 \tl_new:N \__spintent_luaset_dec_sep_tl
180 \tl_new:N \__spintent_luaset_part_dec_tl
181 \tl_new:N \__spintent_luaset_part_period_tl
182 \str_new:N \__spintent_luaset_has_integer_str
183 \str_new:N \__spintent_luaset_has_natural_str

```

(End of definition for `\__spintent_luaset_sign_tl` and others.)

```

\__spintent_luaset_only_part_int_str
\__spintent_luaset_only_part_dec_str
\__spintent_luaset_only_part_period_str
\__spintent_luaset_format_part_int_str
\__spintent_luaset_format_part_dec_str

```

The literal entries of the numbers that we will pass to the *intent function* :`number` of `\MathMLintent` are stored in `\__spintent_luaset_only_part_int_str` for the “integer part”, in `\__spintent_luaset_only_part_dec_str` for the “finite decimal part”, and in `\__spintent_luaset_only_part_period_str` for the “infinite periodic decimal part”.

Formatted versions (`cifras-sep=true`) are stored in `\__spintent_luaset_format_part_int_str` for the “integer part” and `\__spintent_luaset_format_part_dec_str` for the “finite decimal part”.

```

184 \str_new:N \__spintent_luaset_only_part_int_str
185 \str_new:N \__spintent_luaset_only_part_dec_str
186 \str_new:N \__spintent_luaset_only_part_period_str
187 \str_new:N \__spintent_luaset_format_part_int_str
188 \str_new:N \__spintent_luaset_format_part_dec_str

```

(End of definition for `\__spintent_luaset_only_part_int_str` and others.)

```

\__spintent_luaset_millions_str
\__spintent_luaset_decimal_period_str
\__spintent_luaset_not_decimal_period_str
\__spintent_luaset_not_decimal_not_period_str
\__spintent_luaset_dec_is_all_zeros_str
\__spintent_luaset_has_exponent_str
\__spintent_luaset_exponent_tl

```

The flag variable `\__spintent_luaset_millions_str` indicates whether the integer part translates exactly to a multiple of one million when passed to `\MathMLintent`. To properly structure the reading and writing of numbers in `MathML`, we use the flag variables `\__spintent_luaset_decimal_period_str`, `\__spintent_luaset_not_decimal_period_str`, and `\__spintent_luaset_not_decimal_not_period_str`. The flag variable `\__spintent_luaset_dec_is_all_zeros_str` monitors whether the “finite decimal part” is completely zero (useful for “pure periodic decimals”).

The flag variable `\__spintent_luaset_has_exponent_str` indicates whether active “scientific notation” was detected at the end of the entered number, passed with ‘e’ or ‘E’. If true, it will store the digits of the exponent in `\__spintent_luaset_exponent_tl`.

```

189 \str_new:N \__spintent_luaset_millions_str

```

```

190 \str_new:N \__spintent_luaset_decimal_period_str
191 \str_new:N \__spintent_luaset_not_decimal_period_str
192 \str_new:N \__spintent_luaset_not_decimal_not_period_str
193 \str_new:N \__spintent_luaset_dec_is_all_zeros_str
194 \str_new:N \__spintent_luaset_has_exponent_str
195 \tl_new:N \__spintent_luaset_exponent_tl

```

(End of definition for `\__spintent_luaset_millions_str` and others.)

```

\__spintent_luaset_has_units_str
\__spintent_luaset_status_str
\__spintent_luaset_arg_above_tl
\__spintent_luaset_arg_below_tl
\__spintent_luaset_denom_has_number_str

```

Since the function `\__spintent_luafun_clean_split_and_set:n` is common to `\spnum` (§11.12) and `\spunit` (§11.13) (as well as others), it determines the following unit-related variables after processing the `{⟨arguments⟩}` via the Lua module `spintent.lua`.

First, it checks if there is any residual text *after* the processed number that is a candidate for physical “units”. It sets the flag variable `\__spintent_luaset_has_units_str` accordingly. Following this, it verifies if this text contains more than one “/”. It then sets the variable `\__spintent_luaset_status_str` to `valid` or `multislash`. If the structure is `valid`, it splits the unit into `\__spintent_luaset_arg_above_tl` and `\__spintent_luaset_arg_below_tl`. Finally, it analyzes `\__spintent_luaset_arg_below_tl`; if the denominator starts with a number, it sets `\__spintent_luaset_denom_has_number_str` to `true`, otherwise to `false`.

```

196 \str_new:N \__spintent_luaset_has_units_str
197 \str_new:N \__spintent_luaset_status_str
198 \tl_new:N \__spintent_luaset_arg_above_tl
199 \tl_new:N \__spintent_luaset_arg_below_tl
200 \str_new:N \__spintent_luaset_denom_has_number_str

```

(End of definition for `\__spintent_luaset_has_units_str` and others.)

11.11 Common messages for text mode and math mode

Messages shared by the different commands for text mode or math mode.

```

201 \msg_new:nnnn { spintent } { need-math-mode }
202 {
203   Math~mode~required~for~'!#1'.
204 }
205 {
206   Please~wrap~the~command~inside~$...$~or~a~math~environment.
207 }
208 \msg_new:nnnn { spintent } { need-text-mode }
209 {
210   Text~mode~required~for~'!#1'.
211 }
212 {
213   Please~move~the~command~outside~of~$...$~or~the~math~environment.
214 }

```

11.12 The command `\spnum`

The user command `\spnum` is *first* of “the big four” provided by the `spintent` package and is fundamental to the implementation of other commands. It handles the processing of numerical input, scientific notation, and units of measurement, using the Lua module `spintent.lua`, which is responsible for separating and assigning variables defined in `expl3`.

From the `expl3` side, we handle the printing logic and the `intent` functions passed to `\MathMLintent` to achieve a valid `MathML` structure that is accepted by `MathCAT`.

11.12.1 Definition of keys and variables for `\spnum`

```

\__spintent_spnum_read_terse_bool
\__spintent_spnum_read_period_sep_str
\__spintent_spnum_notation_sym_tl
\__spintent_spnum_intent_read_sign_str
\__spintent_spnum_intent_read_dec_sep_str

```

The variable `\__spintent_spnum_read_terse_bool` handled by the key `read-sign`, controls the `intent` passed to `\MathMLintent` to spoken the ‘+’ or ‘−’. If set to `clear`, it will say “*positivo*” or “*negativo*” *after* reading the number; if set to `terse`, it will say “*más*” or “*menos*” *before* reading the number.

The variable `\__spintent_spnum_read_period_sep_str` handled by the key `read-period-sep`, controls how the decimal separator ‘;’ (periodic part) is printed and spoken when the `{⟨argument⟩}` is a *pure decimal periodic* and is passed the `intent` to `\MathMLintent`. If set to `coma`, it will say “*coma*” and print ‘,’; if set to `punto` it will say “*punto*” and print ‘.’.

The variable `\__spintent_spnum__notation_sym_tl`, handled by the key `notation-sym`, determines whether ‘×’ will be printed when set to `times` or ‘·’ when set to `cdot`.

The variable `\__spintent_spnum_intent_read_sign_str` will *store* the spoken translation used in the `intent` function passed to `\MathMLintent` for the ‘+’ or ‘−’ according to the value passed to the key `read-sign`.

The variable `\__spintent_spnun_intent_read_dec_sep_str` stores the spoken translation that will be used in the `\intent` function passed to `\MathMLintent` for reading the decimal separation as a “coma” or a “punto” according to the value found in the variable `\__spintent_luaset_dec_sep_tl`.

```

215 \bool_new:N \__spintent_spnun_read_terse_bool
216 \str_new:N \__spintent_spnun_read_period_sep_str
217 \tl_new:N \__spintent_spnun_notation_sym_tl
218 \str_new:N \__spintent_spnun_intent_read_sign_str
219 \str_new:N \__spintent_spnun_intent_read_dec_sep_str

```

(End of definition for `\__spintent_spnun_read_terse_bool` and others.)

Now we add the keys `cifras-sep`, `read-sign`, `read-period-sep` and `notation-sym`.

```

cifras-sep
read-sign
read-period-sep
notation-sym
220 \keys_define:nn { spintent/spnun }
221 {
222   cifras-sep .bool_set:N = \__spintent_spnun_cifras_sep_bool,
223   cifras-sep .initial:n = true,
224   cifras-sep .value_required:n = true,
225   read-sign .choices:nn =
226     { terse , clear }
227   {
228     \int_case:nn { \_keys_choice_int }
229     {
230       {1} { \bool_set_true:N \__spintent_spnun_read_terse_bool }
231       {2} { \bool_set_false:N \__spintent_spnun_read_terse_bool }
232     }
233   },
234   read-sign .initial:n = terse,
235   read-sign .value_required:n = true,
236   read-sign / unknown .code:n =
237     \msg_error:nneee { spintent } { unknown-choice }
238     { read-sign } { terse , ~clear } { \exp_not:n {#1} },
239   read-period-sep .choices:nn =
240     { coma , punto }
241     {
242       \int_case:nn { \_keys_choice_int }
243       {
244         {1} { \str_set:Nn \__spintent_spnun_read_period_sep_str { coma } }
245         {2} { \str_set:Nn \__spintent_spnun_read_period_sep_str { punto } }
246       }
247     },
248   read-period-sep .initial:n = coma,
249   read-period-sep .value_required:n = true,
250   read-period-sep / unknown .code:n =
251     \msg_error:nneee { spintent } { unknown-choice }
252     { read-period-sep } { coma , ~punto } { \exp_not:n {#1} },
253   notation-sym .choices:nn =
254     { times , cdot }
255     {
256       \int_case:nn { \_keys_choice_int }
257       {
258         {1} { \tl_set:Nn \__spintent_spnun_notation_sym_tl { \times } }
259         {2} { \tl_set:Nn \__spintent_spnun_notation_sym_tl { \cdot } }
260       }
261     },
262   notation-sym .initial:n = cdot,
263   notation-sym .value_required:n = true,
264   notation-sym / unknown .code:n =
265     \msg_error:nneee { spintent } { unknown-choice }
266     { notation-sym } { times , ~cdot } { \exp_not:n {#1} },
267   unknown .code:n =
268     {
269       \str_set:Nn \__spintent_command_name_str { spnun }
270       \__spintent_unknown_keys_cmd:n {#1}
271     },
272 }

```

(End of definition for `cifras-sep` and others.)

11.12.2 Internal functions for `\spnun`

```

\__spintent_spnun_render_part:n
\__spintent_spnun_render_int:
\__spintent_spnun_render_dec:

```

The function `\__spintent_spnun_render_part:n` processes the `{\argument}` passed in ‘#1’ used by the functions `\__spintent_spnun_render_int:` and `\__spintent_spnun_render_dec:` to define the

intent function :**number** for the integer and decimal part passed to `\MathMLintent`. It evaluates whether the key `cifras-sep` is active or not.

```

273 \cs_new_protected:Nn \__spintent_snum_render_part:n
274 {
275   \MathMLintent { \str_use:c { \__spintent_luaset_only_part_#1_str } :number }
276   {
277     \bool_if:NTF \__spintent_snum_cifras_sep_bool
278     { \str_use:c { \__spintent_luaset_format_part_#1_str } }
279     { \tl_use:c { \__spintent_luaset_part_#1_tl } }
280   }
281 }
282 \cs_new_protected:Nn \__spintent_snum_render_int:
283 {
284   \__spintent_snum_render_part:n { int }
285 }
286 \cs_new_protected:Nn \__spintent_snum_render_dec:
287 {
288   \__spintent_snum_render_part:n { dec }
289 }

```

(End of definition for `\__spintent_snum_render_part:n`, `\__spintent_snum_render_int:`, and `\__spintent_snum_render_dec:`.)

`\__spintent_snum_render_only_period:`
`\__spintent_snum_print_period_bar:`
`\__spintent_snum_render_period_sep:`

The function `\__spintent_snum_render_only_period:` isolates the “*infinite periodic decimal part*” from the “*finite decimal part*”, assigning the *intent function* :**number** passed to `\MathMLintent` which will be used by `\MathMLarg` in the function `\__spintent_snum_print_period_bar:`.

```

290 \cs_new_protected:Nn \__spintent_snum_render_only_period:
291 {
292   \MathMLintent { \__spintent_luaset_only_part_period_str :number }
293   {
294     \__spintent_luaset_part_period_tl
295   }
296 }

```

The function `\__spintent_snum_print_period_bar:` inserts the function `\__spintent_snum_render_only_period:` as an argument within the *intent function* `_periódico:postfix($dp)` passed to `\MathMLintent`, generating the correct spoken for the “*infinite periodic decimal part*”.

```

297 \cs_new_protected:Nn \__spintent_snum_print_period_bar:
298 {
299   \MathMLintent { _periódico:postfix($dp) }
300   {
301     {
302       \overline
303       {
304         \MathMLarg { dp } { \__spintent_snum_render_only_period: }
305       }
306     }
307   }
308 }

```

The function `\__spintent_snum_render_period_sep:` sets how the decimal separator handled by the key `read-period-sep` will be printed and spoken when a “*pure periodic decimal*” is passed.

```

309 \cs_new_protected:Nn \__spintent_snum_render_period_sep:
310 {
311   \str_if_eq:VnTF \__spintent_snum_read_period_sep_str { coma }
312   {
313     \MathMLintent { _coma:prefix($pp) }
314     {
315       \mathord{,}
316       \MathMLarg { pp } { \__spintent_snum_print_period_bar: }
317     }
318   }
319   {
320     \MathMLintent { _punto:prefix($pp) }
321     {
322       \mathord{.}
323       \MathMLarg { pp } { \__spintent_snum_print_period_bar: }
324     }
325   }
326 }

```

(End of definition for `\__spintent_snum_render_only_period:`, `\__spintent_snum_print_period_bar:`, and `\__spintent_snum_render_period_sep:`.)

`\__spintent_snum_render_decimal_sep:`

The function `\__spintent_snum_render_decimal_sep:` parses the value of the variable `\l__spintent_luaset_dec_sep_tl` and sets the variable `\l__spintent_snum_intent_read_dec_sep_str` with the function `_coma:prefix($dp)` for reading a “coma” or `_punto:prefix($dp)` for reading a “punto” to `\MathMLintent`. We print the decimal separator stored in `\l__spintent_luaset_dec_sep_tl` using `\mathord` to remove spaces around it and then use the command `\MathMLarg` where we insert `\__spintent_invisible_sep:` for compatibility with MathML and finally call the function `\__spintent_snum_render_dec:` to print the finite decimal part.

```

327 \cs_new_protected:Nn \__spintent_snum_render_decimal_sep:
328 {
329   \str_if_eq:NnTF \l__spintent_luaset_dec_sep_tl { , }
330   { \str_set:Nn \l__spintent_snum_intent_read_dec_sep_str { _coma:prefix($dp) } }
331   { \str_set:Nn \l__spintent_snum_intent_read_dec_sep_str { _punto:prefix($dp) } }
332   \MathMLintent { \l__spintent_snum_intent_read_dec_sep_str }
333   {
334     \mathord { \l__spintent_luaset_dec_sep_tl }
335     \MathMLarg { dp }
336     {
337       \__spintent_invisible_sep:
338       \__spintent_snum_render_dec:
339     }
340   }
341   % period part
342   \str_if_eq:NnTF \l__spintent_luaset_decimal_period_str { true }
343   {
344     \str_if_eq:NnTF \l__spintent_luaset_dec_is_all_zeros_str { true }
345     {
346       \__spintent_invisible_sep:
347       \__spintent_snum_print_period_bar:
348     }
349     {
350       \MathMLintent { _con:prefix($pnum) }
351       {
352         \__spintent_invisible_sep:
353         \MathMLarg { pnum }
354         {
355           \__spintent_snum_print_period_bar:
356         }
357       }
358     }
359   }
360 }

```

(End of definition for `\__spintent_snum_render_decimal_sep:`.)

`\__spintent_snum_print_part_dec:`
`\__spintent_snum_print_part_period:`

The function `\__spintent_snum_print_part_dec:` checks if a decimal part exists. If it does, it determines whether the decimal separator is a comma or a period to set the correct speech intent (“coma” or “punto”). It then prints the separator, renders the decimal digits, and if a periodic sequence follows, it decides how to attach the periodic bar.

The function `\__spintent_snum_print_part_period:` handles cases where a periodic part exists. It evaluates if it should render the periodic separator directly, particularly when the number has a periodic part but lacks a standard decimal part.

```

361 \cs_new_protected:Nn \__spintent_snum_print_part_dec:
362 {
363   \tl_if_empty:NF \l__spintent_luaset_part_dec_tl
364   {
365     \__spintent_snum_render_decimal_sep:
366   }
367 }
368 \cs_new_protected:Nn \__spintent_snum_print_part_period:
369 {
370   \tl_if_empty:NF \l__spintent_luaset_part_period_tl
371   {
372     \str_if_eq:NnTF \l__spintent_luaset_not_decimal_period_str { true }
373     { \__spintent_snum_render_period_sep: }
374     {
375       \tl_if_empty:NT \l__spintent_luaset_part_dec_tl
376       {

```

```

377         \__spintent_snum_render_period_sep:
378     }
379 }
380 }
381 }

```

(End of definition for `\__spintent_snum_print_part_dec:` and `\__spintent_snum_print_part_period:`)

```

\__spintent_snum_print_integer:
\__spintent_snum_print_num_start:
\__spintent_snum_print_number_core:

```

The function `\__spintent_snum_print_integer:` checks if the number consists only of an integer. If so, it simply renders it. If a decimal or periodic part exists, it sets up a `MathML` intent to link the integer part (respecting digit separators if enabled) to the subsequent decimal or periodic functions.

The function `\__spintent_snum_print_num_start:` checks if an explicit sign is present. If it finds one, it evaluates the terse reading flag to decide whether the sign should be read as a prefix (“*más*” or “*menos*”) or as a postfix (“*positivo*” or “*negativo*”), and wraps the intent around the rest of the number.

The function `\__spintent_snum_print_number_core:` acts as the main wrapper. It triggers the sign and base number evaluation, and then checks if a scientific exponent is present. If it is, it appends the correct base-10 notation symbol and exponent to complete the sequence.

 Note for `\__spintent_invisible_sep:`.

```

382 \cs_new_protected:Nn \__spintent_snum_print_integer:
383 {
384     \str_if_eq:VnTF \__spintent_luaset_not_decimal_not_period_str { true }
385     { \__spintent_snum_render_int: }
386     {
387         \MathMLintent { \__spintent_luaset_only_part_int_str :number:prefix($next) }
388         {
389             \bool_if:NTF \__spintent_snum_cifras_sep_bool
390             { \__spintent_luaset_format_part_int_str }
391             { \__spintent_luaset_part_int_tl }
392             \MathMLarg { next }
393             {
394                 \tl_if_empty:NTF \__spintent_luaset_part_dec_tl
395                 { \__spintent_snum_print_part_period: }
396                 { \__spintent_snum_print_part_dec: }
397             }
398         }
399     }
400 }
401 \cs_new_protected:Nn \__spintent_snum_print_num_start:
402 {
403     \tl_if_empty:NTF \__spintent_luaset_sign_tl
404     {
405         \__spintent_invisible_sep: % need
406         \__spintent_snum_print_integer:
407     }
408     {
409         \bool_if:NTF \__spintent_snum_read_terse_bool
410         {
411             \str_if_eq:VnTF \__spintent_luaset_sign_tl { + }
412             { \str_set:Nn \__spintent_snum_intent_read_sign_str { _más:prefix($n) } }
413             { \str_set:Nn \__spintent_snum_intent_read_sign_str { _menos:prefix($n) } }
414         }
415         {
416             \str_if_eq:VnTF \__spintent_luaset_sign_tl { + }
417             { \str_set:Nn \__spintent_snum_intent_read_sign_str { _positivo:postfix($n) } }
418             { \str_set:Nn \__spintent_snum_intent_read_sign_str { _negativo:postfix($n) } }
419         }
420         \__spintent_invisible_sep: % need
421         \MathMLintent { \__spintent_snum_intent_read_sign_str }
422         {
423             \__spintent_luaset_sign_tl
424             \MathMLarg { n } { \__spintent_snum_print_integer: }
425         }
426     }
427 }
428 \cs_new_protected:Nn \__spintent_snum_print_number_core:
429 {
430     \__spintent_snum_print_num_start:
431     \str_if_eq:VnT \__spintent_luaset_has_exponent_str { true }
432     {
433         \MathMLintent { _por } { \__spintent_snum_notation_sym_tl }

```



```

434         10^{\l__spintent_luaset_exponent_tl }
435     }
436 }

```

(End of definition for `\__spintent_snum_print_integer:`, `\__spintent_snum_print_num_start:`, and `\__spintent_snum_print_number_core:`.)

11.12.3 Definition of error messages

spnum-has-units
spnum-no-digits

We define two error messages for the `\spnum` command. The first “error” triggers if the user includes “units” in the $\langle argument \rangle$ passed in ‘#1’, advising them to use the `\spunit` command instead. The second “error” is raised if the provided $\langle argument \rangle$ contains no numerical digits.

```

437 \msg_new:nnnn { spintent } { spnum-has-units }
438 {
439     The~\token_to_str:N \spnum \c_space_tl command~does~not~accept~units~('#1').
440 }
441 {
442     Use~\token_to_str:N \spunit \c_space_tl if~you~need~to~print~numbers~with~units.
443 }
444 \msg_new:nnnn { spintent } { spnum-no-digits }
445 {
446     The~\token_to_str:N \spnum \c_space_tl command~requires~a~valid~numerical~value.
447 }
448 {
449     You~provided~'#1',~which~contains~no~digits.
450 }

```

(End of definition for `spnum-has-units` and `spnum-no-digits`.)

11.12.4 Implementation of the `\spnum` command

`\spnum`

The public user command `\spnum` accepts an optional $[key = val]$ argument in ‘#1’ and a mandatory $\langle rational number \rangle$ in ‘#2’. It first checks if it is executed inside math mode, raising an error if it is not. If correct, it opens a local group to apply the options and passes the input to Lua for parsing via `\__spintent_luafun_clean_split_and_set:n`.

Before printing, it validates the extracted data. It throws an error if it finds trailing units in the input or if the integer part is completely empty (no valid digits). If all checks pass, it calls `\__spintent_snum_print_number_core:` to render the final number, and finally closes the group.

```

451 \NewDocumentCommand \spnum { 0{} m }
452 {
453     \mode_if_math:TF
454     {
455         \group_begin:
456         \keys_set:nn { spintent/spnum } { #1 }
457         \__spintent_luafun_clean_split_and_set:n { \exp_not:n { #2 } }
458         \str_if_eq:VnTF \l__spintent_luaset_has_units_str { true }
459         { \msg_error:nnn { spintent } { spnum-has-units } { #2 } }
460         {
461             \tl_if_empty:NTF \l__spintent_luaset_part_int_tl
462             { \msg_error:nnn { spintent } { spnum-no-digits } { #2 } }
463             { \__spintent_snum_print_number_core: }
464         }
465         \group_end:
466     }
467     { \msg_error:nnn { spintent } { need-math-mode } { \spnum } }
468 }

```

(End of definition for `\spnum`. This function is documented on page 3.)

11.13 The command `\spunit`

The user command `\spunit` is the *second* of “the big four” provided by the `spintent` package and is used to parse and print physical units and angular systems. It sends the $\langle argument \rangle$ to Lua module `spintent.lua` via `\__spintent_luafun_clean_split_and_set:n` to separate and clean the $\langle argument \rangle$.

This ensures that all “units” (such as SI fractions, products, or sexagesimal angles) are processed consistently by the `spintent.lua`.

11.13.1 Definition of variables

These string variables store the data returned by the Lua module `spintent.lua`. They hold the canonical “unit symbol”, the dictionary lookup status, the correct speech text for screen readers, and specific flags that indicate if the “unit” is compact or represents a sexagesimal angle.

```

469 \str_new:N \l__spintent_spunit_luaset_canonical_str
470 \str_new:N \l__spintent_spunit_luaset_lookup_status_str

```

```

471 \str_new:N \l__spintent_spunit_luaset_read_str
472 \str_new:N \l__spintent_spunit_luaset_is_compact_str
473 \str_new:N \l__spintent_spunit_luaset_compact_exp_str
474 \str_new:N \l__spintent_spunit_luaset_is_sexagesimal_str
475 \str_new:N \l__spintent_spunit_luaset_status_str

```

(End of definition for `\l__spintent_spunit_luaset_canonical_str` and others.)

These variables are used to process and format the “unit”. The sequences `\l__spintent_spunit_mult_seq` and `\l__spintent_spunit_exp_seq` split compound “units” at multiplication ‘`*`’ or exponent ‘`^`’ boundaries.

The boolean variables control specific rendering options, such as whether the multiplication dot ‘`\cdot`’ is explicitly read aloud or silently skipped.

```

476 \tl_new:N \l__spintent_spunit_exponent_tl
477 \tl_new:N \l__spintent_spunit_unit_name_tl
478 \str_new:N \l__spintent_spunit_read_slash_str
479 \seq_new:N \l__spintent_spunit_exp_seq
480 \seq_new:N \l__spintent_spunit_mult_seq
481 \bool_new:N \l__spintent_spunit_is_mult_bool
482 \bool_new:N \l__spintent_spunit_read_cdod_bool

```

(End of definition for `\l__spintent_spunit_exponent_tl` and others.)

11.13.2 Definition of error messages

We define three error messages for the `\spunit` command. The first “error” triggers if the “unit expression” provided in ‘`#1`’ is unknown or malformed. The second “error” occurs if no “units” are found in the $\langle \textit{argument} \rangle$, advising the user to use `\spnum` for pure numbers. The third “error” is raised if a numeric coefficient is illegally placed in the denominator of an inline fraction (after a slash).

```

483 \msg_new:nnnn { spintent } { spunit-invalid-unit }
484 {
485   The~expression~'#1'~contains~an~unknown~or~malformed~unit~structure.
486 }
487 {
488   Check~spelling~or~verify~the~unit~is~defined~in~the~dictionary.
489 }
490 \msg_new:nnnn { spintent } { spunit-no-units }
491 {
492   The~\token_to_str:N \spunit \c_space_tl command~requires~at~least~one~unit~('#1').
493 }
494 {
495   Use~\token_to_str:N \spnum \c_space_tl if~you~only~need~to~format~a~pure~number.
496 }
497 \msg_new:nnnn { spintent } { inline-numeric-denominator }
498 {
499   Illegal~numeric~coefficient~in~inline~denominator~'#1'.
500 }
501 {
502   The~inline~mode~of~\token_to_str:N \spunit\ does~not~allow~numbers~
503   after~the~slash~(/).
504 }

```

(End of definition for `spunit-invalid-unit`, `spunit-no-units`, and `inline-numeric-denominator`.)

11.13.3 Definition of keys for `\spunit`

Now we add the keys `print-cdot`, `read-cdot`, `read-slash`, `cifras-sep`, `read-period-sep` and `notation-sym`.

```

505 \keys_define:nn { spintent/spunit }
506 {
507   print-cdot      .bool_set:N = \l__spintent_spunit_print_cdod_bool,
508   print-cdot      .initial:n  = false,
509   print-cdot      .value_required:n = true,
510   read-cdot       .bool_set:N = \l__spintent_spunit_read_cdod_bool,
511   read-cdot       .initial:n  = false,
512   read-cdot       .value_required:n = true,
513   read-slash      .choices:nn =
514     { por , partido-por }
515   {
516     \int_case:nn { \l_keys_choice_int }
517     {
518       {1} { \str_set:Nn \l__spintent_spunit_read_slash_str { por } }
519       {2} { \str_set:Nn \l__spintent_spunit_read_slash_str { partido-por } }
520     }

```

```

521     },
522     read-slash      .initial:n = por,
523     read-slash      .value_required:n = true,
524     read-slash / unknown .code:n =
525       \msg_error:nneee { spintenc } { unknown-choice }
526       { read-slash } { por, ~partido-por } { \exp_not:n {#1} },
527     cifras-sep      .meta:nn = { spintenc/spnum } { cifras-sep = {#1} },
528     cifras-sep      .value_required:n = true,
529     read-period-sep .meta:nn = { spintenc/spnum } { read-period-sep = {#1} },
530     read-period-sep .value_required:n = true,
531     notation-sym     .meta:nn = { spintenc/spnum } { notation-sym = {#1} },
532     notation-sym     .value_required:n = true,
533     unknown          .code:n =
534       {
535         \str_set:Nn \l__spintenc_command_name_str { spunit }
536         \l__spintenc_unknown_keys_cmd:n {#1}
537       },
538   }

```

(End of definition for read-cdot and others.)

11.13.4 Internal functions for \spunit

__spintenc_spunit_format_canonical:
 __spintenc_spunit_process_base_exp:n

The function `\__spintenc_spunit_format_canonical:` handles the visual formatting of the “unit symbol”. It specifically checks for the *liter symbol* ‘ℓ’ to print it as `\ell`, and formats all other valid “units” in standard upright text using `\mathrm`.

The function `\__spintenc_spunit_process_base_exp:n` evaluates a “single unit” along with its exponent. It first looks up the unit ‘#1’ in the dictionary. If the “unit” is not found, it flags it as invalid. If found, it handles the spacing and multiplication dot ‘\cdot’ intents, and then renders the base unit and its exponent (either using a pre-defined compact exponent from Lua or by manually splitting the string at the ‘^’ symbol).

```

539 \cs_new_protected:Nn \__spintenc_spunit_format_canonical:
540 {
541   \str_case:VnF \l__spintenc_spunit_luaaset_canonical_str
542   {
543     { ℓ } { \ell }
544   }
545   { \mathrm { \l__spintenc_spunit_luaaset_canonical_str } }
546 }
547 \cs_new_protected:Nn \__spintenc_spunit_process_base_exp:n
548 {
549   \tl_clear:N \l__spintenc_spunit_exponent_tl
550   \__spintenc_luafun_spunit_lookup_alias:n { #1 }
551   \str_if_eq:VnTF \l__spintenc_spunit_luaaset_lookup_status_str { notfound }
552   {
553     \str_set:Nn \l__spintenc_luaaset_status_str { invalid }
554   }
555   {
556     \bool_if:NF \l__spintenc_spunit_is_mult_bool
557     {
558       \bool_if:NTF \l__spintenc_spunit_print_cdot_bool
559       {
560         \bool_if:NTF \l__spintenc_spunit_read_cdot_bool
561         { \MathMLintent { por } { \cdot } }
562         { \MathMLintent { :silent } { \cdot } }
563       }
564       {
565         \bool_if:NTF \l__spintenc_spunit_read_cdot_bool
566         { \MathMLintent { por } { \, } }
567         { \, }
568       }
569     }
570     \str_if_eq:VnTF \l__spintenc_spunit_luaaset_is_compact_str { true }
571     {
572       {
573         \MathMLintent { \l__spintenc_spunit_luaaset_read_str }
574         {
575           \__spintenc_spunit_format_canonical:
576         }
577       }
578       ^ { \l__spintenc_spunit_luaaset_compact_exp_str }
579     }
580   }

```

```

581         \seq_set_split:Nnn \l__spintent_spunit_exp_seq { ^ } { #1 }
582         \seq_pop_left:NN \l__spintent_spunit_exp_seq \l__spintent_spunit_unit_name_tl
583         \seq_if_empty:NF \l__spintent_spunit_exp_seq
584         {
585             \seq_pop_left:NN \l__spintent_spunit_exp_seq \l__spintent_spunit_exponent_tl
586         }
587         \MathMLintent { \l__spintent_spunit_luaset_read_str }
588         {
589             \__spintent_spunit_format_canonical:
590         }
591         \tl_if_empty:NF \l__spintent_spunit_exponent_tl
592         {
593             ^ { \l__spintent_spunit_exponent_tl }
594         }
595     }
596     \bool_set_false:N \l__spintent_spunit_is_mult_bool
597 }
598 }

```

(End of definition for `\__spintent_spunit_format_canonical:` and `\__spintent_spunit_process_base_exp:n`.)

`\__spintent_spunit_process_mult:n`
`\__spintent_spunit_process_mult:V`

The function `\__spintent_spunit_process_mult:n` handles compound “units” joined by an explicit multiplication star `*`. It splits the $\{\langle argument \rangle\}$ at each “star symbol” and applies `\__spintent_spunit_process_base_exp:n` to evaluate and render each resulting unit individually.

```

599 \cs_new_protected:Nn \__spintent_spunit_process_mult:n
600 {
601     \bool_set_true:N \l__spintent_spunit_is_mult_bool
602     \seq_set_split:Nnn \l__spintent_spunit_mult_seq { * } { #1 }
603     \seq_map_function:NN \l__spintent_spunit_mult_seq \__spintent_spunit_process_base_exp:n
604 }
605 \cs_generate_variant:Nn \__spintent_spunit_process_mult:n { V }

```

(End of definition for `\__spintent_spunit_process_mult:n`.)

`\__spintent_print_spunit_hierarchy:`

The function `\__spintent_print_spunit_hierarchy:` handles the layout of inline “unit” fractions. It first processes the “units” before the slash (the numerator). If a denominator exists, it prints the slash / ‘/’ with its corresponding speech intent, and then processes the remaining “units”.

```

606 \cs_new_protected:Nn \__spintent_print_spunit_hierarchy:
607 {
608     \tl_if_empty:NF \l__spintent_luaset_arg_above_tl
609     {
610         \__spintent_spunit_process_mult:V \l__spintent_luaset_arg_above_tl
611         \tl_if_empty:NF \l__spintent_luaset_arg_below_tl
612         {
613             \MathMLintent { \l__spintent_spunit_read_slash_str } { / }
614             \__spintent_spunit_process_mult:V \l__spintent_luaset_arg_below_tl
615         }
616     }
617 }

```

(End of definition for `\__spintent_print_spunit_hierarchy:.`)

`\__spintent_spunit_safe_exec:n`

The function `\__spintent_spunit_safe_exec:n` performs the main validation and layout for “units”. It first checks if the $\{\langle argument \rangle\}$ has units, is a valid expression, and does not contain illegal numbers in the denominator, throwing the appropriate “errors” if any check fails.

If the unit is valid, it checks if it is a sexagesimal angle (degrees, minutes, or seconds). For sexagesimal “units”, it prints the number and the symbol “without any space in between”, applying a slight negative space ‘\!’ for arcseconds. For standard units, it prints the number followed by a thin space ‘\,’ and then processes the unit sequence.

```

618 \cs_new_protected:Nn \__spintent_spunit_safe_exec:n
619 {
620     \str_if_eq:VnTF \l__spintent_luaset_has_units_str { false }
621     { \msg_error:nnn { spintent } { spunit-no-units } { #1 } }
622     {
623         \str_if_eq:VnTF \l__spintent_luaset_status_str { valid }
624         {
625             \str_if_eq:VnTF
626             \l__spintent_luaset_denom_has_number_str { true }
627             {
628                 \msg_error:nnn { spintent }
629                 { inline-numeric-denominator } { #1 }

```

```

630     }
631   {
632     \__spintent_luafun_spunit_lookup_alias:V \__spintent_luaaset_arg_above_tl
633     \str_if_eq:VnTF \__spintent_spunit_luaaset_is_sexagesimal_str { true }
634     {
635       \tl_if_empty:NF \__spintent_luaaset_part_int_tl
636       {
637         \__spintent_spnum_print_number_core:
638       }
639       \MathMLintent { \__spintent_spunit_luaaset_read_str }
640       {
641         \mathord
642         {
643           \str_case:Vn \__spintent_spunit_luaaset_canonical_str
644             {
645               { " } { \prime \! \prime }
646               { ' } { \prime
647               { ° } { °
648             }
649           }
650         }
651       }
652     {
653       \tl_if_empty:NF \__spintent_luaaset_part_int_tl
654       {
655         \__spintent_spnum_print_number_core: \,
656       }
657       \__spintent_print_spunit_hierarchy:
658     }
659   }
660 }
661 { \msg_error:nnn { spintent } { spunit-invalid-unit } { #1 } }
662 }
663 }

```

(End of definition for `\__spintent_spunit_safe_exec:n`)

11.13.5 Implementation of the `\spunit` command

`\spunit` The public user command `\spunit` accepts an optional [`<key = val>`] argument in ‘#1’ and a mandatory `{<unit expression>}` in ‘#2’. It first checks if it is executed inside *math mode*, raising an “error” if it is not. If correct, it opens a local group to apply the options and passes the `{<argument>}` to Lua module for parsing via `\__spintent_luafun_clean_split_and_set:n` function.

Finally, it delegates the execution to the function `\__spintent_spunit_safe_exec:n` to perform the safety checks and render the parsed data, before closing the group.

```

664 \NewDocumentCommand \spunit { 0{ } m }
665 {
666   \mode_if_math:TF
667   {
668     \group_begin:
669     \keys_set:nn { spintent/spunit } { #1 }
670     \__spintent_luafun_clean_split_and_set:n { \exp_not:n { #2 } }
671     \__spintent_spunit_safe_exec:n { #2 }
672   \group_end:
673   }
674   { \msg_error:nnn { spintent } { need-math-mode } { \spunit } }
675 }

```

(End of definition for `\spunit`. This function is documented on page 4.)

11.14 The commands `\spnewunit` and `\spunitalias`

The user commands `\spnewunit` and `\spunitalias` allow users to add new custom “units” or “aliases” to the package’s dictionary. Similar to `\spunit`, these commands delegate the processing and validation to the `spintent.lua` via `\__spintent_luafun_define_custom_unit:nn` and `\__spintent_luafun_define_spunit_alias:nn`.

During the definition, the Lua module checks if the new “unit” or “alias” is valid and ensures it does not conflict with existing “units”. It returns the result of this operation to TeX using the `\__spintent_spunit_luaaset_status_str` variable. If a collision is detected or the `{<argument>}` is invalid, the package throws the corresponding “error” message.

11.14.1 Definition of error messages

spnewunit-duplicate
spunitalias-duplicate
spunitalias-invalid-expr

We define three error messages for the creation of “units” and “aliases”. The first two “errors” trigger if the user attempts to define a “new unit” or alias passed in ‘#1’ that already exists in the dictionary. The third “error” is raised by `\spunitalias` if the target expression provided in ‘#2’ is invalid (for example, if it contains unknown “units” or incorrect syntax).

```

676 \msg_new:nnnn { spintent } { spnewunit-duplicate }
677 {
678   The~unit~symbol~or~alias~'#1'~is~already~defined!
679 }
680 {
681   You~cannot~redefine~an~existing~unit~with~\token_to_str:N \spnewunit.
682 }
683 \msg_new:nnnn { spintent } { spunitalias-duplicate }
684 {
685   The~alias~'#1'~is~already~defined~or~reserved!
686 }
687 {
688   You~cannot~redefine~an~existing~unit~or~another~alias.
689 }
690 \msg_new:nnnn { spintent } { spunitalias-invalid-expr }
691 {
692   The~expression~'#2'~for~alias~'#1'~is~invalid.
693 }
694 {
695   It~must~contain~only~defined~units,~exponents~and~one~slash~(/).
696 }

```

(End of definition for `spnewunit-duplicate`, `spunitalias-duplicate`, and `spunitalias-invalid-expr`.)

`\spnewunit`

The public command `\spnewunit` defines a new custom unit. It passes the new unit symbol ‘#1’ and its speech text ‘#2’ to Lua via `\__spintent_luafun_define_custom_unit:nn`. If the module reports that the unit already exists, it throws the corresponding duplicate error.

```

697 \NewDocumentCommand \spnewunit { m m }
698 {
699   \mode_if_math:TF
700   {
701     \msg_error:nnn { spintent } { need-text-mode } { \spnewunit }
702   }
703   {
704     \__spintent_luafun_define_custom_unit:nn { #1 } { #2 }
705     \str_if_eq:VnT \l__spintent_spunit_luaset_status_str { duplicate }
706     {
707       \msg_error:nnn { spintent } { spnewunit-duplicate } { #1 }
708     }
709   }
710 }

```

(End of definition for `\spnewunit`. This function is documented on page 6.)

`\spunitalias`

The public command `\spunitalias` creates a shortcut for a complex unit expression. It passes the new alias name ‘#1’ and the target expression ‘#2’ to Lua via `\__spintent_luafun_define_spunit_alias:nn`. It throws an error if the alias already exists, or if the provided target expression contains invalid syntax or unknown units.

```

711 \NewDocumentCommand \spunitalias { m m }
712 {
713   \mode_if_math:TF
714   {
715     \msg_error:nnn { spintent } { need-text-mode } { \spunitalias }
716   }
717   {
718     \__spintent_luafun_define_spunit_alias:nn { #1 } { #2 }
719     \str_if_eq:VnT \l__spintent_spunit_luaset_status_str { duplicate }
720     {
721       \msg_error:nnn { spintent } { spunitalias-duplicate } { #1 }
722     }
723     \str_if_eq:VnT \l__spintent_spunit_luaset_status_str { invalid-expr }
724     {
725       \msg_error:nnnn { spintent } { spunitalias-invalid-expr } { #1 } { #2 }
726     }
727   }
728 }

```


(End of definition for `\spuntalias`. This function is documented on page 6.)

11.15 The command `\spmoney`

The user command `\spmoney` is the *third* of “the big four” provided by the `spintent` package, designed to format monetary values and currencies. It delegates $\{\langle argument \rangle\}$ parsing to the Lua module `spintent.lua` via `\__spintent_luafun_clean_split_and_set:n`. The module then retrieves the correct “currency symbol”, pluralization rules, and layout structures from the database.

The command accepts *optional arguments* to override the default “currency symbol” or enable “subunit” rendering. Finally, it constructs the `MathML` intent for `MathCAT`.

These string variables store the currency data returned by the Lua module `spintent.lua`. Upon a successful database lookup, the module populates these variables with the “currency symbol”, its layout position (pre or post), the appropriate singular and plural names for both the main unit and subunits, and the execution status flags. $\TeX$ then uses this information to render the visual layout and construct the correct `MathML` speech intents.

```
\__spintent_spmoney_luaset_symbol_str
\__spintent_spmoney_luaset_position_str
\__spintent_spmoney_luaset_sing_str
\__spintent_spmoney_luaset_plur_str
\__spintent_spmoney_luaset_conde_str
\__spintent_spmoney_luaset_subsing_str
\__spintent_spmoney_luaset_subplur_str
\__spintent_spmoney_luaset_print_iso_str
\__spintent_spmoney_luaset_currency_arg_str
\__spintent_spmoney_luaset_status_str
```

```
729 \str_new:N \__spintent_spmoney_luaset_symbol_str
730 \str_new:N \__spintent_spmoney_luaset_position_str
731 \str_new:N \__spintent_spmoney_luaset_sing_str
732 \str_new:N \__spintent_spmoney_luaset_plur_str
733 \str_new:N \__spintent_spmoney_luaset_conde_str
734 \str_new:N \__spintent_spmoney_luaset_subsing_str
735 \str_new:N \__spintent_spmoney_luaset_subplur_str
736 \str_new:N \__spintent_spmoney_luaset_print_iso_str
737 \str_new:N \__spintent_spmoney_luaset_currency_arg_str
738 \str_new:N \__spintent_spmoney_luaset_status_str
```

(End of definition for `\__spintent_spmoney_luaset_symbol_str` and others.)

The boolean variable tracks whether subunit rendering is enabled. The string variables assemble the specific speech text (intents) for screen readers, including the sign, the main currency, and the subunits. Finally, the remaining variables temporarily store the numeric value and currency code extracted from the user’s input.

```
\__spintent_spmoney_sub_units_bool
\__spintent_spmoney_intent_money_str
\__spintent_spmoney_intent_sign_str
\__spintent_spmoney_intent_sub_unit_str
\__spintent_spmoney_res_main_voice_str
\__spintent_spmoney_subnum_int
\__spintent_spmoney_extracted_curr_str
\__spintent_spmoney_extracted_num_tl
```

```
739 \bool_new:N \__spintent_spmoney_sub_units_bool
740 \str_new:N \__spintent_spmoney_intent_money_str
741 \str_new:N \__spintent_spmoney_intent_sign_str
742 \str_new:N \__spintent_spmoney_intent_sub_unit_str
743 \str_new:N \__spintent_spmoney_res_main_voice_str
744 \int_new:N \__spintent_spmoney_subnum_int
745 \str_new:N \__spintent_spmoney_extracted_curr_str
746 \tl_new:N \__spintent_spmoney_extracted_num_tl
```

(End of definition for `\__spintent_spmoney_sub_units_bool` and others.)

11.15.1 Definition of error messages

We define two messages to handle invalid currency configurations. The first “error” warns the user if subunit rendering is requested for a currency that does not support fractional divisions (such as JPY or CLP), safely ignoring the flag. The second “error” is triggered if the provided currency identifier is not registered in the Lua database.

```
currency-no-subunits
invalid-currency
```

```
747 \msg_new:nnnn { spintent } { currency-no-subunits }
748 {
749   The~currency~'#1'~does~not~support~subunits.~
750   The~option~'sub-units=true'~is~ignored.
751 }
752 {
753   Monies~like~the~Japanese~Yen~(JPY)~or~Chilean~Peso~(CLP)~
754   do~not~have~fractional~subunits~registered~in~Lua~module.~
755 }
756 \msg_new:nnnn { spintent } { invalid-currency }
757 {
758   Invalid~currency~identifier~'#1'~in~\token_to_str:N \spmoney .
759 }
760 {
761   The~provided~symbol,~alias,~or~ISO~code~does~not~exist~in~
762   the~Lua~module.
763 }
```

(End of definition for `currency-no-subunits` and `invalid-currency`.)

11.15.2 Auxiliary functions for \spmoney

The function `\__spintent_spmoney_set_currency:n` validates the currency identifier provided in ‘`#1`’ against the database. It delegates the lookup to the helper function `\__spintent_spmoney_normalize_key:n`, which interfaces with the Lua module `spintent.lua`. If the module returns a not found status, an “error” is triggered. Otherwise, it stores the valid currency identifier in `\__spintent_spmoney_luaset_currency_arg_str` for subsequent processing.

```

764 \cs_new_protected:Nn \__spintent_spmoney_set_currency:n
765 {
766   \__spintent_spmoney_normalize_key:n {#1}
767   \str_if_eq:VnTF \__spintent_spmoney_luaset_status_str { notfound }
768   {
769     \msg_error:nnn { spintent } { invalid-currency } {#1}
770   }
771   { \str_set:Nn \__spintent_spmoney_luaset_currency_arg_str {#1} }
772 }
773 \cs_new_protected:Nn \__spintent_spmoney_normalize_key:n
774 {
775   \__spintent_luafun_spmoney_normalize_key:n { #1 }
776 }
777 \cs_generate_variant:Nn \__spintent_spmoney_normalize_key:n { V }

```

(End of definition for `\__spintent_spmoney_set_currency:n` and `\__spintent_spmoney_normalize_key:n`.)

11.15.3 Definition of keys for \spmoney

Now we add the keys `currency`, `sub-units`, `dolar-math` and `cifras-sep`.

```

778 \keys_define:nn { spintent / spmoney }
779 {
780   currency      .code:n      = { \__spintent_spmoney_set_currency:n {#1} },
781   currency      .initial:n    = { pesos },
782   currency      .value_required:n = true,
783   sub-units     .bool_set:N   = \__spintent_spmoney_sub_units_bool,
784   sub-units     .value_required:n = true,
785   dolar-math    .bool_set:N   = \__spintent_spmoney_dolar_math_bool,
786   dolar-math    .initial:n    = { true },
787   dolar-math    .value_required:n = true,
788   cifras-sep    .meta:nn     = { spintent/spnum } { cifras-sep = {#1} },
789   cifras-sep    .value_required:n = true,
790   unknown      .code:n      =
791   {
792     \str_set:Nn \__spintent_command_name_str { spmoney }
793     \__spintent_unknown_keys_cmd:n {#1}
794   },
795 }

```

(End of definition for `currency` and others.)

11.15.4 Internal functions for \spmoney

The function `\__spintent_spmoney_validate_money:` verifies if the parsed input contains an embedded currency identifier by checking `\__spintent_luaset_has_units_str`. If a currency is detected, it extracts and normalizes the key from `\__spintent_luaset_arg_above_tl`. It throws an “error” if the currency is not found in the database. If valid, it updates `\__spintent_spmoney_luaset_currency_arg_str` and retrieves the full currency metadata from the `spintent.lua` via `\__spintent_luafun_spmoney_lookup_data:V`.

```

796 \cs_new_protected:Nn \__spintent_spmoney_validate_money:
797 {
798   \str_if_eq:VnT \__spintent_luaset_has_units_str { true }
799   {
800     \__spintent_spmoney_normalize_key:V \__spintent_luaset_arg_above_tl
801     \str_if_eq:VnTF \__spintent_spmoney_luaset_status_str { notfound }
802     { \msg_error:nne { spintent } { invalid-currency } { \__spintent_luaset_arg_above_tl }
803     {
804       \str_set:NV
805         \__spintent_spmoney_luaset_currency_arg_str
806         \__spintent_luaset_arg_above_tl
807     }
808   }
809   \str_set:Ne \__spintent_spmoney_luaset_status_str { found }
810   \__spintent_luafun_spmoney_lookup_data:V \__spintent_spmoney_luaset_currency_arg_str
811 }

```

(End of definition for `\__spintent_spmoney_validate_money:`.)

`\__spintent_spmoney_print_symbol:` The function `\__spintent_spmoney_print_symbol:` renders the “*currency symbol*”. If `\l__spintent_spmoney_luaset_print_iso_str` is true, it outputs the ISO code using `\mathrm`. Otherwise, it prints the “*standard symbol*”. For dollar signs ‘\$’, it checks `\l__spintent_spmoney_dolar_math_bool` to determine whether to render it directly as a math character ‘\$’ or wrap it inside `\text`.

```

812 \cs_new_protected:Nn \__spintent_spmoney_print_symbol:
813 {
814   \str_if_eq:VnTF \l__spintent_spmoney_luaset_print_iso_str { true }
815   { \mathrm { \l__spintent_spmoney_luaset_currency_arg_str } }
816   {
817     \str_if_eq:VnTF \l__spintent_spmoney_luaset_symbol_str { $ }
818     {
819       \bool_if:NTF \l__spintent_spmoney_dolar_math_bool
820       { \$ } { \text { \$ } }
821     }
822     { \l__spintent_spmoney_luaset_symbol_str }
823   }
824 }

```

(End of definition for `\__spintent_spmoney_print_symbol:`.)

`\__spintent_spmoney_render_layout:` The function `\__spintent_spmoney_render_layout:` determines the layout order of the “*currency symbol*” relative to the numeric value. It evaluates `\l__spintent_spmoney_luaset_position_str`; if set to pre, it renders the “*currency symbol*” *before* the number node. Otherwise, it places the number first, followed by the “*currency symbol*”.

```

825 \cs_new_protected:Nn \__spintent_spmoney_render_layout:
826 {
827   \str_if_eq:VnTF \l__spintent_spmoney_luaset_position_str { pre }
828   {
829     \__spintent_spmoney_print_symbol:
830     \MathMLarg { n } { \__spintent_spmoney_render_number_node: }
831   }
832   {
833     \MathMLarg { n } { \__spintent_spmoney_render_number_node: }
834     \__spintent_spmoney_print_symbol:
835   }
836 }

```

(End of definition for `\__spintent_spmoney_render_layout:`.)

`\__spintent_spmoney_render_number_node:` The function `\__spintent_spmoney_render_number_node:` renders the full numeric value, encompassing both the integer and decimal components. Conversely, the function `\__spintent_spmoney_render_only_integer_node:` outputs exclusively the integer portion, omitting any decimal data.

```

837 \cs_new_protected:Nn \__spintent_spmoney_render_number_node:
838 {
839   \__spintent_spmoney_render_int:
840   \__spintent_spmoney_print_part_dec:
841 }
842 \cs_new_protected:Nn \__spintent_spmoney_render_only_integer_node:
843 {
844   \__spintent_spmoney_render_int:
845 }

```

(End of definition for `\__spintent_spmoney_render_number_node:` and `\__spintent_spmoney_render_only_integer_node:`.)

`\__spintent_spmoney_print_int:` The function `\__spintent_spmoney_print_int:` evaluates the presence of an explicit sign in `\l__spintent_luaset_sign_tl`. If a sign is detected, it assigns a prefix speech intent ‘más’ or ‘menos’ and embeds the sign and subsequent layout inside a `\MathMLintent`. If no sign is present, it routes execution based on whether subunits are enabled.

The function `\__spintent_spmoney_intent_int:` determines the correct grammatical inflection for the currency string (singular, plural, or the prepositional ‘de’ variant for exact millions). It evaluates the integer component, decimal boundaries, and zero-fills to select the appropriate string register, applying it as a postfix `MathML` intent over the layout rendered by `\__spintent_spmoney_render_layout:`.

```

846 \cs_new_protected:Nn \__spintent_spmoney_print_int:
847 {
848   \tl_if_empty:NTF \l__spintent_luaset_sign_tl
849   {
850     \bool_if:NTF \l__spintent_spmoney_sub_units_bool
851     { \__spintent_spmoney_set_and_print_sub_units: }
852     { \__spintent_spmoney_intent_int: }
853   }

```

```

854     {
855         \str_if_eq:VnTF \l__spintent_luaset_sign_tl { + }
856         {
857             \str_set:Nn \l__spintent_spmoney_intent_sign_str { _más:prefix ($n) }
858         }
859         {
860             \str_set:Nn \l__spintent_spmoney_intent_sign_str { _menos:prefix ($n) }
861         }
862         \__spintent_invisible_sep:
863         \MathMLintent { \l__spintent_spmoney_intent_sign_str }
864         {
865             \l__spintent_luaset_sign_tl
866             \MathMLarg { n }
867             {
868                 \bool_if:NTF \l__spintent_spmoney_sub_units_bool
869                 { \__spintent_spmoney_set_and_print_sub_units: }
870                 { \__spintent_spmoney_intent_int: }
871             }
872         }
873     }
874 }
875 \cs_new_protected:Nn \__spintent_spmoney_intent_int:
876 {
877     \tl_if_empty:NTF \l__spintent_luaset_dec_sep_tl
878     {
879         \str_if_eq:VnTF \l__spintent_luaset_only_part_int_str { 1 }
880         {
881             \str_set:NV \l__spintent_spmoney_intent_money_str \l__spintent_spmoney_luaset_sing_str
882         }
883         {
884             \str_set:NV \l__spintent_spmoney_intent_money_str \l__spintent_spmoney_luaset_plur_str
885         }
886         \str_if_eq:VnT \l__spintent_luaset_millions_str { true }
887         {
888             \str_set:NV \l__spintent_spmoney_intent_money_str \l__spintent_spmoney_luaset_conde_s
889         }
890     }
891     {
892         \str_set:NV \l__spintent_spmoney_intent_money_str \l__spintent_spmoney_luaset_plur_str
893         \str_if_eq:VnT \l__spintent_luaset_millions_str { true }
894         {
895             \str_if_eq:VnTF \l__spintent_luaset_dec_is_all_zeros_str { true }
896             {
897                 \str_set:NV \l__spintent_spmoney_intent_money_str \l__spintent_spmoney_luaset_conde
898             }
899             {
900                 \str_set:NV \l__spintent_spmoney_intent_money_str \l__spintent_spmoney_luaset_plur
901             }
902         }
903     }
904     \__spintent_invisible_sep: % need for MathCAT
905     \MathMLintent { \l__spintent_spmoney_intent_money_str :postfix($n) }
906     {
907         \__spintent_spmoney_render_layout:
908     }
909 }

```

(End of definition for `\__spintent_spmoney_print_int:` and `\__spintent_spmoney_intent_int:`)

`\__spintent_spmoney_check_sub_units:`
`\__spintent_spmoney_set_and_print_sub_units:`

The function `\__spintent_spmoney_check_sub_units:` verifies whether the selected currency supports “subunits”. If the database returns empty subunit data, it triggers a warning and automatically disables the “subunit” rendering flag.

The function `\__spintent_spmoney_set_and_print_sub_units:` processes and formats the speech intents when “subunits” are active. It normalizes the decimal component (for instance, padding a single-digit decimal like ‘5’ to ‘50’), evaluates the correct singular or plural inflections for both the main “currency” and the “subunits”, and delegates the final `MathML` tree assembly to `\__spintent_spmoney_build_tree:`.

```

910 \cs_new_protected:Nn \__spintent_spmoney_check_sub_units:
911 {
912     \str_if_empty:NT \l__spintent_spmoney_luaset_subsing_str
913     {
914         \msg_warning:nne { spintent } { currency-no-subunits }

```

```

915         { \l__spintent_spmoney_luaset_currency_arg_str }
916         \bool_set_false:N \l__spintent_spmoney_sub_units_bool
917     }
918 }
919 \cs_new_protected:Nn \l__spintent_spmoney_set_and_print_sub_units:
920 {
921     \int_compare:nTF { \str_count:N \l__spintent_luaset_only_part_dec_str == 1 }
922     {
923         \int_set:Nn \l__spintent_spmoney_subnum_int
924         { \l__spintent_luaset_only_part_dec_str * 10 }
925         \str_set:Ne \l__spintent_luaset_only_part_dec_str
926         { \l__spintent_luaset_only_part_dec_str 0 }
927     }
928     {
929         \int_set:Nn \l__spintent_spmoney_subnum_int
930         { \l__spintent_luaset_only_part_dec_str }
931     }
932     \int_compare:nTF { \l__spintent_spmoney_subnum_int == 1 }
933     {
934         \str_set:Ne \l__spintent_spmoney_intent_sub_unit_str
935         { \l__spintent_spmoney_luaset_subsing_str :postfix($sub) }
936     }
937     {
938         \str_set:Ne \l__spintent_spmoney_intent_sub_unit_str
939         { \l__spintent_spmoney_luaset_subplur_str :postfix($sub) }
940     }
941     \str_if_eq:VnTF \l__spintent_luaset_only_part_int_str { 1 }
942     {
943         \str_set:NV \l__spintent_spmoney_res_main_voice_str
944         \l__spintent_spmoney_luaset_sing_str
945     }
946     {
947         \str_set:NV \l__spintent_spmoney_res_main_voice_str
948         \l__spintent_spmoney_luaset_plur_str
949     }
950     \str_if_eq:VnT \l__spintent_luaset_millions_str { true }
951     {
952         \str_set:NV \l__spintent_spmoney_res_main_voice_str
953         \l__spintent_spmoney_luaset_conde_str
954     }
955     \l__spintent_spmoney_build_tree:
956 }

```

(End of definition for `\l__spintent_spmoney_check_sub_units:` and `\l__spintent_spmoney_set_and_print_sub_units:`)

```

\l__spintent_spmoney_build_tree:
\l__spintent_spmoney_print_integer_continue:
\l__spintent_spmoney_print_sep_con:
\l__spintent_spmoney_print_dec_sub_unit:

```

The function `\l__spintent_spmoney_build_tree:` structures the final MathML tree assembly for fractional “*currencies*”. It arranges the integer component, the decimal separator, and the “*subunit*” component based on the layout position (pre or post) of the “*currency symbol*”.

The function `\l__spintent_spmoney_print_integer_continue:` isolates the rendering of the integer node with its associated main “*currency*” intent. Furthermore, `\l__spintent_spmoney_print_sep_con:` applies the specific speech intent ‘con’ to the decimal separator, while `\l__spintent_spmoney_print_dec_sub_unit:` binds the computed “*subunit*” intent to the decimal digits, ensuring accurate accessibility output.

```

957 \cs_new_protected:Nn \l__spintent_spmoney_build_tree:
958 {
959     \str_if_eq:VnTF \l__spintent_spmoney_luaset_position_str { pre }
960     {
961         \MathMLintent { \l__spintent_spmoney_res_main_voice_str :postfix($n) }
962         {
963             \l__spintent_spmoney_print_symbol:
964             \MathMLarg { n } { \l__spintent_spmoney_render_only_integer_node: }
965         }
966         \l__spintent_spmoney_print_sep_con:
967         \l__spintent_spmoney_print_dec_sub_unit:
968     }
969     {
970         \MathMLintent { \l__spintent_spmoney_res_main_voice_str :postfix($n) }
971         {
972             \l__spintent_invisible_sep:
973             \MathMLarg { n } { \l__spintent_spmoney_render_only_integer_node: }
974         }
975     }
976 }

```

```

975     \__spintent_spmoney_print_sep_con:
976     \__spintent_spmoney_print_dec_sub_unit:
977     \MathMLintent { :silent } { \__spintent_spmoney_print_symbol: }
978   }
979 }
980 \cs_new_protected:Nn \__spintent_spmoney_print_integer_continue:
981 {
982   \MathMLintent { \l__spintent_spmoney_res_main_voice_str :postfix($n) }
983   {
984     \MathMLarg { n } { \__spintent_spmoney_render_only_integer_node: }
985   }
986 }
987 \cs_new_protected:Nn \__spintent_spmoney_print_sep_con:
988 {
989   \MathMLintent { con }
990   {
991     \mathord { \l__spintent_luaset_dec_sep_tl }
992   }
993 }
994 \cs_new_protected:Nn \__spintent_spmoney_print_dec_sub_unit:
995 {
996   \MathMLintent { \l__spintent_spmoney_intent_sub_unit_str }
997   {
998     \__spintent_invisible_sep:
999     \MathMLarg { sub }
1000     {
1001       \__spintent_spmoney_render_dec:
1002     }
1003   }
1004 }

```

(End of definition for `\__spintent_spmoney_build_tree:` and others.)

`\__spintent_spmoney_parse_and_route:n`

The internal function `\__spintent_spmoney_parse_and_route:n` processes the $\langle argument \rangle$ passed in ‘#1’ using Lua module to extract the numeric value and any “currency” identifier. If a “currency” is found (`\l__spintent_spmoney_extracted_curr_str`), it sets the corresponding variables. It then calls `\__spintent_spmoney_validate_money:` to verify the “currency”. If “subunit” rendering is enabled (`\l__spintent_spmoney_sub_units_bool`), it calls `\__spintent_spmoney_check_sub_units:`. Finally, it calls `\__spintent_spmoney_print_int:` to format the output.

```

1005 \cs_new_protected:Nn \__spintent_spmoney_parse_and_route:n
1006 {
1007   \__spintent_luafun_spmoney_prepare_input:n { #1 }
1008   \__spintent_luafun_clean_split_and_set:V \l__spintent_spmoney_extracted_num_tl
1009   \str_if_empty:NF \l__spintent_spmoney_extracted_curr_str
1010   {
1011     \str_set:Ne \l__spintent_luaset_has_units_str { true }
1012     \str_set:NV \l__spintent_luaset_arg_above_tl \l__spintent_spmoney_extracted_curr_str
1013   }
1014   \__spintent_spmoney_validate_money:
1015   \bool_if:NT \l__spintent_spmoney_sub_units_bool
1016   { \__spintent_spmoney_check_sub_units: }
1017   \__spintent_spmoney_print_int:
1018 }

```

(End of definition for `\__spintent_spmoney_parse_and_route:n`.)

`\spmoney`

The public document command `\spmoney` formats monetary values with accessible speech intents. It accepts an optional argument ‘#1’ for local key-value configurations and a mandatory argument ‘#2’ containing the numeric value and optional embedded currency identifier. It first evaluates the typesetting context, triggering a fatal error if executed outside of math mode. Upon validation, it establishes a local TeX group to safely isolate the key assignments, delegates the raw input string to `\__spintent_spmoney_parse_and_route:n` for parsing and rendering, and subsequently closes the group to prevent scope leakage.

```

1019 \NewDocumentCommand \spmoney { 0{} m }
1020 {
1021   \mode_if_math:TF
1022   {
1023     \group_begin:
1024     \keys_set:nn { spintent / spmoney } { #1 }
1025     \__spintent_spmoney_parse_and_route:n { #2 }
1026     \group_end:
1027   }

```



```

1028     { \msg_error:nnn { spintent } { need-math-mode } { \spmoney } }
1029 }

```

(End of definition for `\spmoney`. This function is documented on page 6.)

11.16 The command `\spshort`

The user command `\spshort` is the *fourth* and *final* of “the big four”. It manages the semantic injection and typographical formatting of *text mode* abbreviations, ordinals, and acronyms. Unlike *math mode* commands, which rely on `MathML` attributes, this command uses the PDF *tagging tools* provided by the `tagpdf`[29] package. Specifically, it uses the `E` (expanded) key to ensure that `NVDA` interpret the full form of the abbreviation, ordinal, or acronym in *text mode*.

These internal string variables serve as the primary data interface with the Lua module `spintent.lua`. During processing, the Lua module “stores” within these registers the execution status, the targeted typographical layout parameters, the phonetic replacement text for the `E` key intended for `NVDA`, the final sanitized output string, and the isolated structural components for ordinals (base and suffix).

```

1030 \str_new:N \__spintent_spshort_luaset_status_str
1031 \str_new:N \__spintent_spshort_luaset_layout_str
1032 \str_new:N \__spintent_spshort_luaset_expanded_str
1033 \str_new:N \__spintent_spshort_luaset_output_str
1034 \str_new:N \__spintent_spshort_luaset_base_str
1035 \str_new:N \__spintent_spshort_luaset_suffix_str

```

(End of definition for `\__spintent_spshort_luaset_status_str` and others.)

11.16.1 Definition of error messages for `\spshort`

unknown-abbreviation

The internal “error” message `unknown-abbreviation` is triggered when the command `\spshort` encounters an invalid lookup state from the Lua module. It acts as a strict fallback exception when the provided `{⟨argument⟩}` cannot be mapped to any registered database entry in `spintent.lua` nor successfully resolved through the grammatical ordinal expansion rules.

```

1036 \msg_new:nnnn { spintent } { unknown-abbreviation }
1037 { The~abbreviation~or~ordinal~'#1'~is~invalid~or~not~recognized. }
1038 { Verify~the~spelling~or~check~the~grammar~rules~for~ordinals. }

```

(End of definition for `unknown-abbreviation`.)

11.16.2 Accessibility interface

`\__spintent_spshort_tag_span:nn`
`\__spintent_spshort_tag_span:en`

The internal function `\__spintent_spshort_tag_span:nn` processes the `{⟨argument⟩}` passed in ‘#2’ using the tools provided by `tagpdf`. By utilizing the `tag` and `E` keys, it creates a native PDF `/E` (`#1`) structure and expands the `{⟨argument⟩}` passed in ‘#1’ for proper reading with `NVDA`.

```

1039 \cs_new_protected:Nn \__spintent_spshort_tag_span:nn
1040 {
1041   \tag_mc_end_push:
1042   \tag_struct_begin:n { tag=Span, E={ #1 } }
1043   \tag_mc_begin:n { tag=Span }
1044   \tag_suspend:n { \spshort }
1045   #2
1046   \tag_resume:n { \spshort }
1047   \tag_mc_end:
1048   \tag_struct_end:
1049   \tag_mc_begin_pop:n {}
1050 }
1051 \cs_generate_variant:Nn \__spintent_spshort_tag_span:nn { en }

```

(End of definition for `\__spintent_spshort_tag_span:nn`.)

11.16.3 Definition of keys for `\spshort`

small-caps
 read-arg

Now we add the keys `small-caps` and `read-arg`.

```

1052 \keys_define:nn { spintent / spshort }
1053 {
1054   small-caps .bool_set:N = \__spintent_spshort_smallcaps_bool,
1055   small-caps .initial:n = false,
1056   small-caps .value_required:n = true,
1057   read-arg .str_set:N = \__spintent_spshort_read_arg_str,
1058   read-arg .initial:n = {},
1059   read-arg .value_required:n = true,
1060   unknown .code:n =
1061   {
1062     \str_set:Nn \__spintent_command_name_str { spshort }
1063     \__spintent_unknown_keys_cmd:n {#1}
1064   },
1065 }

```

(End of definition for `small-caps` and `read-arg`.)

11.16.4 Internal functions for `\spshort`

`\__spintent_spshort_print:nn`

The internal function `\__spintent_spshort_print:nn` acts as the primary formatter for text abbreviations. It clears the string variable `\__spintent_spshort_read_arg_str` and evaluates the optional $\langle keys \rangle$ passed in `#1`. It then checks if a manual reading override was provided via those keys. If that string variable remains empty, it calls `\__spintent_spshort_process_and_render:n` to process the $\langle argument \rangle$ passed in `#2`. Otherwise, if an override is detected, it directly delegates the execution to `\__spintent_spshort_render_bypass:n`.

```

1066 \cs_new_protected:Nn \__spintent_spshort_print:nn
1067 {
1068   \str_clear:N \__spintent_spshort_read_arg_str
1069   \keys_set:nn { spintent / spshort } {#1}
1070   \str_if_empty:NTF \__spintent_spshort_read_arg_str
1071   {
1072     \__spintent_spshort_process_and_render:n {#2}
1073   }
1074   {
1075     \__spintent_spshort_render_bypass:n {#2}
1076   }
1077 }

```

(End of definition for `\__spintent_spshort_print:nn`.)

`\__spintent_spshort_process_and_render:n`

The internal function `\__spintent_spshort_process_and_render:n` interfaces with the Lua module `spintent.lua` function `\__spintent_luafun_spshort_process:n` to resolve the abbreviation or ordinal $\langle argument \rangle$ passed in `#1`.

It subsequently evaluates the execution state stored within the string variable `\__spintent_spshort_luaset_status_str` and dynamically routes execution to the corresponding rendering subroutine success, fallback or error. Any unrecognized status automatically defaults to the “error” handler.

```

1078 \cs_new_protected:Nn \__spintent_spshort_process_and_render:n
1079 {
1080   \__spintent_luafun_spshort_process:n {#1}
1081   \str_case:VnF \__spintent_spshort_luaset_status_str
1082   {
1083     { success } { \__spintent_spshort_render_success: }
1084     { fallback } { \__spintent_spshort_render_fallback: }
1085     { error } { \__spintent_spshort_render_error:n {#1} }
1086   }
1087   { \__spintent_spshort_render_error:n {#1} }
1088 }

```

(End of definition for `\__spintent_spshort_process_and_render:n`.)

`\__spintent_spshort_render_success:`

The function `\__spintent_spshort_render_success:` injects the key E and executes the targeted typographical layout. The `\__spintent_spshort_render_fallback:` function handles default cases, applying optional formatting such as SMALL CAPS to unrecognized or partially resolved terms.

`\__spintent_spshort_render_fallback:`

`\__spintent_spshort_render_bypass:n`

`\__spintent_spshort_render_error:n`

Conversely, the function `\__spintent_spshort_render_bypass:n` directly maps the user-defined “speech” to the $\langle argument \rangle$, bypassing the automated lookup Lua module. Finally, the function `\__spintent_spshort_render_error:n` triggers a structural exception while yielding the unformatted $\langle argument \rangle$ to prevent fatal compilation.

```

1089 \cs_new_protected:Nn \__spintent_spshort_render_success:
1090 {
1091   \__spintent_spshort_tag_span:en
1092   { \__spintent_spshort_luaset_expanded_str }
1093   { \__spintent_spshort_execute_layout: }
1094 }
1095 \cs_new_protected:Nn \__spintent_spshort_render_fallback:
1096 {
1097   \__spintent_spshort_tag_span:en
1098   { \__spintent_spshort_luaset_expanded_str }
1099   {
1100     \bool_if:NTF \__spintent_spshort_smallcaps_bool
1101     { \textsc{ \__spintent_spshort_luaset_output_str } }
1102     { \__spintent_spshort_luaset_output_str }
1103   }
1104 }
1105 \cs_new_protected:Nn \__spintent_spshort_render_bypass:n
1106 {
1107   \__spintent_spshort_tag_span:en { \__spintent_spshort_read_arg_str } { #1 }

```

```

1108 }
1109 \cs_new_protected:Nn \__spintent_spshort_render_error:n
1110 {
1111   \msg_error:nnn { spintent } { unknown-abbreviation } {#1}
1112 }

```

(End of definition for `\__spintent_spshort_render_success:` and others.)

`\__spintent_spshort_execute_layout:`

The internal function `\__spintent_spshort_execute_layout:` executes the typographical layout. It evaluates the descriptor “*stored*” within the string variable `\__spintent_spshort_luaset_layout_str` (such as `linear_regular`, `superscript`, or `small_caps_pure`) and maps it to the corresponding $\text{\LaTeX}$ formatting. Furthermore, it manages complex nested configurations, dynamically applying combinations such as superscripts integrated with conditional SMALL CAPS based on the active boolean variables.

```

1113 \cs_new_protected:Nn \__spintent_spshort_execute_layout:
1114 {
1115   \str_case:NnF \__spintent_spshort_luaset_layout_str
1116   {
1117     { linear_regular }
1118     {
1119       \bool_if:NTF \__spintent_spshort_smallcaps_bool
1120       { \textsc{ \__spintent_spshort_luaset_output_str } }
1121       { \__spintent_spshort_luaset_output_str }
1122     }
1123     { linear_caps }
1124     { \__spintent_spshort_luaset_output_str }
1125     { linear_mixed }
1126     { \__spintent_spshort_luaset_output_str }
1127     { superscript }
1128     {
1129       \__spintent_spshort_luaset_base_str
1130       \bool_if:NTF \__spintent_spshort_smallcaps_bool
1131       {
1132         \textsuperscript{
1133           \textsc{
1134             \str_lowercase:V \__spintent_spshort_luaset_suffix_str
1135           }
1136         }
1137       }
1138       {
1139         \textsuperscript{ \__spintent_spshort_luaset_suffix_str }
1140       }
1141     }
1142     { small_caps_pure }
1143     {
1144       \bool_if:NTF \__spintent_spshort_smallcaps_bool
1145       {
1146         \textsc{
1147           \str_lowercase:V \__spintent_spshort_luaset_output_str
1148         }
1149       }
1150       { \__spintent_spshort_luaset_output_str }
1151     }
1152     { acronym_long }
1153     { \__spintent_spshort_luaset_output_str }
1154   }
1155   { \__spintent_spshort_luaset_output_str }
1156 }

```

(End of definition for `\__spintent_spshort_execute_layout:`.)

`\spshort`

The public document command `\spshort` manages the accessible formatting of *text mode* abbreviations, acronyms, and ordinals. It first triggers an error if invoked within a *math mode*, upon validation, puts `\mode_leave_vertical:` and establishes a local group to call the function `\__spintent_spshort_print:nn` for processing and printing.

```

1157 \NewDocumentCommand \spshort { 0{} m }
1158 {
1159   \mode_if_math:TF
1160   {
1161     \msg_error:nnn { spintent } { need-text-mode } { \spshort }
1162   }
1163   {

```

```

1164     \mode_leave_vertical:
1165     \group_begin:
1166         \__spintent_spsshort_print:nn {#1} {#2}
1167     \group_end:
1168 }
1169 }

```

(End of definition for `\spsshort`. This function is documented on page 7.)

11.17 The command `\spsiglo`

The public document command `\spsiglo` provides a semantic interface for typesetting century designations (e.g., siglo XXI). It accepts both Arabic and Roman numerals as $\langle \textit{argument} \rangle$, interfacing with the Lua module `spintent.lua` to validate, parse, and normalize chronological values.

This command work in dual-mode context awareness, operating uniformly across both *text mode* and *math mode*. When executed within *math mode*, it constructs an accessible **MathML** intent expression tree. Conversely, when invoked in standard *text mode*, it encapsulates the rendered output within a native PDF tagging structure (such as `/Alt (#1)`) containing an `alt` key.

```

\l__spintent_spsiglo_luaset_error_str
\l__spintent_spsiglo_luaset_arabic_str
\l__spintent_spsiglo_luaset_roman_min_str
\l__spintent_spsiglo_print_era_str

```

These internal string variables handle data transfer and synchronization with the Lua module `spintent.lua`. They isolate the exception evaluation states, the Arabic numeral required for speech synthesis, the lowercase Roman numeral designated for small caps typesetting via `\textsc`, and the localized chronological era modifier.

```

1170 \str_new:N \l__spintent_spsiglo_luaset_error_str
1171 \str_new:N \l__spintent_spsiglo_luaset_arabic_str
1172 \str_new:N \l__spintent_spsiglo_luaset_roman_min_str
1173 \str_new:N \l__spintent_spsiglo_print_era_str

```

(End of definition for `\l__spintent_spsiglo_luaset_error_str` and others.)

11.17.1 Definition of error messages for `\spsiglo`

invalid-century-format

This internal error message is issued when the provided century identifier fails both Arabic and Roman numeral validation. It is triggered when the Lua module `spintent.lua` cannot map the $\langle \textit{argument} \rangle$ to an integer within the valid range of 1 to 4000, preventing incorrect speech synthesis.

```

1174 \msg_new:nnnn { spintent } { invalid-century-format }
1175 {
1176     Invalid~century~format~in~'#1'.
1177 }
1178 {
1179     The~argument~'#2'~is~not~a~valid~Arabic~or~Roman~numeral.~
1180     Ensure~the~value~is~between~1~and~4000.
1181 }

```

(End of definition for `invalid-century-format`.)

11.17.2 Definition of keys for `\spsiglo`

print-era

Now we add the key `print-era`. User configuration determining if an era designation is appended, available values are `none`, `ac` (antes de cristo) and `dc` (después de cristo).

```

1182 \keys_define:nn { spintent / spsiglo }
1183 {
1184     print-era .choices:nn =
1185     { none , ac , dc }
1186     {
1187         \int_case:nn { \l_keys_choice_int }
1188         {
1189             {1} { \str_set:Nn \l__spintent_spsiglo_print_era_str { none } }
1190             {2} { \str_set:Nn \l__spintent_spsiglo_print_era_str { ac } }
1191             {3} { \str_set:Nn \l__spintent_spsiglo_print_era_str { dc } }
1192         }
1193     },
1194     print-era .initial:n = none,
1195     print-era .value_required:n = true,
1196     print-era / unknown .code:n =
1197     \msg_error:nneee { spintent } { unknown-choice }
1198     { print-era } { none, ~ac, ~dc } { \exp_not:n {#1} },
1199     unknown .code:n =
1200     {
1201         \str_set:Nn \l__spintent_command_name_str { spsiglo }
1202         \__spintent_unknown_keys_cmd:n {#1}
1203     },
1204 }

```

(End of definition for `print-era`.)

11.17.3 Internal functions for \spsiglo

The functions `\__spintent_spsiglo_render_simple:nn` and `\__spintent_spsiglo_render_with_era:nnn` generate `MathML` structures when century designations are evaluated within *math mode*. They construct `MathML` intent structures, mapping the Arabic numerals to their phonetic equivalents. Furthermore, `\__spintent_spsiglo_render_with_era:nnn` integrates the era modifiers `ac` or `dc`. It adds invisible spacing and postfix intents to ensure correct screen reader pronunciation.

```

1205 \cs_new_protected:Nn \__spintent_spsiglo_render_simple:nn
1206 {
1207   \MathMLintent { #1:number } { \text { \textsc {#2} } }
1208 }
1209 \cs_generate_variant:Nn \__spintent_spsiglo_render_simple:nn { VV }
1210 \cs_new_protected:Nn \__spintent_spsiglo_render_with_era:nnn
1211 {
1212   \str_if_eq:nnTF { #1 } { ac }
1213   {
1214     \MathMLintent { antes-de-cristo:postfix($z) }
1215     {
1216       \__spintent_invisible_sep:
1217       \MathMLarg { z }
1218       {
1219         \MathMLintent { #2:number }
1220         {
1221           \text { \textsc {#3}~a.~C. }
1222         }
1223       }
1224     }
1225   }
1226   {
1227     \MathMLintent { después-de-cristo:postfix($z) }
1228     {
1229       \__spintent_invisible_sep:
1230       \MathMLarg { z }
1231       {
1232         \MathMLintent { #2:number }
1233         {
1234           \text { \textsc {#3}~d.~C. }
1235         }
1236       }
1237     }
1238   }
1239 }
1240 \cs_generate_variant:Nn \__spintent_spsiglo_render_with_era:nnn { VVV }

```

(End of definition for `\__spintent_spsiglo_render_simple:nn` and `\__spintent_spsiglo_render_with_era:nnn`.)

The function `\__spintent_spsiglo_tag_span:nn` handles century formatting within *text mode*. It utilizes the `tagpdf` package to enclose the content inside a PDF /Alt (#1) structure. By passing the `{⟨argument⟩}` to the `alt` key, it embeds the chronological value directly into the PDF, ensuring correct screen reader pronunciation without relying on `MathML` intents.

```

1241 \cs_new_protected:Nn \__spintent_spsiglo_tag_span:nn
1242 {
1243   \tag_mc_end_push:
1244   \tag_struct_begin:n { tag=Span, alt={ #1 } }
1245   \tag_mc_begin:n { tag=Span }
1246   \tag_suspend:n { \spsiglo }
1247   #2
1248   \tag_resume:n { \spsiglo }
1249   \tag_mc_end:
1250   \tag_struct_end:
1251   \tag_mc_begin_pop:n {}
1252 }
1253 \cs_generate_variant:Nn \__spintent_spsiglo_tag_span:nn { en }

```

(End of definition for `\__spintent_spsiglo_tag_span:nn`.)

The functions `\__spintent_spsiglo_print_math:n` and `\__spintent_spsiglo_print_text:n` format century designations based on the active mode. Both first check the “error” status in the variable `\__spintent_spsiglo_luaset_error_str` returned by the Lua module `spintent.lua` and issue an “error” if validation fails.

If the $\langle argument \rangle$ is valid, the function `\__spintent_spsiglo_print_math:n` operates in *math mode*, applying `MathML` structures depending on whether an era modifier is present. Conversely, the function `\__spintent_spsiglo_print_text:n` operates in *text mode*, utilizing the `tag` and `alt` keys to expand the value of the `print-era` option into full text (e.g., “antes de cristo”) for screen readers.

```

1254 \cs_new_protected:Nn \__spintent_spsiglo_print_math:n
1255 {
1256   \str_if_eq:VnTF \__spintent_spsiglo_luaset_error_str { true }
1257   {
1258     \msg_error:nnnn { spintent } { invalid-century-format }
1259     { \spsiglo } { #1 }
1260   }
1261   {
1262     \str_if_eq:VnTF \__spintent_spsiglo_print_era_str { none }
1263     {
1264       \__spintent_spsiglo_render_simple:VV
1265       \l__spintent_spsiglo_luaset_arabic_str
1266       \l__spintent_spsiglo_luaset_roman_min_str
1267     }
1268     {
1269       \__spintent_spsiglo_render_with_era:VVV
1270       \l__spintent_spsiglo_print_era_str
1271       \l__spintent_spsiglo_luaset_arabic_str
1272       \l__spintent_spsiglo_luaset_roman_min_str
1273     }
1274   }
1275 }
1276 \cs_new_protected:Nn \__spintent_spsiglo_print_text:n
1277 {
1278   \str_if_eq:VnTF \__spintent_spsiglo_luaset_error_str { true }
1279   {
1280     \msg_error:nnnn { spintent } { invalid-century-format }
1281     { \spsiglo } { #1 }
1282   }
1283   {
1284     \str_case:Vn \l__spintent_spsiglo_print_era_str
1285     {
1286       { none }
1287       {
1288         \__spintent_spsiglo_tag_span:en
1289         { \l__spintent_spsiglo_luaset_arabic_str }
1290         { \textsc { \l__spintent_spsiglo_luaset_roman_min_str } }
1291       }
1292       { ac }
1293       {
1294         \__spintent_spsiglo_tag_span:en
1295         { \l__spintent_spsiglo_luaset_arabic_str ~ antes ~ de ~ cristo }
1296         { \textsc { \l__spintent_spsiglo_luaset_roman_min_str } ~ a.~C. }
1297       }
1298       { dc }
1299       {
1300         \__spintent_spsiglo_tag_span:en
1301         { \l__spintent_spsiglo_luaset_arabic_str ~ después ~ de ~ cristo }
1302         { \textsc { \l__spintent_spsiglo_luaset_roman_min_str } ~ d.~C. }
1303       }
1304     }
1305   }
1306 }

```

(End of definition for `\__spintent_spsiglo_print_math:n` and `\__spintent_spsiglo_print_text:n`)

`\spsiglo` The public command `\spsiglo` is the main interface for century typesetting. It opens a local group and evaluates any optional $\langle keys \rangle$ passed in ‘#1’, then executes the function `\__spintent_luafun_spsiglo_parse:n` defined in the Lua module `spintent.lua` passed in ‘#2’ and formats the output for *math mode* or *text mode*.

```

1307 \NewDocumentCommand \spsiglo { 0{} m }
1308 {
1309   \group_begin:
1310   \keys_set:nn { spintent / spsiglo } { #1 }
1311   \__spintent_luafun_spsiglo_parse:n { #2 }
1312   \mode_if_math:TF
1313   { \__spintent_spsiglo_print_math:n { #2 } }

```

```

1314     {
1315         \mode_leave_vertical:
1316         \__spintent_spsiglo_print_text:n { #2 }
1317     }
1318     \group_end:
1319 }

```

(End of definition for \spsiglo. This function is documented on page 11.)

11.18 The command \sptime

The command \sptime is the main public interface for typesetting and tagging time expressions. It parses time string formats (such as HH:MM or HH:MM:SS) using the Lua function __spintent_luafun_sptime_parse:n. This separates the hours, minutes, seconds, and fractions to construct a MathML intent tree for screen readers.

```

\__spintent_sptime_luaset_seconds_str
\__spintent_sptime_luaset_has_seconds_str
\__spintent_sptime_luaset_error_str
\__spintent_sptime_luaset_is_fraction_str
\__spintent_sptime_luaset_base_str

```

These internal string variables store the values returned from the Lua module spintent.lua. They hold the extracted time units, error flags, and boolean indicators (such as the presence of seconds or decimal fractions) used during formatting.

```

1320 \str_new:N \__spintent_sptime_luaset_seconds_str
1321 \str_new:N \__spintent_sptime_luaset_has_seconds_str
1322 \str_new:N \__spintent_sptime_luaset_error_str
1323 \str_new:N \__spintent_sptime_luaset_is_fraction_str
1324 \str_new:N \__spintent_sptime_luaset_base_str

```

(End of definition for __spintent_sptime_luaset_seconds_str and others.)

11.18.1 Definition of error messages

```
invalid-time-format
```

The error message invalid-time-format defines the text displayed when an invalid time input is detected. It is triggered when an argument fails the format checks or contains values outside normal ranges, such as hours exceeding 23 or minutes and seconds exceeding 59.

```

1325 \msg_new:nnnn { spintent } { invalid-time-format }
1326 {
1327     Invalid~time~format~in~#1. \\\
1328     The~argument~'~#2'~does~not~match~any~supported~time~format.
1329 }
1330 {
1331     Allowed~formats:~HH:MM,~HH:MM:SS,~or~HH:MM:SS.ss. \\\
1332     Please~verify~that~hours~are~0-23~and~minutes/seconds~are~0-59.
1333 }

```

(End of definition for invalid-time-format.)

11.18.2 Internal functions for \sptime

```
\__spintent_sptime_build_seconds:nn
```

The internal function __spintent_sptime_build_seconds:nn builds the MathML structure for the seconds component of a time expression. It wraps the numeric value '#2' in speech intent tags, applying a connector prefix (passed in '#1') and the "segundos" postfix so it is read correctly by screen readers.

```

1334 \cs_new_protected:Nn \__spintent_sptime_build_seconds:nn
1335 {
1336     \MathMLintent { #1 : prefix($z) }
1337     {
1338         \text { : }
1339         \MathMLarg { z }
1340         {
1341             \MathMLintent { segundos : postfix($m) }
1342             {
1343                 \__spintent_invisible_sep:
1344                 \MathMLarg { m }
1345                 {
1346                     \text { #2 }
1347                 }
1348             }
1349         }
1350     }
1351 }

```

(End of definition for __spintent_sptime_build_seconds:nn.)

```
\__spintent_sptime_build_tree:
```

The internal function __spintent_sptime_build_tree: builds the complete MathML structure for the time expression. It checks the string variables __spintent_sptime_luaset_has_seconds_str and __spintent_sptime_luaset_is_fraction_str to determine if the time includes seconds or decimal

fractions. Based on these values, it inserts the base time string and either calls `\__spintent_sptime_build_seconds:nn` with the appropriate prefix ('con' or 'minutos-con'), or simply appends the 'minutos' intent tag if no seconds are present.

```

1352 \cs_new_protected:Nn \__spintent_sptime_build_tree:
1353 {
1354   \str_if_eq:VnTF \l__spintent_sptime_luaaset_has_seconds_str { true }
1355   {
1356     \MathMLintent { :time }
1357     {
1358       \text { \l__spintent_sptime_luaaset_base_str }
1359     }
1360     \str_if_eq:VnTF \l__spintent_sptime_luaaset_is_fraction_str { true }
1361     {
1362       \__spintent_sptime_build_seconds:nn { con }
1363       { \l__spintent_sptime_luaaset_seconds_str }
1364     }
1365     {
1366       \__spintent_sptime_build_seconds:nn { minutos-con }
1367       { \l__spintent_sptime_luaaset_seconds_str }
1368     }
1369   }
1370   {
1371     \MathMLintent { :time }
1372     {
1373       \text { \l__spintent_sptime_luaaset_base_str }
1374     }
1375     \str_if_eq:VnTF \l__spintent_sptime_luaaset_is_fraction_str { false }
1376     {
1377       \MathMLintent { minutos }
1378       {
1379         \__spintent_invisible_sep:
1380       }
1381     }
1382   }
1383 }

```

(End of definition for `\__spintent_sptime_build_tree:.`)

\sptime The command `\sptime` is the public interface for formatting time expressions. It first checks if it is being used in math mode, issuing an error if not. Inside a local group, it parses the input using `\__spintent_luafun_sptime_parse:n` and checks the error status in `\l__spintent_sptime_luaaset_error_str`. If an invalid format is detected, it issues the `invalid-time-format` error; otherwise, it calls `\__spintent_sptime_build_tree:` to assemble the `MathML` output.

```

1384 \NewDocumentCommand \sptime { m }
1385 {
1386   \mode_if_math:TF
1387   {
1388     \group_begin:
1389     \__spintent_luafun_sptime_parse:n { #1 }
1390     \str_if_eq:VnTF \l__spintent_sptime_luaaset_error_str { true }
1391     {
1392       \msg_error:nnnn { spintent } { invalid-time-format }
1393       { \sptime } { #1 }
1394     }
1395     {
1396       \__spintent_sptime_build_tree:
1397     }
1398   \group_end:
1399 }
1400 { \msg_error:nnn { spintent } { need-math-mode } { \sptime } }
1401 }

```

(End of definition for `\sptime`. This function is documented on page 11.)

11.19 The command `\spdate`

The command `\spdate` is the main public command for typesetting and tagging date expressions. It parses date string formats (such as DD/MM/YYYY or YYYY-MM-DD) using the Lua function `\__spintent_luafun_spdate_parse:n`. This verifies the input and constructs a `MathML` `intent` tree for screen readers.

```
\l__spintent_spdate_luaset_output_str
\l__spintent_spdate_luaset_error_str
```

These internal string variables store the values returned from the Lua module `spintent.lua`. They hold the formatted date string and the error status used during formatting.

```
1402 \str_new:N \l__spintent_spdate_luaset_output_str
1403 \str_new:N \l__spintent_spdate_luaset_error_str
```

(End of definition for `\l__spintent_spdate_luaset_output_str` and `\l__spintent_spdate_luaset_error_str`.)

11.19.1 Definition of error messages

```
invalid-date-format
```

The error message `invalid-date-format` defines the text displayed when an invalid date input is detected. It is triggered when an argument fails the format checks or represents an invalid calendar date, such as `31/02/2024` or using an unsupported pattern like `YYYY/MM/DD`.

```
1404 \msg_new:nnnn { spintent } { invalid-date-format }
1405 {
1406   Invalid~date~or~unsupported~format~in~#1. \\\
1407   The~argument~'#2'~does~not~match~a~supported~format.
1408 }
1409 {
1410   Allowed~formats:~DD\slash MM\slash YYYY,~DD-MM-YYYY,~or~YYYY-MM-DD. \\\
1411   Ensure~the~day~and~month~are~valid~(e.g.,~avoid~31\slash 02\slash 2024).
1412 }
```

(End of definition for `invalid-date-format`.)

11.19.2 Internal functions for `\spdate`

```
\__spintent_spdate_process:n
```

The internal function `\__spintent_spdate_process:n` parses the input using `\__spintent_luafun_spdate_parse:n` and checks the error status in `\l__spintent_spdate_luaset_error_str`. If an invalid format is detected, it issues the `invalid-date-format` error. Otherwise, it builds the `MathML intent` structure using the formatted date stored in `\l__spintent_spdate_luaset_output_str`.

```
1413 \cs_new_protected:Nn \__spintent_spdate_process:n
1414 {
1415   \__spintent_luafun_spdate_parse:n { #1 }
1416   \str_if_eq:VnTF \l__spintent_spdate_luaset_error_str { true }
1417   {
1418     \msg_error:nnnn { spintent } { invalid-date-format }
1419     { \spdate } { #1 }
1420   }
1421   {
1422     \MathMLintent { :date }
1423     {
1424       \text { \l__spintent_spdate_luaset_output_str }
1425     }
1426   }
1427 }
```

(End of definition for `\__spintent_spdate_process:n`.)

```
\spdate
```

The command `\spdate` is the public interface for formatting date expressions. It first checks if it is being used in math mode, issuing an error if not. Inside a local group, it calls `\__spintent_spdate_process:n` to process the input.

```
1428 \NewDocumentCommand \spdate { m }
1429 {
1430   \mode_if_math:TF
1431   {
1432     \group_begin:
1433     \__spintent_spdate_process:n { #1 }
1434     \group_end:
1435   }
1436   { \msg_error:nnn { spintent } { need-math-mode } { \spdate } }
1437 }
```

(End of definition for `\spdate`. This function is documented on page 11.)

11.20 Semantic Operators

This section provides purely semantic operators. Unlike their formatting counterparts (which parse and format numbers), these commands do not process mandatory arguments. Instead, they solely inject a mathematical glyph wrapped in its corresponding regional `MathCAT` intent. This grants teachers absolute control when building complex algebraic expressions, mixed numbers, or fraction divisions.

11.21 The command `\spusep`

For the use of parentheses, we will provide the command `\spusep` with only two $\langle keys \rangle$ `size` and `silent`. Most of the time we can use `\left` and `\right` or `\Uleft` and `\Uright`, but both forms create a group, and this hinders the code for *tagged* PDF when we want to interrupt something between them.

`spusep-empty-arg`
`spusep-invalid-arg`

The first “error” message `spusep-empty-arg` is raised when `\spusep` receives an empty or blank $\{\langle argument \rangle\}$, the second this is when an input is invalid.

```

1438 \msg_new:nnnn { spintent } { spusep-empty-arg }
1439 { Empty~argument~in~\token_to_str:N \spusep. }
1440 { \token_to_str:N \spusep~requires~a~non-empty~argument. }
1441 \msg_new:nnn { spintent } { spusep-invalid-arg }
1442 {
1443   Invalid~argument~'#1'~for~\token_to_str:N \spusep.
1444 }

```

(End of definition for `spusep-empty-arg` and `spusep-invalid-arg`.)

The variable `\l__spintent_spusep_arg_tl` stores the $\{\langle argument \rangle\}$, the variable `\l__spintent_spusep_left_tl` and `\l__spintent_spusep_right_tl` stores the parentheses.

```

1445 \tl_new:N \l__spintent_spusep_arg_tl
1446 \tl_new:N \l__spintent_spusep_left_tl
1447 \tl_new:N \l__spintent_spusep_right_tl

```

(End of definition for `\l__spintent_spusep_arg_tl`, `\l__spintent_spusep_left_tl`, and `\l__spintent_spusep_right_tl`.)

`size`
`silent`
`unknown`

We define the $\langle keys \rangle$ `size` and `silent`.

```

1448 \keys_define:nn { spintent / spusep }
1449 {
1450   size .choices:nn =
1451     { none , big , Big, bigg, Bigg }
1452     {
1453       \int_case:nn { \l_keys_choice_int }
1454       {
1455         { 1 }
1456         {
1457           \tl_clear:N \l__spintent_spusep_left_tl
1458           \tl_clear:N \l__spintent_spusep_right_tl
1459         }
1460         { 2 }
1461         {
1462           \tl_set:Nn \l__spintent_spusep_left_tl { \bigl }
1463           \tl_set:Nn \l__spintent_spusep_right_tl { \bigr }
1464         }
1465         { 3 }
1466         {
1467           \tl_set:Nn \l__spintent_spusep_left_tl { \Bigl }
1468           \tl_set:Nn \l__spintent_spusep_right_tl { \Bigr }
1469         }
1470         { 4 }
1471         {
1472           \tl_set:Nn \l__spintent_spusep_left_tl { \biggl }
1473           \tl_set:Nn \l__spintent_spusep_right_tl { \biggr }
1474         }
1475         { 5 }
1476         {
1477           \tl_set:Nn \l__spintent_spusep_left_tl { \Biggl }
1478           \tl_set:Nn \l__spintent_spusep_right_tl { \Biggr }
1479         }
1480       }
1481     },
1482   size .initial:n = none,
1483   size .value_required:n = true,
1484   size / unknown .code:n =
1485     \msg_error:nneee { spintent } { unknown-choice }
1486     { size } { none,~big,~Big,~bigg,~Bigg } { \exp_not:n {#1} },
1487   silent .bool_set:N = \l__spintent_spusep_silent_bool,
1488   silent .initial:n = false,
1489   silent .value_required:n = true,
1490   unknown .code:n =
1491     {
1492       \str_set:Nn \l__spintent_command_name_str { spusep }

```

```

1493     \__spintnt_unknown_keys_cmd:n {#1}
1494   },
1495 }

```

(End of definition for `size`, `silent`, and `unknown`.)

`\__spintnt_spusep_set_parens:nn`

The function `\__spintnt_spusep_set_parens:nn` will place the argument passed in ‘#2’ to the right of the variables `\__spintnt_spusep_left_tl` and `\__spintnt_spusep_right_tl` and then check the state of the variable `\__spintnt_spusep_silent_bool` handled by the key `silent` to print.

```

1496 \cs_new_protected:Nn \__spintnt_spusep_set_parens:nn
1497 {
1498   \tl_put_right:cn { \__spintnt_spusep_ #1 _tl } { #2 }
1499   \bool_if:NTF \__spintnt_spusep_silent_bool
1500   {
1501     \MathMLintnt { :silent } { \tl_use:c { \__spintnt_spusep_ #1 _tl } }
1502   }
1503   {
1504     \tl_use:c { \__spintnt_spusep_ #1 _tl }
1505   }
1506 }

```

(End of definition for `\__spintnt_spusep_set_parens:nn`.)

`\__spintnt_spusep_convert:n`

The function `\__spintnt_spusep_convert:n` captures and stores the $\langle argument \rangle$ passed in ‘#1’ in the variable `\__spintnt_spusep_arg_tl`, validates the cases by converting them to the macros provided by `unicode-math`, and passes them to the function `\__spintnt_spusep_set_parens:nn` for printing. If the input is not supported or is empty, we return an “error”.

```

1507 \cs_new_protected:Nn \__spintnt_spusep_convert:n
1508 {
1509   \tl_set:Ne \__spintnt_spusep_arg_tl { \tl_trim_spaces:n {#1} }
1510   \tl_if_empty:NT \__spintnt_spusep_arg_tl
1511   {
1512     \msg_error:nnn { spintnt } { spusep-invalid-arg } {#1}
1513   }
1514   \str_case:VnF \__spintnt_spusep_arg_tl
1515   {
1516     { ( }
1517     {
1518       \__spintnt_spusep_set_parens:nn { left } { \lparen }
1519     }
1520     { ) }
1521     {
1522       \__spintnt_spusep_set_parens:nn { right } { \rparen }
1523     }
1524     { [ ]
1525     {
1526       \__spintnt_spusep_set_parens:nn { left } { \lbrack }
1527     }
1528     { ] }
1529     {
1530       \__spintnt_spusep_set_parens:nn { right } { \rbrack }
1531     }
1532     { \{ }
1533     {
1534       \__spintnt_spusep_set_parens:nn { left } { \lbrace }
1535     }
1536     { \} }
1537     {
1538       \__spintnt_spusep_set_parens:nn { right } { \rbrace }
1539     }
1540   }
1541   {
1542     \msg_error:nnn { spintnt } { spusep-invalid-arg } {#1}
1543   }
1544 }

```

(End of definition for `\__spintnt_spusep_convert:n`.)

`\spusep`

The command `\spusep` checks that it is running in math mode, opens a local group, sets the keys passed in ‘#1’ and processes the $\langle argument \rangle$ passed in ‘#2’ by means of the function `\__spintnt_spusep_convert:n`.

```

1545 \NewDocumentCommand \spusep { 0 } m {

```

```

1546 {
1547   \mode_if_math:TF
1548   {
1549     \group_begin:
1550     \keys_set:nn { spintent / spusep } {#1}
1551     \__spintent_spusep_convert:n {#2}
1552     \group_end:
1553   }
1554   { \msg_error:nnn { spintent } { need-math-mode } { \spusep } }
1555 }

```

(End of definition for `\spusep`. This function is documented on page ??.)

11.21.1 The command `\spread`

The command `\spread` provides a semantic read for `MathCAT` of mathematical symbols for situations in which it is necessary to specify gender or plurality in Spanish.

`spread-empty-args` The error message `spread-empty-args` is raised when `\spread` receives an empty or blank $\langle argument \rangle$.

```

1556 \msg_new:nnnn { spintent } { spread-empty-args }
1557 { Empty~argument~in~\token_to_str:N \spread. }
1558 { \token_to_str:N \spread~requires~two~non-empty~arguments. }

```

(End of definition for `spread-empty-args`.)

`\__spintent_spread_intent_tl` The variable `\__spintent_spread_intent_tl` stores the sanitized semantic reading $\langle argument \rangle$ for symbol to be injected into the `\MathMLintent`.

```

1559 \tl_new:N \__spintent_spread_intent_tl

```

(End of definition for `\__spintent_spread_intent_tl`.)

`\__spintent_spread_exec:nn` The function `\__spintent_spread_exec:nn` implements the core logic for `\spread`. It verifies that neither $\langle argument \rangle$ is empty or blank. If valid, it evaluates the *reading* $\langle argument \rangle$ ‘#1’ using `\__spintent-sanitize_affix:Nn` and embeds it via `\MathMLintent` into the *math symbol* ‘#2’.

```

1560 \cs_new_protected:Nn \__spintent_spread_exec:nn
1561 {
1562   \bool_lazy_or:nnTF { \tl_if_blank_p:n { #1 } } { \tl_if_blank_p:n { #2 } }
1563   { \msg_error:nn { spintent } { spread-empty-args } }
1564   {
1565     \__spintent_sanitize_affix:Nn \__spintent_spread_intent_tl { #1 }
1566     \MathMLintent { \__spintent_spread_intent_tl } { #2 }
1567   }
1568 }

```

(End of definition for `\__spintent_spread_exec:nn`.)

`\spread` The command `\spread` assigns a custom semantic *reading* ‘#1’ to any mathematical *symbol* ‘#2’. It enforces math mode and passes execution to `\__spintent_spread_exec:nn`.

```

1569 \NewDocumentCommand \spread { m m }
1570 {
1571   \mode_if_math:TF
1572   {
1573     \group_begin:
1574     \__spintent_spread_exec:nn { #1 } { #2 }
1575     \group_end:
1576   }
1577   { \msg_error:nnn { spintent } { need-math-mode } { \spread } }
1578 }

```

(End of definition for `\spread`. This function is documented on page 11.)

11.21.2 The command `\spdsym`

`\__spintent_spdsym_read_sym_tl` Variables to store the regional reading string, the visual operator, and the optional semantic affixes specifically for `\spdsym`.

```

\__spintent_spdsym_symbol_tl
\__spintent_spdsym_prefix_tl
\__spintent_spdsym_suffix_tl
1579 \tl_new:N \__spintent_spdsym_read_sym_tl
1580 \tl_new:N \__spintent_spdsym_symbol_tl
1581 \tl_new:N \__spintent_spdsym_prefix_tl
1582 \tl_new:N \__spintent_spdsym_suffix_tl

```

(End of definition for `\__spintent_spdsym_read_sym_tl` and others.)

```

prefix We define the (keys) prefix, suffix, set-sym and read-sym for \spdsym.
suffix
set-sym
read-sym
unknown
1583 \keys_define:nn { spintent / spdsym }
1584 {
1585   prefix .code:n =
1586     { \__spintent_sanitizetl { \__spintent_spdsym_prefix_tl {#1} },
1587     prefix .initial:n = ,
1588     prefix .value_required:n = true,
1589     suffix .code:n =
1590       { \__spintent_sanitizetl { \__spintent_spdsym_suffix_tl {#1} },
1591       suffix .initial:n = ,
1592       suffix .value_required:n = true,
1593       set-sym .choices:nn =
1594         { old-style , two-dots , slash }
1595         {
1596           \int_case:nn { \l_keys_choice_int }
1597           {
1598             {1} { \tl_set:Nn \__spintent_spdsym_symbol_tl { \div } }
1599             {2} { \tl_set:Nn \__spintent_spdsym_symbol_tl { : } }
1600             {3} { \tl_set:Nn \__spintent_spdsym_symbol_tl { / } }
1601           }
1602         },
1603       set-sym .initial:n = two-dots,
1604       set-sym .value_required:n = true,
1605       set-sym / unknown .code:n =
1606         \msg_error:nneee { spintent } { unknown-choice }
1607         { set-sym } { old-style, ~two-dots, ~slash } { \exp_not:n {#1} },
1608       read-sym .choices:nn =
1609         { dividido , dividido-por , dividido-entre }
1610         {
1611           \int_case:nn { \l_keys_choice_int }
1612           {
1613             {1} { \tl_set:Nn \__spintent_spdsym_read_sym_tl { dividido } }
1614             {2} { \tl_set:Nn \__spintent_spdsym_read_sym_tl { dividido-por } }
1615             {3} { \tl_set:Nn \__spintent_spdsym_read_sym_tl { dividido-entre } }
1616           }
1617         },
1618       read-sym .initial:n = dividido,
1619       read-sym .value_required:n = true,
1620       read-sym / unknown .code:n =
1621         \msg_error:nneee { spintent } { unknown-choice }
1622         { read-sym } { dividido, ~dividido-por, ~dividido-entre }
1623         { \exp_not:n {#1} },
1624       unknown .code:n =
1625         {
1626           \str_set:Nn \__spintent_command_name_str { spdsym }
1627           \__spintent_unknown_keys_cmd:n {#1}
1628         },
1629 }

```

(End of definition for prefix and others.)

\spdsym Defines the **\spdsym** operator. It takes a single optional argument for local setup and outputs the configured intent structure, safely attaching prefixes and suffixes if present.

```

1630 \NewDocumentCommand \spdsym { 0{} }
1631 {
1632   \mode_if_math:TF
1633   {
1634     \group_begin:
1635     \keys_set:nn { spintent / spdsym } { #1 }
1636     \MathMLintent
1637     {
1638       \tl_if_empty:NF \__spintent_spdsym_prefix_tl
1639       { \__spintent_spdsym_prefix_tl - }
1640       \__spintent_spdsym_read_sym_tl
1641       \tl_if_empty:NF \__spintent_spdsym_suffix_tl
1642       { - \__spintent_spdsym_suffix_tl }
1643     }
1644     { \__spintent_spdsym_symbol_tl }
1645     \group_end:
1646   }
1647   { \msg_error:nnn { spintent } { need-math-mode } { \spdsym } }
1648 }

```

(End of definition for `\spdsym`. This function is documented on page 12.)

11.21.3 The command `\spmsym`

`\l__spinttent_spmsym_read_sym_tl`
`\l__spinttent_spmsym_symbol_tl`
`\l__spinttent_spmsym_prefix_tl`
`\l__spinttent_spmsym_suffix_tl`

Variables to store the regional reading string, the visual operator, and the optional semantic affixes specifically for `\spmsym`.

```
1649 \tl_new:N \l__spinttent_spmsym_read_sym_tl
1650 \tl_new:N \l__spinttent_spmsym_symbol_tl
1651 \tl_new:N \l__spinttent_spmsym_prefix_tl
1652 \tl_new:N \l__spinttent_spmsym_suffix_tl
```

(End of definition for `\l__spinttent_spmsym_read_sym_tl` and others.)

```
prefix We define the <keys> prefix, suffix, set-sym and read-sym for \spmsym.
suffix
set-sym
read-sym
unknown

1653 \keys_define:nn { spintent / spmsym }
1654 {
1655   prefix .code:n =
1656   { \l__spinttent_sanitizize_affix:Nn \l__spinttent_spmsym_prefix_tl {#1} },
1657   prefix .initial:n = ,
1658   prefix .value_required:n = true,
1659   suffix .code:n =
1660   { \l__spinttent_sanitizize_affix:Nn \l__spinttent_spmsym_suffix_tl {#1} },
1661   suffix .initial:n = ,
1662   suffix .value_required:n = true,
1663   set-sym .choices:nn =
1664   { cross , dot , asterisk }
1665   {
1666     \int_case:nn { \l_keys_choice_int }
1667     {
1668       {1} { \tl_set:Nn \l__spinttent_spmsym_symbol_tl { \times } }
1669       {2} { \tl_set:Nn \l__spinttent_spmsym_symbol_tl { \cdot } }
1670       {3} { \tl_set:Nn \l__spinttent_spmsym_symbol_tl { \ast } }
1671     }
1672   },
1673   set-sym .initial:n = dot,
1674   set-sym .value_required:n = true,
1675   set-sym / unknown .code:n =
1676   \msg_error:nneee { spintent } { unknown-choice }
1677   { set-sym } { cross, ~dot, ~asterisk } { \exp_not:n {#1} },
1678   read-sym .choices:nn =
1679   { por , multiplicado-por , veces }
1680   {
1681     \int_case:nn { \l_keys_choice_int }
1682     {
1683       {1} { \tl_set:Nn \l__spinttent_spmsym_read_sym_tl { por } }
1684       {2} { \tl_set:Nn \l__spinttent_spmsym_read_sym_tl { multiplicado-por } }
1685       {3} { \tl_set:Nn \l__spinttent_spmsym_read_sym_tl { veces } }
1686     }
1687   },
1688   read-sym .initial:n = por,
1689   read-sym .value_required:n = true,
1690   read-sym / unknown .code:n =
1691   \msg_error:nneee { spintent } { unknown-choice }
1692   { read-sym } { por, ~multiplicado-por, ~veces } { \exp_not:n {#1} },
1693   unknown .code:n =
1694   {
1695     \str_set:Nn \l__spinttent_command_name_str { spmsym }
1696     \l__spinttent_unknown_keys_cmd:n {#1}
1697   },
1698 }
```

(End of definition for `prefix` and others.)

`\spmsym` Defines the `\spmsym` operator. Similar to `\spdsym`, it outputs the correct mathematical glyph based on the local *<keys>*, safely attaching prefixes and suffixes if present.

```
1699 \NewDocumentCommand \spmsym { 0{} }
1700 {
1701   \mode_if_math:TF
1702   {
1703     \group_begin:
1704     \keys_set:nn { spintent / spmsym } { #1 }
1705     \MathMLintent
```



```

1706         {
1707             \tl_if_empty:NF \l__spintent_spmsym_prefix_tl
1708             { \l__spintent_spmsym_prefix_tl - }
1709             \l__spintent_spmsym_read_sym_tl
1710             \tl_if_empty:NF \l__spintent_spmsym_suffix_tl
1711             { - \l__spintent_spmsym_suffix_tl }
1712         }
1713         { \l__spintent_spmsym_symbol_tl }
1714     \group_end:
1715 }
1716 { \msg_error:nnn { spintent } { need-math-mode } { \spmsym } }
1717 }

```

(End of definition for `\spmsym`. This function is documented on page 12.)

11.22 The command `\spdiv`

The user-level command `\spdiv` formats division representations. It takes two mandatory arguments: the `{⟨dividend⟩}` and the `{⟨divisor⟩}`. Both `{⟨arguments⟩}` are internally processed by the `\spnum` command to ensure proper digit separation. This command enforces strict validation, ensuring that neither the `{⟨dividend⟩}` nor the `{⟨divisor⟩}` is *blank*.

11.22.1 Definition of keys for `\spdiv`

We define the `⟨keys⟩` `wrap-parens` and `read-parens` for `\spdiv` command. Number-related `⟨keys⟩` `cifras-sep`, `read-sign`, `read-period-sep` and `notation-sym` are forwarded to `\spnum` command using `.meta:nn`, while semantic operator `⟨keys⟩` `set-sym` and `read-sym` are forwarded to the `\spdsym` command using `.meta:nn`.

```

1718 \keys_define:nn { spintent / spdiv }
1719 {
1720     cifras-sep      .meta:nn = { spintent / spnum } { cifras-sep      = {#1} },
1721     read-sign       .meta:nn = { spintent / spnum } { read-sign       = {#1} },
1722     read-period-sep .meta:nn = { spintent / spnum } { read-period-sep = {#1} },
1723     notation-sym    .meta:nn = { spintent / spnum } { notation-sym    = {#1} },
1724     set-sym         .meta:nn = { spintent / spdsym } { set-sym         = {#1} },
1725     read-sym        .meta:nn = { spintent / spdsym } { read-sym        = {#1} },
1726     wrap-parens     .bool_set:N = \l__spintent_spdiv_wrap_parens_bool,
1727     wrap-parens     .initial:n = false,
1728     wrap-parens     .value_required:n = true,
1729     read-parens     .bool_set:N = \l__spintent_spdiv_read_parens_bool,
1730     read-parens     .initial:n = true,
1731     read-parens     .value_required:n = true,
1732     unknown         .code:n =
1733     {
1734         \str_set:Nn \l__spintent_command_name_str { spdiv }
1735         \l__spintent_unknown_keys_cmd:n {#1}
1736     },
1737 }

```

(End of definition for `cifras-sep` and others.)

11.22.2 Definition of error messages

`spdiv-empty-argument` Raised when either the `{⟨dividend⟩}` or the `{⟨divisor⟩}` is missing or *blank*.

```

1738 \msg_new:nnnn { spintent } { spdiv-empty-argument }
1739 { The~arguments~for~\token_to_str:N \spdiv~cannot~be~empty. }
1740 {
1741     You~must~provide~both~a~dividend~and~a~divisor.\\
1742 }

```

(End of definition for `spdiv-empty-argument`.)

11.22.3 Internal functions for `\spdiv`

`\__spintent_spdiv_render_op:nn` The function `\__spintent_spdiv_render_op:nn` parses both `{⟨arguments⟩}` through the `\spnum` command and embeds the semantic operator from `\spdsym`.

```

1743 \cs_new_protected:Nn \__spintent_spdiv_render_op:nn
1744 {
1745     \spnum {#1}
1746     \MathMLintent { \l__spintent_spdsym_read_sym_tl }
1747     {
1748         \l__spintent_spdsym_symbol_tl
1749     }
1750     \spnum {#2}
1751 }

```

(End of definition for `\__spintent_spdiv_render_op:nn`.)

`\__spintent_spdiv_render_parens:nn`

The function `\__spintent_spdiv_render_parens:nn` handles the structural parentheses around the division.

```

1752 \cs_new_protected:Nn \__spintent_spdiv_render_parens:nn
1753 {
1754   \bool_if:NTF \__spintent_spdiv_read_parens_bool
1755   {
1756     ( \__spintent_spdiv_render_op:nn {#1} {#2} )
1757   }
1758   {
1759     \MathMLintent { :silent } { ( }
1760     \__spintent_spdiv_render_op:nn {#1} {#2}
1761     \MathMLintent { :silent } { ) }
1762   }
1763 }

```

(End of definition for `\__spintent_spdiv_render_parens:nn`.)

`\__spintent_spdiv_process:nnn`

The function `\__spintent_spdiv_process:nnn` sets $\langle keys \rangle$, validates $\{ \langle arguments \rangle \}$, and decides on parenthesis wrapping.

```

1764 \cs_new_protected:Nn \__spintent_spdiv_process:nnn
1765 {
1766   \keys_set:nn { spintent / spdiv } { #1 }
1767   \tl_if_blank:nTF { #2 }
1768   { \msg_error:nnn { spintent } { spdiv-empty-argument } }
1769   {
1770     \tl_if_blank:nTF { #3 }
1771     { \msg_error:nnn { spintent } { spdiv-empty-argument } }
1772     {
1773       \bool_if:NTF \__spintent_spdiv_wrap_parens_bool
1774       { \__spintent_spdiv_render_parens:nn {#2} {#3} }
1775       { \__spintent_spdiv_render_op:nn {#2} {#3} }
1776     }
1777   }
1778 }

```

(End of definition for `\__spintent_spdiv_process:nnn`.)

`\spdiv` Defines the user command `\spdiv`. It strictly relies on *math mode*.

```

1779 \NewDocumentCommand \spdiv { 0{} m m }
1780 {
1781   \mode_if_math:TF
1782   {
1783     \group_begin:
1784     \__spintent_spdiv_process:nnn { #1 } { #2 } { #3 }
1785     \group_end:
1786   }
1787   { \msg_error:nnn { spintent } { need-math-mode } { \spdiv } }
1788 }

```

(End of definition for `\spdiv`. This function is documented on page 13.)

11.23 The command `\spmul`

The user-level command `\spmul` formats multiplication representations. It takes two mandatory arguments: the $\{ \langle first factor \rangle \}$ and the $\{ \langle second factor \rangle \}$. Both $\{ \langle arguments \rangle \}$ are internally processed by the `\spnum` command to ensure proper digit separation. This command enforces strict validation, ensuring that neither the $\{ \langle first factor \rangle \}$ nor the $\{ \langle second factor \rangle \}$ is *blank*.

11.23.1 Definition of keys for `\spmul`

We define the $\langle keys \rangle$ wrap-parens and read-parens for `\spmul` command. Number-related $\langle keys \rangle$ `cifras-sep`, `read-sign`, `read-period-sep` and `notation-sym` are forwarded to `\spnum` command using `.meta:nn`, while semantic operator $\langle keys \rangle$ `set-sym` and `read-sym` are forwarded to the `\spmsym` command using `.meta:nn`.

```

1789 \keys_define:nn { spintent / spmul }
1790 {
1791   cifras-sep      .meta:nn = { spintent / spnum } { cifras-sep      = {#1} },
1792   read-sign       .meta:nn = { spintent / spnum } { read-sign       = {#1} },
1793   read-period-sep .meta:nn = { spintent / spnum } { read-period-sep = {#1} },
1794   notation-sym    .meta:nn = { spintent / spnum } { notation-sym    = {#1} },
1795   set-sym         .meta:nn = { spintent / spmsym } { set-sym         = {#1} },

```

```

1796   read-sym      .meta:nn = { spintent / spmsym } { read-sym      = {#1} },
1797   wrap-parens   .bool_set:N = \l__spintent_spmul_wrap_parens_bool,
1798   wrap-parens   .initial:n = false,
1799   wrap-parens   .value_required:n = true,
1800   read-parens   .bool_set:N = \l__spintent_spmul_read_parens_bool,
1801   read-parens   .initial:n = true,
1802   read-parens   .value_required:n = true,
1803   unknown       .code:n =
1804   {
1805     \str_set:Nn \l__spintent_command_name_str { spmul }
1806     \__spintent_unknown_keys_cmd:n {#1}
1807   },
1808 }

```

(End of definition for *cifras-sep* and others.)

11.23.2 Definition of error messages

`spmul-empty-argument` Raised when either the $\langle\textit{first factor}\rangle$ or the $\langle\textit{second factor}\rangle$ is missing or *blank*.

```

1809 \msg_new:nnnn { spintent } { spmul-empty-argument }
1810 { The~arguments~for~\token_to_str:N \spmul~cannot~be~empty. }
1811 {
1812   You~must~provide~both~first~factor~and~second~factor.\\
1813 }

```

(End of definition for *spmul-empty-argument*.)

11.23.3 Internal functions for `\spmul`

`\__spintent_spmul_render_op:nn` The function `\__spintent_spmul_render_op:nn` parses both $\langle\textit{arguments}\rangle$ through the `\spnum` command and embeds the semantic operator from `\spmsym`.

```

1814 \cs_new_protected:Nn \__spintent_spmul_render_op:nn
1815 {
1816   \spnum {#1}
1817   \MathMLintent { \l__spintent_spmsym_read_sym_tl }
1818   {
1819     \l__spintent_spmsym_symbol_tl
1820   }
1821   \spnum {#2}
1822 }

```

(End of definition for `\__spintent_spmul_render_op:nn`.)

`\__spintent_spmul_render_parens:nn` The function `\__spintent_spmul_render_parens:nn` handles the structural parentheses around the multiplication.

```

1823 \cs_new_protected:Nn \__spintent_spmul_render_parens:nn
1824 {
1825   \bool_if:NTF \l__spintent_spmul_read_parens_bool
1826   {
1827     ( \__spintent_spmul_render_op:nn {#1} {#2} )
1828   }
1829   {
1830     \MathMLintent { :silent } { ( }
1831     \__spintent_spmul_render_op:nn {#1} {#2}
1832     \MathMLintent { :silent } { ) }
1833   }
1834 }

```

(End of definition for `\__spintent_spmul_render_parens:nn`.)

`\__spintent_spmul_process:nnn` The function `\__spintent_spmul_process:nnn` sets $\langle\textit{keys}\rangle$, validates $\langle\textit{arguments}\rangle$, and decides on parenthesis wrapping.

```

1835 \cs_new_protected:Nn \__spintent_spmul_process:nnn
1836 {
1837   \keys_set:nn { spintent / spmul } { #1 }
1838   \tl_if_blank:nTF { #2 }
1839   { \msg_error:nnn { spintent } { spmul-empty-argument } }
1840   {
1841     \tl_if_blank:nTF { #3 }
1842     { \msg_error:nnn { spintent } { spmul-empty-argument } }
1843     {
1844       \bool_if:NTF \l__spintent_spmul_wrap_parens_bool
1845       { \__spintent_spmul_render_parens:nn {#2} {#3} }
1846       { \__spintent_spmul_render_op:nn {#2} {#3} }

```

```

1847     }
1848   }
1849 }

```

(End of definition for `\__spintent_spmul_process:nnn`.)

`\spmul` Defines the user command `\spmul`. It strictly relies on *math mode*.

```

1850 \NewDocumentCommand \spmul { O{} m m }
1851 {
1852   \mode_if_math:TF
1853   {
1854     \group_begin:
1855       \__spintent_spmul_process:nnn { #1 } { #2 } { #3 }
1856     \group_end:
1857   }
1858   { \msg_error:nnn { spintent } { need-math-mode } { \spmul } }
1859 }

```

(End of definition for `\spmul`. This function is documented on page 13.)

11.24 The commands `\spgcd` and `\spmcm`

The user-level commands `\spgcd` and `\spmcm` provide a semantic interface for the Greatest Common Divisor (MCD in Spanish) and the Least Common Multiple (MCM in Spanish). They delegate argument validation and calculation to the Lua module `spintent.lua`, constructing a valid `MathCAT` intent.

11.24.1 Variables for `\spgcd` and `\spmcm`

Internal variables generated dynamically via a comma-separated list to avoid code duplication for both symmetric commands.

```

\__spintent_spmcm_spmcd_lua_set_error_str
\__spintent_mc_args_seq
\__spintent_spmcd_print_result_bool
\__spintent_spmcm_print_result_bool
\__spintent_spmcd_read_terse_bool
\__spintent_spmcm_read_terse_bool
\__spintent_spmcd_print_upper_bool
\__spintent_spmcm_print_upper_bool
\__spintent_spmcd_prefix_tl
\__spintent_spmcm_prefix_tl
\__spintent_spmcd_suffix_tl
\__spintent_spmcm_suffix_tl
1860 \str_new:N \__spintent_spmcm_spmcd_lua_set_error_str
1861 \seq_new:N \__spintent_mc_args_seq
1862 \clist_map_inline:nn { mcd , mcm }
1863 {
1864   \tl_new:c { \__spintent_sp #1 _lua_set_ #1 _value_tl }
1865   \bool_new:c { \__spintent_sp #1 _print_result_bool }
1866   \bool_new:c { \__spintent_sp #1 _read_terse_bool }
1867   \bool_new:c { \__spintent_sp #1 _print_upper_bool }
1868   \tl_new:c { \__spintent_sp #1 _prefix_tl }
1869   \tl_new:c { \__spintent_sp #1 _suffix_tl }
1870 }

```

(End of definition for `\__spintent_spmcm_spmcd_lua_set_error_str` and others.)

11.24.2 Definition of error messages

`mc-invalid-arguments` Exception triggered when the comma-separated sequence contains negative integers, floats, or alphabetical characters.

```

1871 \msg_new:nnnn { spintent } { mc-invalid-arguments }
1872 {
1873   Invalid~arguments~provided~to~#1.
1874 }
1875 {
1876   The~arguments~must~be~a~comma-separated~list. \\\
1877 }

```

(End of definition for `mc-invalid-arguments`.)

11.24.3 Keys for `\spgcd` and `\spmcm`

`result` We define the shared *keys* `result`, `prefix`, `read-cmd`, `upper-case`, `sample-arg` and `silent-cmd`.

```

prefix
read-cmd
upper-case
sample-arg
silent-cmd
1878 \clist_map_inline:nn { mcd , mcm }
1879 {
1880   \keys_define:nn { spintent / sp #1 }
1881   {
1882     result .bool_set:c = { \__spintent_sp #1 _print_result_bool },
1883     result .initial:n = false,
1884     result .value_required:n = true,
1885     prefix .code:n =
1886       { \__spintent_sanitize_affix:cn { \__spintent_sp #1 _prefix_tl } {##1} },
1887     prefix .initial:n = ,
1888     prefix .value_required:n = true,
1889     read-cmd .choices:nn =
1890       { terse , clear }
1891     {
1892       \int_case:nn { \l_keys_choice_int }

```

```

1893         {
1894             {1} { \bool_set_true:c { l__spintent_sp #1 _read_terse_bool } }
1895             {2} { \bool_set_false:c { l__spintent_sp #1 _read_terse_bool } }
1896         }
1897     },
1898     read-cmd .initial:n = clear,
1899     read-cmd .value_required:n = true,
1900     read-cmd / unknown .code:n =
1901         \msg_error:nneee { spintent } { unknown-choice }
1902         { read-cmd } { terse , ~clear } { \exp_not:n {##1} },
1903     upper-case .bool_set:c = { l__spintent_sp #1 _print_upper_bool },
1904     upper-case .initial:n = false,
1905     upper-case .value_required:n = true,
1906     sample-arg .bool_set:c = { l__spintent_sp #1 _sample_arg_bool },
1907     sample-arg .initial:n = false,
1908     sample-arg .value_required:n = true,
1909     silent-cmd .bool_set:c = { l__spintent_sp #1 _silent_cmd_bool },
1910     silent-cmd .initial:n = false,
1911     silent-cmd .value_required:n = true,
1912     unknown .code:n =
1913         {
1914             \str_set:Nn \l__spintent_command_name_str { sp #1 }
1915             \__spintent_unknown_keys_cmd:n {##1}
1916         },
1917     }
1918 }

```

(End of definition for *result* and others.)

11.24.4 Internal functions for \spmc and \spmc

__spintent_mc_process_args:nn

The function `\__spintent_mc_process_args:nn` parses the operation ‘#1’ and its arguments ‘#2’, injecting **MathML** intents.

```

1919 \cs_new_protected:Nn \__spintent_mc_process_args:nn
1920 {
1921     \seq_set_from_clist:Nn \l__spintent_mc_args_seq {#2}
1922     \bool_if:cTF { l__spintent_sp #1 _silent_cmd_bool }
1923     {
1924         \seq_set_map:Nnn \l__spintent_mc_args_seq \l__spintent_mc_args_seq
1925         { \MathMLintent { :silent } { ##1 } }
1926         \MathMLintent { entre:silent } { ( }
1927         \seq_use:Nnnn \l__spintent_mc_args_seq
1928         { \MathMLintent { _y:silent } { , } }
1929         { \MathMLintent { :silent } { , } }
1930         { \MathMLintent { _y:silent } { , } }
1931     }
1932     {
1933         \MathMLintent { entre } { ( }
1934         \seq_use:Nnnn \l__spintent_mc_args_seq
1935         { \MathMLintent { _y } { , } }
1936         { \MathMLintent { :silent } { , } }
1937         { \MathMLintent { _y } { , } }
1938     }
1939     \MathMLintent { :silent } { ) }
1940 }

```

(End of definition for `\__spintent_mc_process_args:nn`.)

__spintent_spmc_process:nnnn

The internal function `\__spintent_spmc_process:nnnn` serves as a shared central handler for both the `\spmc` and `\spmc` commands. It takes the command identifier ‘#1’, its full reading string ‘#2’, the local *(keys)* ‘#3’, and the optional values ‘#4’. It dynamically resolves variables using the identifier to build the **MathML** *intent* string (including any prefixes or suffixes) and prints the math operator. If values are provided in ‘#4’, it calculates the result using the Lua backend. If an error occurs, it throws the `mc-invalid-arguments` error; otherwise, it processes the arguments and conditionally prints the calculated result.

```

1941 \cs_new_protected:Nn \__spintent_spmc_process:nnnn
1942 {
1943     \keys_set:nn { spintent / sp #1 } { #3 }
1944     \MathMLintent
1945     {
1946         \tl_if_empty:cF { l__spintent_sp #1 _prefix_tl }
1947         { \tl_use:c { l__spintent_sp #1 _prefix_tl } - }
1948         \bool_if:cTF { l__spintent_sp #1 _read_terse_bool }

```

```

1949     { #1 } { #2 }
1950     \bool_if:cT { l__spintent_sp #1 _silent_cmd_bool }
1951     {
1952         \tl_if_novalue:nTF {#4}
1953         { :silent }
1954         { \bool_if:cT { l__spintent_sp #1 _sample_arg_bool } { :silent } }
1955     }
1956 }
1957 {
1958     \operatorname
1959     {
1960         \bool_if:cTF { l__spintent_sp #1 _print_upper_bool }
1961         { \text_uppercase:n {#1} } { #1 }
1962     }
1963 }
1964 \tl_if_novalue:nF { #4 }
1965 {
1966     \bool_if:cTF { l__spintent_sp #1 _sample_arg_bool }
1967     {
1968         \__spintent_mc_process_args:nn {#1} {#4}
1969     }
1970     {
1971         \use:c { __spintent_luafun_calculate_ #1 :n } { #4 }
1972         \str_if_eq:VnTF \l__spintent_spmcm_spmcd_luaset_error_str { true }
1973         {
1974             \msg_error:nne { spintent } { mc-invalid-arguments } { \exp_not:c { sp #1 } }
1975         }
1976         {
1977             \__spintent_mc_process_args:nn {#1} {#4}
1978             \bool_if:cT { l__spintent_sp #1 _print_result_bool }
1979             {
1980                 \MathMLintent { es-igual-a } { = }
1981                 \tl_use:c { l__spintent_sp #1 _luaset_ #1 _value_tl }
1982             }
1983         }
1984     }
1985 }
1986 }

```

(End of definition for `\__spintent_spmc_process:nnnn`.)

`\spmcd` The commands `\spmcd` and `\spmcm` act as the public interface wrappers for typesetting the greatest common divisor and least common multiple. They enforce math mode, open a local group, and defer execution to the main processing macro `\__spintent_spmc_process:nnnn`, passing along the appropriate reading strings and arguments.

```

1987 \NewDocumentCommand \spmcd { 0{} d() }
1988 {
1989     \mode_if_math:TF
1990     {
1991         \group_begin:
1992         \__spintent_spmc_process:nnnn { mcd } { máximo-común-divisor } {#1} {#2}
1993         \group_end:
1994     }
1995     { \msg_error:nnn { spintent } { need-math-mode } { \spmcd } }
1996 }
1997 \NewDocumentCommand \spmcm { 0{} d() }
1998 {
1999     \mode_if_math:TF
2000     {
2001         \group_begin:
2002         \__spintent_spmc_process:nnnn { mcm } { mínimo-común-múltiplo } {#1} {#2}
2003         \group_end:
2004     }
2005     { \msg_error:nnn { spintent } { need-math-mode } { \spmcm } }
2006 }

```

(End of definition for `\spmcd` and `\spmcm`. These functions are documented on page 14.)

11.25 The commands `\spnM` and `\spnD`

The user-level commands `\spnM` and `\spnD` provide a semantic interface for the Multiples of a number (“*múltiplos*” in Spanish) and the Divisors of a number (“*divisores*” in Spanish). They delegate argument validation and calculation to the Lua module `spintent.lua`, constructing a valid `MathCAT` intent.

11.25.1 Variables for \spnM and \spnD

Internal variables generated dynamically for multiples and divisors.

```

\l__spinttent_md_out_seq
\l__spinttent_spnM_luaset_result_tl
\l__spinttent_spnD_luaset_result_tl
\l__spinttent_spnM_luaset_is_truncated_str
\l__spinttent_spnD_luaset_is_truncated_str
\l__spinttent_spnM_print_result_bool
\l__spinttent_spnD_print_result_bool
\l__spinttent_spnM_silent_cmd_bool
\l__spinttent_spnD_silent_cmd_bool
\l__spinttent_spnM_prefix_tl
\l__spinttent_spnD_prefix_tl
\l__spinttent_spnM_print_result_num_tl
\l__spinttent_spnD_print_result_num_tl
2007 \seq_new:N \l__spinttent_md_out_seq
2008 \clist_map_inline:nn { nM , nD }
2009 {
2010     \tl_new:c { \l__spinttent_sp #1_luaset_result_tl }
2011     \str_new:c { \l__spinttent_sp #1_luaset_is_truncated_str }
2012     \bool_new:c { \l__spinttent_sp #1_print_result_bool }
2013     \bool_new:c { \l__spinttent_sp #1_silent_cmd_bool }
2014     \tl_new:c { \l__spinttent_sp #1_prefix_tl }
2015     \tl_new:c { \l__spinttent_sp #1_print_result_num_tl }
2016 }

```

(End of definition for \l__spinttent_md_out_seq and others.)

11.25.2 Keys for \spnM and \spnD

We define the shared *(keys)* result, silent-cmd, result-num and prefix.

```

result
silent-cmd
result-num
prefix
sample-arg
2017 \clist_map_inline:nn { nM , nD }
2018 {
2019     \keys_define:nn { spinttent / sp #1 }
2020     {
2021         result .bool_set:c = { \l__spinttent_sp #1_print_result_bool },
2022         result .initial:n = false,
2023         result .value_required:n = true,
2024         silent-cmd .bool_set:c = { \l__spinttent_sp #1_silent_cmd_bool },
2025         silent-cmd .initial:n = false,
2026         silent-cmd .value_required:n = true,
2027         result-num .tl_set:c = { \l__spinttent_sp #1_print_result_num_tl },
2028         result-num .initial:n = ,
2029         prefix .code:n =
2030             { \__spinttent_sanitizize_affix:cn { \l__spinttent_sp #1_prefix_tl } {##1} },
2031         prefix .initial:n = ,
2032         prefix .value_required:n = true,
2033         sample-arg .bool_set:c = { \l__spinttent_sp #1_sample_arg_bool },
2034         sample-arg .initial:n = false,
2035         sample-arg .value_required:n = true,
2036         unknown .code:n =
2037             {
2038                 \str_set:Nn \l__spinttent_command_name_str { sp #1 }
2039                 \__spinttent_unknown_keys_cmd:n {##1}
2040             },
2041     }
2042 }

```

(End of definition for result and others.)

11.25.3 Definition of error messages for \spnM and \spnD

Exception triggered when the argument for multiples or divisors is invalid.

```

md-invalid-argument
2043 \msg_new:nnnn { spinttent } { md-invalid-argument }
2044 { Invalid~argument~provided~to~#1. }
2045 {
2046     The~argument~must~be~a~single~positive~integer. \l
2047 }

```

(End of definition for md-invalid-argument.)

11.25.4 Internal functions for \spnM and \spnD

The internal function `\__spinttent_spnM_spnD_process:nnnnn` acts as the primary entry point for the multiples and divisors commands. It takes the command identifier ‘#1’, its printed mathematical symbol ‘#2’, its base reading string ‘#3’, the local key-value options ‘#4’, and the optional argument ‘#5’. Its sole purpose is to evaluate and set the provided keys within the current scope before delegating the core branching and typesetting logic to `\__spinttent_spnM_and_D_exec:nnnn`.

```

2048 \cs_new_protected:Nn \__spinttent_spnM_spnD_process:nnnnn
2049 {
2050     \keys_set:nn { spinttent / sp #1 } { #4 }
2051     \__spinttent_spnM_spnD_exec:nnnn { #1 } { #2 } { #3 } { #5 }
2052 }

```

(End of definition for __spinttent_spnM_spnD_process:nnnnn.)

`\__spintent_spnM_spnD_intent:nn`

The internal function `\__spintent_spnM_spnD_intent:nn` resolves the main reading intent string. It takes the command identifier ‘`#1`’ (such as “nM” or “nD”) and the base reading string ‘`#2`’. It automatically prepends the localized prefix if defined and handles specific string normalizations, such as ensuring the correct accentuation for “múltiplos”.

```
2053 \cs_new:Nn \__spintent_spnM_spnD_intent:nn
2054 {
2055   \tl_if_empty:cF { \__spintent_sp #1 _prefix_tl }
2056   { \tl_use:c { \__spintent_sp #1 _prefix_tl } - }
2057   \str_case:nnF { #2 }
2058   { { multiplos } { múltiplos } }
2059   { #2 }
2060 }
```

(End of definition for `\__spintent_spnM_spnD_intent:nn`.)

`\__spintent_spnM_spnD_silent:nn`

The internal function `\__spintent_spnM_and_D_silent:nn` prints the atomized notation structure in full silent mode. It takes the mathematical symbol ‘`#1`’ (such as “D” or “M”) and the variable or explicit argument ‘`#2`’. Every component is wrapped in a silent intent, and an invisible separator is injected between the parentheses to prevent assistive technologies from applying default contextual reading rules like “de”.

```
2061 \cs_new_protected:Nn \__spintent_spnM_spnD_silent:nn
2062 {
2063   \MathMLintent { :silent } { \operatorname { #1 } }
2064   \MathMLintent { :silent } { ( }
2065   \MathMLintent { :silent } { \__spintent_invisible_sep: }
2066   \MathMLintent { :silent } { #2 }
2067   \MathMLintent { :silent } { ) }
2068 }
```

(End of definition for `\__spintent_spnM_spnD_silent:nn`.)

`\__spintent_spnM_spnD_result:n`

The internal function `\__spintent_spnM_and_D_result:n` handles the calculation and accessible formatting of the computed mathematical set. It takes the command identifier ‘`#1`’. It invokes the corresponding Lua backend calculation, maps the resulting clist into a sequence, and formats the final printed output utilizing custom accessibility intents for both separators and conjunctions. It also safely manages truncation indicators.

```
2069 \cs_new_protected:Nn \__spintent_spnM_spnD_result:n
2070 {
2071   \use:c { __spintent_luafun_calcular_ #1 :nn }
2072   { \l__spintent_luaset_part_int_tl }
2073   { \tl_use:c { \__spintent_sp #1 _print_result_num_tl } }
2074   \MathMLintent { son } { = \{ }
2075   \seq_set_from_clist:Ne
2076   \l__spintent_md_out_seq { \tl_use:c { \__spintent_sp #1 _luaset_result_tl } }
2077   \seq_use:Nnnn \l__spintent_md_out_seq
2078   { \MathMLintent { _y } { , } }
2079   { \MathMLintent { :silent } { , } }
2080   { \MathMLintent { _y } { , } }
2081   \str_if_eq:vnT
2082   { \__spintent_sp #1 _luaset_is_truncated_str } { true }
2083   {
2084     \MathMLintent { :silent } { , } ~ \dots
2085   }
2086   \MathMLintent { :silent } { \} }
2087 }
```

(End of definition for `\__spintent_spnM_spnD_result:n`.)

`\__spintent_spnM_spnD_exec:nnnn`

The function `\__spintent_spnM_and_D_exec:nnnn` implements the multiples and divisors commands. It takes the $\langle argument \rangle$ ‘`#1`’ (the command name), the $\langle argument \rangle$ ‘`#2`’ (the mathematical symbol), the base reading $\langle argument \rangle$ ‘`#3`’, and the optional $\langle argument \rangle$ ‘`#4`’. If no $\langle argument \rangle$ is provided, it evaluates the key `silent-cmd` using the default token ‘n’. If an $\langle argument \rangle$ is given, it evaluates the key `sample-arg`. If true, it processes the $\langle argument \rangle$ directly and evaluates the key `silent-cmd`. If false, it bypasses the *silent mode*, validates the $\langle argument \rangle$ as a natural number, and evaluates the key `result` optionally calculate and append the resulting set.

```
2088 \cs_new_protected:Nn \__spintent_spnM_spnD_exec:nnnn
2089 {
2090   \tl_if_novalue:nTF { #4 }
2091   {
2092     \bool_if:cTF { \__spintent_sp #1 _silent_cmd_bool }
2093     { \__spintent_spnM_spnD_silent:nn { #2 } { n } }
2094     {
```

```

2095         \MathMLintent
2096         { \__spinttent_spnM_spnD_intent:nn { #1 } { #3 } }
2097         { \operatorname { #2 } ( n ) }
2098     }
2099 }
2100 {
2101     \bool_if:cTF { l__spinttent_sp #1 _sample_arg_bool }
2102     {
2103         \bool_if:cTF { l__spinttent_sp #1 _silent_cmd_bool }
2104         { \__spinttent_spnM_spnD_silent:nn { #2 } { #4 } }
2105         {
2106             \MathMLintent
2107             { \__spinttent_spnM_spnD_intent:nn { #1 } { #3 } }
2108             { \operatorname { #2 } } ( #4 )
2109         }
2110     }
2111     {
2112         \__spinttent_luafun_clean_split_and_set:n { \exp_not:n { #4 } }
2113         \str_if_eq:VnTF \l__spinttent_luaset_has_natural_str { true }
2114         {
2115             \MathMLintent
2116             { \__spinttent_spnM_spnD_intent:nn { #1 } { #3 } }
2117             { \operatorname { #2 } } ( #4 )
2118             \bool_if:cT { l__spinttent_sp #1 _print_result_bool }
2119             { \__spinttent_spnM_spnD_result:n { #1 } }
2120         }
2121         {
2122             \msg_error:nne { spinttent } { md-invalid-argument } { \exp_not:c { sp #1 } }
2123         }
2124     }
2125 }
2126 }

```

(End of definition for `\__spinttent_spnM_spnD_exec:nnnn`.)

`\spnM` The commands `\spnM` and `\spnD` act as the public interface wrappers for typesetting multiples and divisors, respectively. They enforce math mode, open a local group, and defer execution to the function `\__spinttent_spnM_spnD_process:nnnnn`, passing along the optional *keys* ‘`#1`’ and the numeric *argument* ‘`#2`’.

```

2127 \NewDocumentCommand \spnM { 0{} d() }
2128 {
2129     \mode_if_math:TF
2130     {
2131         \group_begin:
2132         \__spinttent_spnM_spnD_process:nnnnn { nM } { M } { multiplos } { #1 } { #2 }
2133         \group_end:
2134     }
2135     { \msg_error:nnn { spinttent } { need-math-mode } { \spnM } }
2136 }
2137 \NewDocumentCommand \spnD { 0{} d() }
2138 {
2139     \mode_if_math:TF
2140     {
2141         \group_begin:
2142         \__spinttent_spnM_spnD_process:nnnnn { nD } { D } { divisores } { #1 } { #2 }
2143         \group_end:
2144     }
2145     { \msg_error:nnn { spinttent } { need-math-mode } { \spnD } }
2146 }

```

(End of definition for `\spnM` and `\spnD`. These functions are documented on page 14.)

11.26 The command `\spang`

The command `\spang` is the main public command for typesetting and tagging sexagesimal angle expressions (degrees, minutes, and seconds). It parses the input using the Lua function `\__spinttent_luafun_spang_parse:n`. This separates the degrees, minutes, and seconds to construct a *MathML intent* tree with the correct postfixes for screen readers.

These internal string variables store the values returned from the Lua module `spinttent.lua`. They hold the extracted numerical values for degrees, minutes, and seconds, as well as the error status used during formatting.

```

2147 \str_new:N \l__spinttent_spang_luaset_grado_str

```

```

2148 \str_new:N \l__spintent_spang_luaset_minuto_str
2149 \str_new:N \l__spintent_spang_luaset_segundo_str
2150 \str_new:N \l__spintent_spang_luaset_error_str

```

(End of definition for `\l__spintent_spang_luaset_grado_str` and others.)

11.26.1 Definition of error messages for `\spang`

`invalid-angle-format`

The error message `invalid-angle-format` defines the text displayed when an invalid angle input is detected. It is triggered when an argument fails the format checks, does not match the expected sexagesimal pattern, or contains invalid measurement symbols.

```

2151 \msg_new:nnnn { spintent } { invalid-angle-format }
2152 { Invalid~angle~format~in~#1. }
2153 {
2154   The~argument~'#2'~could~not~be~parsed. \\\
2155 }

```

(End of definition for `invalid-angle-format`.)

11.26.2 Internal functions for `\spang`

`\__spintent_spang_process:n`

The internal function `\__spintent_spang_process:n` parses the angle input using `\__spintent_luafun_spang_parse:n` after resetting the error flag in `\l__spintent_spang_luaset_error_str`. If an invalid format is detected, it issues the `invalid-angle-format` error. Otherwise, it checks the extracted string variables `\l__spintent_spang_luaset_grado_str`, `\l__spintent_spang_luaset_minuto_str`, and `\l__spintent_spang_luaset_segundo_str`. For each non-empty variable, it constructs the corresponding **MathML** `intent` structure with the correct measurement symbols and postfixes for screen readers.

```

2156 \cs_new_protected:Nn \__spintent_spang_process:n
2157 {
2158   \str_set:Nn \l__spintent_spang_luaset_error_str { false }
2159   \__spintent_luafun_spang_parse:n { #1 }
2160   \str_if_eq:VnTF \l__spintent_spang_luaset_error_str { true }
2161   {
2162     \msg_error:nnnn { spintent } { invalid-angle-format } { \spang } { #1 }
2163   }
2164   {
2165     \str_if_empty:NF \l__spintent_spang_luaset_grado_str
2166     {
2167       \spunit{ \l__spintent_spang_luaset_grado_str °}
2168     }
2169     \str_if_empty:NF \l__spintent_spang_luaset_minuto_str
2170     {
2171       \MathMLintent { minutos : postfix($m) }
2172       {
2173         \MathMLLarg { m }
2174         {
2175           \spunit{ \l__spintent_spang_luaset_minuto_str ' }
2176         }
2177       }
2178     }
2179     \str_if_empty:NF \l__spintent_spang_luaset_segundo_str
2180     {
2181       \MathMLintent { segundos : postfix($s) }
2182       {
2183         \MathMLLarg { s }
2184         {
2185           \spunit { \l__spintent_spang_luaset_segundo_str " }
2186         }
2187       }
2188     }
2189   }
2190 }

```

(End of definition for `\__spintent_spang_process:n`.)

`\spang`

The command `\spang` is the public interface for formatting sexagesimal angle expressions. It first checks if it is being used in math mode, issuing an error if not. Inside a local group, it calls `\__spintent_spang_process:n` to process the `{⟨argument⟩}`.

```

2191 \NewDocumentCommand \spang { m }
2192 {
2193   \mode_if_math:TF
2194   {
2195     \group_begin:

```

```

2196     \__spintenc_spang_process:n { #1 }
2197   \group_end:
2198 }
2199 { \msg_error:nnn { spintenc } { need-math-mode } { \spang } }
2200 }

```

(End of definition for `\spang`. This function is documented on page 16.)

11.27 The command `\spnZ`

The user-level command `\spnZ` formats integers (the set $\mathbb{Z}$). It includes specific options to automatically enclose numbers (particularly negative ones) in visual parentheses and allows you to control whether these parentheses are read by `MathCAT`.

This command verifies that the $\langle \textit{argument} \rangle$ is indeed an *integer*, checking for the complete absence of decimal separators or attached units of measurement.

11.27.1 Definition of keys for `\spnZ`

Now we define the $\langle \textit{keys} \rangle$ `cifras-sep` and `read-sign`. These are `.meta:nn` $\langle \textit{keys} \rangle$ passed directly to the `\spnum` command. We also introduce `wrap-parens`, which determines whether the number is enclosed in parentheses, and `read-parens`, which controls if these parentheses are read by `MathCAT`.

```

cifras-sep
read-sign
wrap-parens
read-parens
2201 \keys_define:nn { spintenc / spnZ }
2202 {
2203   cifras-sep .meta:nn = { spintenc / spnum } { cifras-sep = {#1} },
2204   read-sign .meta:nn = { spintenc / spnum } { read-sign = {#1} },
2205   wrap-parens .bool_set:N = \__spintenc_spnz_wrap_parens_bool,
2206   wrap-parens .initial:n = false,
2207   wrap-parens .value_required:n = true,
2208   read-parens .bool_set:N = \__spintenc_spnz_read_parens_bool,
2209   read-parens .initial:n = true,
2210   read-parens .value_required:n = true,
2211   unknown .code:n =
2212     {
2213       \str_set:Nn \__spintenc_command_name_str { spnZ }
2214       \__spintenc_unknown_keys_cmd:n {#1}
2215     },
2216 }

```

(End of definition for `cifras-sep` and others.)

11.27.2 Definition of error messages

`spnz-not-integer` Raised when the $\langle \textit{argument} \rangle$ contains decimal separators or attached units, which conflict with the constraints of a pure integer representation.

```

2217 \msg_new:nnnn { spintenc } { spnz-not-integer }
2218 { The~value~'#1'~is~not~a~valid~integer. }
2219 {
2220   Command~\token_to_str:N \spnZ~only~accepts~integers.\
2221 }

```

(End of definition for `spnz-not-integer`.)

11.27.3 Internal functions for `\spnZ`

`\__spintenc_spnz_render_parens:` The function `\__spintenc_spnz_render_parens:` first checks the state of the boolean variable `\__spintenc_spnz_read_parens_bool` (controlled by the `read-parens` key). If it is `true`, it prints the number surrounded by parentheses. Otherwise, it applies the `:silent` function intent to the parentheses while printing the number.

```

2222 \cs_new_protected:Nn \__spintenc_spnz_render_parens:
2223 {
2224   \bool_if:NTF \__spintenc_spnz_read_parens_bool
2225     {
2226       ( \__spintenc_spnz_print_num_start: )
2227     }
2228     {
2229       \MathMLintenc { :silent } { ( }
2230       \__spintenc_spnz_print_num_start:
2231       \MathMLintenc { :silent } { ) }
2232     }
2233 }

```

(End of definition for `\__spintenc_spnz_render_parens:`.)

`\_spintent_spnz_process:nn`

The function `\_spintent_spnz_process:nn` first processes the $\langle keys \rangle$ passed in ‘#1’ and then calls `\_spintent_luafun_clean_split_and_set:n` on ‘#2’. It verifies that the input is a valid integer by checking if the variable `\_spintent_luaset_has_integer_str` is `true`. If not `true`, an “error” is raised.

If it is `true`, it checks whether `\_spintent_luaset_part_int_tl` is *empty* and raises an “error” if so. Otherwise, it checks `\_spintent_spnz_wrap_parens_bool` set by `wrap-parens` and calls either the function `\_spintent_spnz_render_parens:` or `\_spintent_spnnum_print_num_start:.`

```

2234 \cs_new_protected:Nn \_spintent_spnz_process:nn
2235 {
2236   \keys_set:nn { spintent / spnZ } {#1}
2237   \_spintent_luafun_clean_split_and_set:n { \exp_not:n {#2} }
2238   \str_if_eq:VnTF \_spintent_luaset_has_integer_str { true }
2239   {
2240     \tl_if_empty:NTF \_spintent_luaset_part_int_tl
2241     { \msg_error:nnn { spintent } { spnum-no-digits } {#2} }
2242     {
2243       \bool_if:NTF \_spintent_spnz_wrap_parens_bool
2244       { \_spintent_spnz_render_parens: }
2245       { \_spintent_spnnum_print_num_start: }
2246     }
2247   }
2248   { \msg_error:nnn { spintent } { spnz-not-integer } {#2} }
2249 }

```

(End of definition for `\_spintent_spnz_process:nn`.)

\spnZ Defines the user command `\spnZ`, which is executed within a *group* and strictly enforces *math mode*.

```

2250 \NewDocumentCommand \spnZ { 0{} m }
2251 {
2252   \mode_if_math:TF
2253   {
2254     \group_begin:
2255     \_spintent_spnz_process:nn { #1 } { #2 }
2256     \group_end:
2257   }
2258   { \msg_error:nnn { spintent } { need-math-mode } { \spnZ } }
2259 }

```

(End of definition for `\spnZ`. This function is documented on page 16.)

11.28 The command `\spmQ`

The command `\spmQ` typesets mixed numbers within Spanish mathematical contexts (e.g., $3\frac{1}{2}$). It accepts an optional key-value list and three mandatory arguments: the integer part, the numerator, and the denominator.

11.28.1 Definition of variables for `\spmQ`

`\_spintent_spmQ_join_word_str`
`\_spintent_spmQ_math_style_tl`
`\_spintent_spnQ_intent_read_sign_str`
`\_spintent_spnQ_wrap_parens_l_tl`
`\_spintent_spnQ_wrap_parens_r_tl`
`\_spintent_saved_luamml_flag_int.`

The variable `\_spintent_spmQ_join_word_str` stores the value set by the key `join-word` which will be verbalized by `MathCAT` as a grammatical connector between the integer and fractional parts.

The variable `\_spintent_spmQ_math_style_tl` stores the current *mathematical style* used by the functions `\_spintent_spmQ_wrap_math:n` and `\_spintent_spmQ_capture_math_style:`

The variable `\_spintent_spnQ_intent_read_sign_str` stores the `:intent` passed to `\MathMLintent` when the signs ‘+’ or ‘−’ are present in integer part used by the function `\_spintent_spmQ_match_sign:.`

The variable `\_spintent_spnQ_wrap_parens_l_tl` and `\_spintent_spnQ_wrap_parens_r_tl` stores the size parens set by the function `\_spintent_spmQ_set_wrap_parens_size:` used by the key `wrap-parens`.

```

2260 \str_new:N \_spintent_spmQ_join_word_str
2261 \tl_new:N \_spintent_spmQ_math_style_tl
2262 \str_new:N \_spintent_spnQ_intent_read_sign_str
2263 \tl_new:N \_spintent_spnQ_wrap_parens_l_tl
2264 \tl_new:N \_spintent_spnQ_wrap_parens_r_tl
2265 \int_new:N \_spintent_saved_luamml_flag_int

```

(End of definition for `\_spintent_spmQ_join_word_str` and others.)

11.28.2 Definition of error messages for \spmQ

spmQ-zero-denominator
spmQ-not-integer

The error message spmQ-zero-denominator is raised when a zero denominator is passed to \spmQ. The second error is returned when the integer part is not actually an integer.

```

2266 \msg_new:nnnn { spintrent } { spmQ-zero-denominator }
2267 { Zero-denominator~in~\token_to_str:N \spmQ. }
2268 {
2269   The~denominator~of~a~fraction~cannot~be~zero.\\
2270 }
2271 \msg_new:nnnn { spintrent } { spmQ-not-integer }
2272 { The~value~'#1'~is~not~a~valid~integer. }
2273 {
2274   Command~\token_to_str:N \spnQ \c_space_tl only~accepts~integers.\\
2275 }
```

(End of definition for spmQ-zero-denominator and spmQ-not-integer.)

11.28.3 Definition of keys for \spmQ

join-word
use-spnZ
wrap-parens
read-parens
print-dfrac
unknown

Now we add the keys join-word, use-spnZ, wrap-parens, read-parens and print-dfrac.

```

2276 \keys_define:nn { spintrent / spmQ }
2277 {
2278   join-word      .choices:nn =
2279   { entero , enteros , y }
2280   {
2281     \int_case:nn { \l_keys_choice_int }
2282     {
2283       {1} { \str_set:Nn \l__spintrent_spmQ_join_word_str { entero } }
2284       {2} { \str_set:Nn \l__spintrent_spmQ_join_word_str { enteros } }
2285       {3} { \str_set:Nn \l__spintrent_spmQ_join_word_str { y } }
2286     }
2287   },
2288   join-word      .initial:n = y,
2289   join-word      .value_required:n = true,
2290   join-word/unknown .code:n =
2291   \msg_error:nneee { spintrent } { unknown-choice }
2292   { join-word } { entero, ~enteros, ~y } { \exp_not:n {#1} },
2293   use-spnZ       .bool_set:N = \l__spintrent_spmQ_use_spnZ_bool,
2294   use-spnZ       .initial:n = true,
2295   use-spnZ       .value_required:n = true,
2296   print-dfrac    .bool_set:N = \l__spintrent_spmQ_print_dfrac_bool,
2297   print-dfrac    .initial:n = false,
2298   print-dfrac    .value_required:n = true,
2299   wrap-parens    .bool_set:N = \l__spintrent_spmQ_wrap_parens_bool,
2300   wrap-parens    .initial:n = false,
2301   wrap-parens    .value_required:n = true,
2302   read-parens    .bool_set:N = \l__spintrent_spmQ_read_parens_bool,
2303   read-parens    .initial:n = true,
2304   read-parens    .value_required:n = true,
2305   unknown        .code:n =
2306   {
2307     \str_set:Nn \l__spintrent_command_name_str { spmQ }
2308     \__spintrent_unknown_keys_cmd:n {#1}
2309   },
2310 }
```

(End of definition for join-word and others.)

11.28.4 Internal functions for \spmQ

__spintrent_spmQ_set_wrap_parens_size:

The function __spintrent_spmQ_set_wrap_parens_size: capture the current *mathematical style* using the primitive \mathstyle from LuaTeX and sets the variables \l__spintrent_spmQ_wrap_parens_l_tl and \l__spintrent_spmQ_wrap_parens_r_tl with the appropriate size for the parentheses.

```

2311 \cs_new_protected:Nn \__spintrent_spmQ_set_wrap_parens_size:
2312 {
2313   \int_case:nnF { \mathstyle }
2314   {
2315     { 0 }
2316     {
2317       \tl_set:Nn \l__spintrent_spmQ_wrap_parens_l_tl { \Bigl \lparen }
2318       \tl_set:Nn \l__spintrent_spmQ_wrap_parens_r_tl { \Bigr \rparen }
2319     }
2320     { 1 }
2321     {
```

```

2322         \tl_set:Nn \l__spintent_spnQ_wrap_parens_l_tl { \Bigl \lparen }
2323         \tl_set:Nn \l__spintent_spnQ_wrap_parens_r_tl { \Bigr \rparen }
2324     }
2325     { 2 }
2326     {
2327         \tl_set:Nn \l__spintent_spnQ_wrap_parens_l_tl { \bigl \lparen }
2328         \tl_set:Nn \l__spintent_spnQ_wrap_parens_r_tl { \bigr \rparen }
2329     }
2330     { 3 }
2331     {
2332         \tl_set:Nn \l__spintent_spnQ_wrap_parens_l_tl { \bigl \lparen }
2333         \tl_set:Nn \l__spintent_spnQ_wrap_parens_r_tl { \bigr \rparen }
2334     }
2335 }
2336 {
2337     \tl_set:Nn \l__spintent_spnQ_wrap_parens_l_tl { \lparen }
2338     \tl_set:Nn \l__spintent_spnQ_wrap_parens_r_tl { \rparen }
2339 }

```

If the key `print-dfrac` is active, we must reset the variables `\l__spintent_spnQ_wrap_parens_l_tl` and `\l__spintent_spnQ_wrap_parens_r_tl`.

```

2340     \bool_if:NT \l__spintent_spmQ_print_dfrac_bool
2341     {
2342         \tl_set:Nn \l__spintent_spnQ_wrap_parens_l_tl { \Bigl \lparen }
2343         \tl_set:Nn \l__spintent_spnQ_wrap_parens_r_tl { \Bigr \rparen }
2344     }
2345 }

```

(End of definition for `\__spintent_spmQ_set_wrap_parens_size:`)

```

\__spintent_spmQ_start_wrap_parens:
\__spintent_spmQ_stop_wrap_parens:

```

The functions `\__spintent_spmQ_start_wrap_parens:` and `\__spintent_spmQ_stop_wrap_parens:` print with the appropriate size for the parentheses set by the key `wrap-parens`. Only if the variable `\l__spintent_spnQ_read_parens_bool` handled by the key `read-parens` is false, we will execute `\MathMLintent{:silent}`.

```

2346 \cs_new_protected:Nn \__spintent_spmQ_start_wrap_parens:
2347 {
2348     \bool_if:NT \l__spintent_spnQ_wrap_parens_bool
2349     {
2350         \__spintent_spmQ_set_wrap_parens_size:
2351         \bool_if:NTF \l__spintent_spnQ_read_parens_bool
2352         {
2353             \l__spintent_spnQ_wrap_parens_l_tl
2354         }
2355         {
2356             \MathMLintent { :silent } { \l__spintent_spnQ_wrap_parens_l_tl }
2357         }
2358     }
2359 }
2360 \cs_new_protected:Nn \__spintent_spmQ_stop_wrap_parens:
2361 {
2362     \bool_if:NT \l__spintent_spnQ_wrap_parens_bool
2363     {
2364         \__spintent_spmQ_set_wrap_parens_size:
2365         \bool_if:NTF \l__spintent_spnQ_read_parens_bool
2366         {
2367             \l__spintent_spnQ_wrap_parens_r_tl
2368         }
2369         {
2370             \MathMLintent { :silent } { \l__spintent_spnQ_wrap_parens_r_tl }
2371         }
2372     }
2373 }

```

(End of definition for `\__spintent_spmQ_start_wrap_parens:` and `\__spintent_spmQ_stop_wrap_parens:`)

```
\__spintent_spmQ_join_word:
```

The function `\__spintent_spmQ_join_word:` captures the ‘+’ or ‘−’ for integer part sign and sets the `:intent` to `\MathMLintent {⟨argument⟩}` if present.

```

2374 \cs_new_protected:Nn \__spintent_spmQ_set_join_word:
2375 {
2376     \str_case:Vn \l__spintent_spmQ_join_word_str
2377     {
2378         { entero }

```



```

2379         {
2380             \MathMLintent { _entero } { \c__spintent_invisible_sep_tl }
2381         }
2382     { enteros }
2383     {
2384         \MathMLintent { _enteros } { \c__spintent_invisible_sep_tl }
2385     }
2386     { y }
2387     {
2388         \MathMLintent { _y } { \c__spintent_invisible_sep_tl }
2389     }
2390 }
2391 }

```

(End of definition for \c__spintent_spmQ_join_word:.)

\c__spintent_spmQ_print_spmZ_dfrac:nn

The function `\c__spintent_spmQ_print_spmZ_dfrac:nn` checks the state of the variable `\l__spintent_spmQ_use_spmZ_bool` handled by the key `use-spmZ` along with the variable `\l__spintent_spmQ_print_dfrac_bool` handled by the key `print-dfrac`. According to their values, it prints `\frac` or `\dfrac` using `\spnZ` when necessary.

```

2392 \cs_new_protected:Nn \c__spintent_spmQ_print_spmZ_dfrac:nn
2393 {
2394     \__spintent_luafun_clean_split_and_set:n { \exp_not:n {#1} }
2395     \str_if_eq:VnT \l__spintent_luaset_has_integer_str { false }
2396     {
2397         \msg_error:nn { spintent } { spmQ-not-integer }
2398     }
2399     \bool_if:NTF \l__spintent_spmQ_use_spmZ_bool
2400     {
2401         \use:c { \bool_if:NT \l__spintent_spmQ_print_dfrac_bool { d } frac }
2402         { \spnZ { #1 } } { \spnZ { #2 } }
2403     }
2404     {
2405         \use:c { \bool_if:NT \l__spintent_spmQ_print_dfrac_bool { d } frac }
2406         { #1 } { #2 }
2407     }
2408 }

```

(End of definition for \c__spintent_spmQ_print_spmZ_dfrac:nn.)

\c__spintent_spmQ_print_scaled_int:

The function `\c__spintent_spmQ_print_scaled_int:` will print the integer part scaled and aligned correctly.

```

2409 \cs_new_protected:Nn \c__spintent_spmQ_print_scaled_int:
2410 {
2411     % Deshabilitar luamml mientras se inyecta el noad del entero.
2412     % Esto evita el <math> fantasma del sub_box procesado en solitario
2413     % antes de que \frac se expanda.
2414     % Flag 0 = skip completamente.
2415     %%\int_set:Nn \l__luamml_flag_int { 0 }
2416     \int_set_eq:Nc \l__spintent_saved_luamml_flag_int { \l__luamml_flag_int }
2417     \int_set:Nn \l__luamml_flag_int { 0 }
2418     \bool_if:NTF \l__spintent_spmQ_print_dfrac_bool
2419     {
2420         \__spintent_luafun_spmQ_scale_int:Vn \l__spintent_luaset_part_int_tl {dfrac}
2421     }
2422     {
2423         \__spintent_luafun_spmQ_scale_int:Vn \l__spintent_luaset_part_int_tl {frac}
2424     }
2425     % Restaurar flag normal (11 = mathml-SE: bits 0,1,3)
2426     % \int_set:Nn \l__luamml_flag_int { 11 }
2427     \int_set_eq:cN { \l__luamml_flag_int } \l__spintent_saved_luamml_flag_int
2428 }

```

(End of definition for \c__spintent_spmQ_print_scaled_int:.)

\c__spintent_spmQ_print_sacle_int:n

The function `\c__spintent_spmQ_print_int:n` takes as its arguments the integer part passed in ‘#1’. First, the function `\__spintent_luafun_clean_split_and_set:n` executes on ‘#1’ and checks the status of the variable `\l__spintent_luaset_has_integer_str`. If it is `true`, checks if the variable `\l__spintent_luaset_sign_tl` is `empty` and calls the function `\c__spintent_spmQ_print_scaled_int:` to print the scaled integer part; otherwise, captures the ‘+’ or ‘-’ for integer part sign, sets the `:intent` to `\MathMLintent` and print.

```

2429 \cs_new_protected:Nn \c__spintent_spmQ_print_sacle_int:n

```

```

2430 {
2431   \__spintent_luafun_clean_split_and_set:n { \exp_not:n {#1} }
2432   \str_if_eq:VnTF \__spintent_luaset_has_integer_str { true }
2433   {
2434     \tl_if_empty:NTF \__spintent_luaset_sign_tl
2435     {
2436       \__spintent_spmQ_print_scale_int:
2437     }
2438     {
2439       \str_if_eq:VnTF \__spintent_luaset_sign_tl { + }
2440       {
2441         \str_set:Nn
2442         \__spintent_spmQ_intent_read_sign_str { _más:prefix $(e) }
2443       }
2444       {
2445         \str_set:Nn
2446         \__spintent_spmQ_intent_read_sign_str { _menos:prefix $(e) }
2447       }
2448       \MathMLintent { \__spintent_spmQ_intent_read_sign_str }
2449       {
2450         \__spintent_luaset_sign_tl \__spintent_invisible_sep: % need
2451         \MathMLarg { e } { \__spintent_spmQ_print_scale_int: }
2452       }
2453     }
2454   }
2455   { \msg_error:nn { spintent } { spmQ-not-integer } }
2456 }

```

(End of definition for __spintent_spmQ_print_sacle_int:n)

__spintent_spmQ_print:nnn

The function `\__spintent_spmQ_render:nnn` takes three arguments: the integer part ‘#1’, the numerator ‘#2’, and the denominator ‘#3’. First, executes the function `\__spintent_spmQ_tart_wrap_parens:` handled by the key `wrap-parens` to open the parentheses if they are active, then we call the function `\__spintent_spmQ_print_sacle_int:n` which will scale the integer part passed in ‘#1’ and print it. Then, we inject the “word” that *joins* the integer part with the fraction handled by the key `join-word`.

Finally, we call the function `\__spintent_spmQ_print_spmZ_dfrac:nn` which is responsible for printing the fraction using the keys `print-dfrac` and `use-spmZ`, then executes the function `\__spintent_spmQ-stop_wrap_parens:` handled by the key `wrap-parens` to close the parentheses if they are active.

```

2457 \cs_new_protected:Nn \__spintent_spmQ_print:nnn
2458 {
2459   \__spintent_spmQ_start_wrap_parens:
2460   \__spintent_spmQ_print_sacle_int:n { #1 }
2461   \__spintent_spmQ_set_join_word:
2462   \__spintent_spmQ_print_spmZ_dfrac:nn { #2 } { #3 }
2463   \__spintent_spmQ_stop_wrap_parens:
2464 }

```

(End of definition for __spintent_spmQ_print:nnn.)

__spintent_spmQ_exec:nnnn

The internal function `\__spintent_spmQ_exec:nnnn` takes four arguments: the optional *(keys)* ‘#1’, the integer ‘#2’, the numerator ‘#3’, and the denominator ‘#4’. First sets the optional *(keys)* from ‘#1’, then, processes the denominator input in ‘#4’ using `\__spintent_luafun_clean_split_and_set:n`.

Now checks the status of `\__spintent_luaset_has_integer_str`. If the status of this variable is `true` and the integer part is exactly zero, it issues the `spmQ-zero-denominator` “error”. Otherwise, if the status is `false` or the integer part is *non-zero*, it calls the function `\__spintent_spmQ_print:nnn` to format the “mixed number”.

```

2465 \cs_new_protected:Nn \__spintent_spmQ_exec:nnnn
2466 {
2467   \keys_set:nn { spintent / spmQ } { #1 }
2468   \__spintent_luafun_clean_split_and_set:n { \exp_not:n { #4 } }
2469   \str_if_eq:VnTF \__spintent_luaset_has_integer_str { true }
2470   {
2471     \int_compare:nNnTF { \__spintent_luaset_part_int_tl } = { 0 }
2472     { \msg_error:nn { spintent } { spmQ-zero-denominator } }
2473     { \__spintent_spmQ_print:nnn { #2 } { #3 } { #4 } }
2474   }
2475   { \__spintent_spmQ_print:nnn { #2 } { #3 } { #4 } }
2476 }

```

(End of definition for __spintent_spmQ_exec:nnnn.)

`\spmQ` The command `\spmQ` is the public interface for formatting mixed numbers. It first checks if it is being used in math mode, issuing an error if not. Inside a local group, it calls `\__spintent_spmQ_exec:nnnn` to process the optional `<keys>` and the `{<arguments>}`.

```

2477 \NewDocumentCommand \spmQ { O{} m m m }
2478 {
2479   \mode_if_math:TF
2480   {
2481     \group_begin:
2482       \__spintent_spmQ_exec:nnnn { #1 } { #2 } { #3 } { #4 }
2483     \group_end:
2484   }
2485   { \msg_error:nnn { spintent } { need-math-mode } { \spmQ } }
2486 }

```

(End of definition for `\spmQ`. This function is documented on page 16.)

11.29 Finish package

Finish package implementation.

```

2487 \file_input_stop:
2488 </package>

```

12 Índice de la Implementación

The italic numbers denote the pages where the corresponding entry is described, the numbers underlined and all others indicate the line on which they are implemented in the package code.

C	
cifras-sep	<u>220</u> , <u>505</u> , <u>778</u> , <u>1718</u> , <u>1789</u> , <u>2201</u>
cmd-key-unknown	<u>140</u>
cmd-key-value-unknown	<u>140</u>
Commands provide by spintent :	
\setspintent	23
\spmoney	37
\spnum	26
\spsiglo	46, 48
\spunit	31
currency	<u>778</u>
currency-no-subunits	<u>747</u>
D	
dolar-math	<u>778</u>
I	
inline-numeric-denominator	<u>483</u>
invalid-angle-format	<u>2151</u>
invalid-century-format	<u>1174</u>
invalid-currency	<u>747</u>
invalid-date-format	<u>1404</u>
invalid-time-format	<u>1325</u>
J	
join-word	<u>2276</u>
K	
Keys for \spdiv provide by spintent :	
cifras-sep	57
notation-sym	57
read-parens	57
read-period-sep	57
read-sign	57
read-sym	57
set-sym	57
wrap-parens	57
Keys for \spdsym provide by spintent :	
prefix	55
read-sym	55
set-sym	55
suffix	55
Keys for \spmcd provide by spintent :	
prefix	60, 63
read-cmd	60
result-num	63
result	60
sample-arg	60
silent-cmd	60
upper-case	60
Keys for \spmcm provide by spintent :	
prefix	60, 63
read-cmd	60
result-num	63
result	60
sample-arg	60
silent-cmd	60
upper-case	60
Keys for \spmoney provide by spintent :	
cifras-sep	38
currency	38
dolar-math	38
sub-units	38
Keys for \spmQ provide by spintent :	
join-word	68, 69
print-dfrac	69
read-parens	69
use-spnZ	69
wrap-parens	69
Keys for \spmsym provide by spintent :	
prefix	56
read-sym	56
set-sym	56
suffix	56
Keys for \spmul provide by spintent :	
cifras-sep	58
notation-sym	58
read-parens	58
read-period-sep	58
read-sign	58
read-sym	58
set-sym	58
wrap-parens	58
Keys for \spnD provide by spintent :	
result	63
silent-cmd	63
Keys for \spnM provide by spintent :	
result	63
silent-cmd	63
Keys for \spnum provide by spintent :	
cifras-sep	25, 27, 28
notation-sym	26, 27
read-period-sep	26–28
read-sign	26, 27
Keys for \spnZ provide by spintent :	
cifras-sep	67
read-parens	67
read-sign	67
wrap-parens	67, 68
Keys for \spshort provide by spintent :	
read-arg	43
small-caps	43
Keys for \spsiglo provide by spintent :	
print-era	46, 48
Keys for \spunit provide by spintent :	
cifras-sep	32
notation-sym	32
print-cdot	32
read-cdot	32
read-period-sep	32
read-slash	32
Keys for \spusep provide by spintent :	
silent	52
size	52
M	
mc-invalid-arguments	<u>1871</u>
md-invalid-argument	<u>2043</u>

Lua module provide by **spintent**:

`spintent.lua` [21](#), [22](#), [25](#), [26](#), [31](#), [35](#), [37](#), [38](#), [43](#), [44](#), [46-49](#),
[51](#), [60](#), [62](#), [65](#)

N

notation-sym [220](#), [505](#), [1718](#), [1789](#)

P

Packages:

`expl3` [21](#), [25](#), [26](#)
`lua-unicode-math` [21](#)
`luamml` [22](#)
`tagpdf` [43](#), [47](#)
`unicode-math` [21](#), [53](#)
prefix [1583](#), [1653](#), [1878](#), [2017](#)
print-dfrac [2276](#)
print-era [1182](#)

R

`read-arg` [1052](#)
`read-cdot` [505](#)
`read-cmd` [1878](#)
`read-parens` [1718](#), [1789](#), [2201](#), [2276](#)
`read-period-sep` [220](#), [505](#), [1718](#), [1789](#)
`read-sign` [220](#), [1718](#), [1789](#), [2201](#)
`read-slash` [505](#)
`read-sym` [1583](#), [1653](#), [1718](#), [1789](#)
`result` [1878](#), [2017](#)
`result-num` [2017](#)

S

`sample-arg` [1878](#), [2017](#)
seq commands:
 `\seq_set_from_clist:Nn` [41](#)
`set-sym` [1583](#), [1653](#), [1718](#), [1789](#)
`\setspintent` [3](#), [23](#), [128](#)
`silent` [1448](#)
`silent-cmd` [1878](#), [2017](#)
`size` [1448](#)
`small-caps` [1052](#)
`\spang` [16](#), [65](#), [2191](#)
`\spdate` [11](#), [50](#), [1428](#)
`\spdiv` [13](#), [57](#), [1779](#)
`spdiv-empty-argument` [1738](#)
`\spdsym` [12](#), [54](#), [1630](#)
spintent internal commands:
 `\__spintent_collapse_hyphens:N` [65](#)
 `\l__spintent_command_name_str` [140](#)
 `\__spintent_invisible_sep:` [51](#)
 `\c__spintent_invisible_sep_tl` [51](#)
 `\__spintent_luafun_clean_split_and_set:n` [46](#),
 [177](#)
 `\__spintent_luafun_spang_parse:n` [66](#)
 `\__spintent_luafun_spdate_parse:n` [51](#)
 `\__spintent_luafun_spmoney_lookup_data:n` [46](#)
 `\__spintent_luafun_spmQ_scale_int:nn` [46](#)
 `\__spintent_luafun_sptime_parse:n` [50](#)
 `\__spintent_luafun_spunit_lookup_alias:n` [46](#)
 `\l__spintent_luaset_arg_above_tl` [196](#)
 `\l__spintent_luaset_arg_below_tl` [196](#)
 `\l__spintent_luaset_dec_is_all_zeros_-`
 `str` [189](#)

`\l__spintent_luaset_dec_sep_tl` [177](#)
`\l__spintent_luaset_decimal_period_str` [189](#)
`\l__spintent_luaset_denom_has_number_-`
 `str` [196](#)
`\l__spintent_luaset_exponent_tl` [189](#)
`\l__spintent_luaset_format_part_dec_str` [184](#)
`\l__spintent_luaset_format_part_int_str` [184](#)
`\l__spintent_luaset_has_exponent_str` [189](#)
`\l__spintent_luaset_has_integer_str` [72](#), [177](#)
`\l__spintent_luaset_has_natural_str` [177](#)
`\l__spintent_luaset_has_units_str` [196](#)
`\l__spintent_luaset_millions_str` [189](#)
`\l__spintent_luaset_not_decimal_not_-`
 `period_str` [189](#)
`\l__spintent_luaset_not_decimal_period_-`
 `str` [189](#)
`\l__spintent_luaset_only_part_dec_str` [184](#)
`\l__spintent_luaset_only_part_int_str` [184](#)
`\l__spintent_luaset_only_part_period_-`
 `str` [184](#)
`\l__spintent_luaset_part_dec_tl` [177](#)
`\l__spintent_luaset_part_int_tl` [177](#)
`\l__spintent_luaset_part_period_tl` [177](#)
`\l__spintent_luaset_sign_tl` [177](#)
`\l__spintent_luaset_status_str` [196](#)
`\l__spintent_mc_args_seq` [1860](#)
`\__spintent_mc_process_args:nn` [1919](#)
`\l__spintent_md_out_seq` [2007](#)
`\__spintent_print_spunit_hierarchy:` [606](#)
`\__spintent_sanitize_affix:Nn` [73](#)
`\l__spintent_saved_luamml_flag_int.` [2260](#)
`\l__spintent_setspintent_input_arg_cmd_-`
 `tl` [91](#)
`\__spintent_setspintent_parse_args:nn` [111](#)
`\__spintent_setspintent_set_keys:nn` [100](#)
`\l__spintent_spang_luaset_error_str` [2147](#)
`\l__spintent_spang_luaset_grado_str` [2147](#)
`\l__spintent_spang_luaset_minuto_str` [2147](#)
`\l__spintent_spang_luaset_segundo_str` [2147](#)
`\__spintent_spang_process:n` [2156](#)
`\l__spintent_spdate_luaset_error_str` [1402](#)
`\l__spintent_spdate_luaset_output_str` [1402](#)
`\__spintent_spdate_process:n` [51](#), [1413](#)
`\__spintent_spdiv_process:nnn` [1764](#)
`\__spintent_spdiv_render_op:nn` [1743](#)
`\__spintent_spdiv_render_parens:nn` [1752](#)
`\l__spintent_spdsym_prefix_tl` [1579](#)
`\l__spintent_spdsym_read_sym_tl` [1579](#)
`\l__spintent_spdsym_suffix_tl` [1579](#)
`\l__spintent_spdsym_symbol_tl` [1579](#)
`\__spintent_spmc_process:nnnn` [1941](#)
`\l__spintent_spmcd_prefix_tl` [1860](#)
`\l__spintent_spmcd_print_result_bool` [1860](#)
`\l__spintent_spmcd_print_upper_bool` [1860](#)
`\l__spintent_spmcd_read_terse_bool` [1860](#)
`\l__spintent_spmcd_suffix_tl` [1860](#)
`\l__spintent_spmcm_prefix_tl` [1860](#)
`\l__spintent_spmcm_print_result_bool` [1860](#)
`\l__spintent_spmcm_print_upper_bool` [1860](#)

<code>\l__spintent_spmcm_read_terse_bool ...</code>	1860	<code>\l__spintent_spmul_render_op:nn ...</code>	1814
<code>\l__spintent_spmcm_spmcd_luaset_error_-</code>	1860	<code>\l__spintent_spmul_render_parens:nn ...</code>	1823
<code>str ...</code>	1860	<code>\l__spintent_spnD_luaset_is_truncated_-</code>	2007
<code>\l__spintent_spmcm_suffix_tl ...</code>	1860	<code>str ...</code>	2007
<code>__spintent_spmoney_build_tree: ...</code>	957	<code>\l__spintent_spnD_luaset_result_tl ...</code>	2007
<code>__spintent_spmoney_check_sub_units: ..</code>	910	<code>\l__spintent_spnD_prefix_tl ...</code>	2007
<code>\l__spintent_spmoney_extracted_curr_str</code>	739	<code>\l__spintent_spnD_print_result_bool ...</code>	2007
<code>\l__spintent_spmoney_extracted_num_tl .</code>	739	<code>\l__spintent_spnD_print_result_num_tl .</code>	2007
<code>__spintent_spmoney_intent_int: ...</code>	846	<code>\l__spintent_spnD_silent_cmd_bool ...</code>	2007
<code>\l__spintent_spmoney_intent_money_str .</code>	739	<code>\l__spintent_spnM_luaset_is_truncated_-</code>	2007
<code>\l__spintent_spmoney_intent_sign_str ..</code>	739	<code>str ...</code>	2007
<code>\l__spintent_spmoney_intent_sub_unit_-</code>	739	<code>\l__spintent_spnM_luaset_result_tl ...</code>	2007
<code>str ...</code>	739	<code>\l__spintent_spnM_prefix_tl ...</code>	2007
<code>\l__spintent_spmoney_luaset_conde_str .</code>	729	<code>\l__spintent_spnM_print_result_bool ...</code>	2007
<code>\l__spintent_spmoney_luaset_currency_arg_-</code>	729	<code>\l__spintent_spnM_print_result_num_tl .</code>	2007
<code>str ...</code>	729	<code>\l__spintent_spnM_silent_cmd_bool ...</code>	2007
<code>\l__spintent_spmoney_luaset_plur_str ..</code>	729	<code>__spintent_spnM_spnD_exec:nnnn ...</code>	2088
<code>\l__spintent_spmoney_luaset_position_-</code>	729	<code>__spintent_spnM_spnD_intent:nn ...</code>	2053
<code>str ...</code>	729	<code>__spintent_spnM_spnD_process:nnnn ...</code>	2048
<code>\l__spintent_spmoney_luaset_print_iso_-</code>	729	<code>__spintent_spnM_spnD_result:n ...</code>	2069
<code>str ...</code>	729	<code>__spintent_spnM_spnD_silent:nn ...</code>	2061
<code>\l__spintent_spmoney_luaset_sing_str ..</code>	729	<code>\l__spintent_spnQ_intent_read_sign_str</code>	2260
<code>\l__spintent_spmoney_luaset_status_str</code>	729	<code>\l__spintent_spnQ_read_parens_bool ...</code>	70
<code>\l__spintent_spmoney_luaset_subplur_str</code>	729	<code>\l__spintent_spnQ_wrap_parens_l_tl ..</code>	69, 70,
<code>\l__spintent_spmoney_luaset_subsing_str</code>	729	2260	
<code>\l__spintent_spmoney_luaset_symbol_str</code>	729	<code>\l__spintent_spnQ_wrap_parens_r_tl ..</code>	69, 70,
<code>__spintent_spmoney_normalize_key:n ...</code>	764	2260	
<code>__spintent_spmoney_parse_and_route:n .</code>	1005	<code>\l__spintent_spnnum__notation_sym_tl ...</code>	26
<code>__spintent_spmoney_print_dec_sub_unit:</code>	957	<code>\l__spintent_spnnum_intent_read_dec_sep_-</code>	27, 29, 215
<code>__spintent_spmoney_print_int: ...</code>	846	<code>str ...</code>	26,
<code>__spintent_spmoney_print_integer_-</code>	957	<code>\l__spintent_spnnum_intent_read_sign_str</code>	215
<code>continue: ...</code>	957	<code>\l__spintent_spnnum_notation_sym_tl ...</code>	215
<code>__spintent_spmoney_print_sep_con: ...</code>	957	<code>__spintent_spnnum_print_integer: ...</code>	382
<code>__spintent_spmoney_print_symbol: ...</code>	812	<code>__spintent_spnnum_print_num_start: ...</code>	382
<code>__spintent_spmoney_render_layout: ...</code>	825	<code>__spintent_spnnum_print_number_core: ..</code>	382
<code>__spintent_spmoney_render_number_node:</code>	837	<code>__spintent_spnnum_print_part_dec: ...</code>	361
<code>__spintent_spmoney_render_only_integer_-</code>	837	<code>__spintent_spnnum_print_part_period: ..</code>	361
<code>node: ...</code>	837	<code>__spintent_spnnum_print_period_bar: ...</code>	290
<code>\l__spintent_spmoney_res_main_voice_str</code>	739	<code>\l__spintent_spnnum_read_period_sep_str .</code>	26,
<code>__spintent_spmoney_set_and_print_sub_-</code>	910	215	
<code>units: ...</code>	910	<code>\l__spintent_spnnum_read_terse_bool .</code>	26, 215
<code>__spintent_spmoney_set_currency:n ...</code>	764	<code>__spintent_spnnum_render_dec: ...</code>	273
<code>\l__spintent_spmoney_sub_units_bool ...</code>	739	<code>__spintent_spnnum_render_decimal_sep: .</code>	327
<code>\l__spintent_spmoney_subnum_int ...</code>	739	<code>__spintent_spnnum_render_int: ...</code>	273
<code>__spintent_spmoney_validate_money: ...</code>	796	<code>__spintent_spnnum_render_only_period: .</code>	290
<code>__spintent_spmQ_exec:nnnn ...</code>	2465	<code>__spintent_spnnum_render_part:n ...</code>	273
<code>__spintent_spmQ_join_word: ...</code>	2374	<code>__spintent_spnnum_render_period_sep: ..</code>	290
<code>\l__spintent_spmQ_join_word_str ...</code>	2260	<code>__spintent_spnz_process:nn ...</code>	2234
<code>\l__spintent_spmQ_math_style_tl ...</code>	2260	<code>__spintent_spnz_render_parens: ...</code>	2222
<code>__spintent_spmQ_print:nnn ...</code>	2457	<code>__spintent_spread_exec:nn ...</code>	1560
<code>\l__spintent_spmQ_print_dfrac_bool ...</code>	71	<code>\l__spintent_spread_intent_tl ...</code>	1559
<code>__spintent_spmQ_print_sacle_int:n ...</code>	2429	<code>__spintent_spsshort_execute_layout: ...</code>	1113
<code>__spintent_spmQ_print_scaled_int: ...</code>	2409	<code>\l__spintent_spsshort_luaset_base_str ..</code>	1030
<code>__spintent_spmQ_print_spnZ_dfrac:nn ..</code>	2392		
<code>__spintent_spmQ_set_wrap_parens_size:</code>	2311		
<code>__spintent_spmQ_start_wrap_parens: ...</code>	2346		
<code>__spintent_spmQ_stop_wrap_parens: ..</code>	2346		
<code>__spintent_spmQ_tart_wrap_parens: ...</code>	72		
<code>\l__spintent_spmQ_use_spnZ_bool ...</code>	71		
<code>\l__spintent_spmul_prefix_tl ...</code>	1649		

<code>\l__spintenc_spshort_luaset_expanded_-str</code>	1030	<code>__spintenc_spusep_convert:n</code>	1507
<code>\l__spintenc_spshort_luaset_layout_str</code>	1030	<code>\l__spintenc_spusep_left_tl</code>	52, 53, 1445
<code>\l__spintenc_spshort_luaset_output_str</code>	1030	<code>\l__spintenc_spusep_right_tl</code>	52, 53, 1445
<code>\l__spintenc_spshort_luaset_status_str</code>	1030	<code>__spintenc_spusep_set_parens:nn</code>	1496
<code>\l__spintenc_spshort_luaset_suffix_str</code>	1030	<code>\l__spintenc_spusep_silent_bool</code>	53
<code>__spintenc_spshort_print:nn</code>	1066	<code>__spintenc_unknown_keys:n</code>	167
<code>__spintenc_spshort_process_and_-render:n</code>	1078	<code>__spintenc_unknown_keys:nn</code>	167
<code>__spintenc_spshort_render_bypass:n</code> ...	1089	<code>\spmc</code>	14, 60, 1987
<code>__spintenc_spshort_render_error:n</code> ...	1089	<code>\spmcn</code>	15, 60, 1987
<code>__spintenc_spshort_render_fallback:</code> ..	1089	<code>\spmoney</code>	6, 37, 1019
<code>__spintenc_spshort_render_success:</code> ...	1089	<code>\spmQ</code>	16, 68, 2477
<code>__spintenc_spshort_tag_span:nn</code>	1039	<code>spmQ-not-integer</code>	2266
<code>\l__spintenc_spsiglo_luaset_arabic_str</code>	1170	<code>spmQ-zero-denominator</code>	2266
<code>\l__spintenc_spsiglo_luaset_error_str</code> ..	1170	<code>\spmsym</code>	12, 56, 1699
<code>\l__spintenc_spsiglo_luaset_roman_min_-str</code>	1170	<code>\spmul</code>	13, 58, 1850
<code>\l__spintenc_spsiglo_print_era_str</code> ...	1170	<code>spmul-empty-argument</code>	1809
<code>__spintenc_spsiglo_print_math:n</code>	1254	<code>\spnD</code>	13, 62, 63, 2127
<code>__spintenc_spsiglo_print_text:n</code>	1254	<code>\spnewunit</code>	6, 35, 697
<code>__spintenc_spsiglo_render_simple:nn</code> ..	1205	<code>spnewunit-duplicate</code>	676
<code>__spintenc_spsiglo_render_with_-era:nnn</code>	1205	<code>\spnM</code>	14, 62, 63, 2127
<code>__spintenc_spsiglo_tag_span:nn</code>	1241	<code>\spnum</code>	3, 26, 27, 451
<code>__spintenc_sptime_build_seconds:nn</code> ...	1334	<code>spnum-has-units</code>	437
<code>__spintenc_sptime_build_tree:</code>	50, 1352	<code>spnum-no-digits</code>	437
<code>\l__spintenc_sptime_luaset_base_str</code> ...	1320	<code>\spnZ</code>	16, 67, 2250
<code>\l__spintenc_sptime_luaset_error_str</code> ..	1320	<code>spnz-not-integer</code>	2217
<code>\l__spintenc_sptime_luaset_has_seconds_-str</code>	1320	<code>\spread</code>	11, 54, 1569
<code>\l__spintenc_sptime_luaset_is_fraction_-str</code>	1320	<code>spread-empty-args</code>	1556
<code>\l__spintenc_sptime_luaset_seconds_str</code>	1320	<code>\spshort</code>	7, 43, 1157
<code>\l__spintenc_spunit_exp_seq</code>	476	<code>\spsiglo</code>	11, 46, 1307
<code>\l__spintenc_spunit_exponent_tl</code>	476	<code>\sptime</code>	11, 49, 1384
<code>__spintenc_spunit_format_canonical:</code> ..	539	<code>\spunit</code>	4, 31, 32, 35, 664
<code>\l__spintenc_spunit_is_mult_bool</code>	476	<code>spunit-invalid-unit</code>	483
<code>\l__spintenc_spunit_luaset_canonical_-str</code>	469	<code>spunit-no-units</code>	483
<code>\l__spintenc_spunit_luaset_compact_exp_-str</code>	469	<code>\spunitalias</code>	6, 35, 711
<code>\l__spintenc_spunit_luaset_is_compact_-str</code>	469	<code>spunitalias-duplicate</code>	676
<code>\l__spintenc_spunit_luaset_is_-sexagesimal_str</code>	469	<code>spunitalias-invalid-expr</code>	676
<code>\l__spintenc_spunit_luaset_lookup_status_-str</code>	469	<code>\spusep</code>	52, 1545
<code>\l__spintenc_spunit_luaset_read_str</code> ...	469	<code>spusep-empty-arg</code>	1438
<code>\l__spintenc_spunit_luaset_status_str</code> ..	469	<code>spusep-invalid-arg</code>	1438
<code>\l__spintenc_spunit_mult_seq</code>	476	<code>str commands:</code>	
<code>__spintenc_spunit_process_base_exp:n</code> ..	539	<code>\str_if_eq:nnTF</code>	41
<code>__spintenc_spunit_process_mult:n</code>	599	<code>\str_lowercase:n</code>	41
<code>\l__spintenc_spunit_read_cdot_bool</code> ...	476	<code>\str_uppercase:n</code>	41
<code>\l__spintenc_spunit_read_slash_str</code> ...	476	<code>sub-units</code>	778
<code>__spintenc_spunit_safe_exec:n</code>	618	<code>suffix</code>	1583, 1653
<code>\l__spintenc_spunit_unit_name_tl</code>	476		
<code>\l__spintenc_spusep_arg_tl</code>	52, 53, 1445		
		U	
		<code>unknown</code>	1448, 1583, 1653, 2276
		<code>unknown-abbreviation</code>	1036
		<code>unknown-choice</code>	140
		<code>unknown-command-setup</code>	91
		<code>upper-case</code>	1878
		<code>use-spnZ</code>	2276
		W	
		<code>wrap-parens</code>	1718, 1789, 2201, 2276